

KAMP 제조AI데이터셋 활용 권한

No. GB20260317-PU1769693310-302036



⚠ 제조AI데이터셋 활용 주의사항

연구/공식적으로 사용하실 때에는 꼭 아래와 같이 'KAMP 출처(reference)'를 남겨주시고, 인용 시 활용한 내용과 문서 등은 아래 이메일로 보내주시기를 바랍니다.
(E-mail : kamp@kaist.ac.kr)

국문 출처 표기 양식

중소벤처기업부, Korea AI Manufacturing Platform(KAMP), 장비이상 조기탐지 AI 데이터셋, KAIST(주)에이비에이치, (주)임픽스, 2021.12.27., www.kamp-ai.kr

영문 출처 표기 양식

Ministry of SMEs and Startups., and KAIST (Korea Advanced Institute of Science and Technology). (2021, December 27). Early detection of equipment abnormalities AI dataset. Korea AI Manufacturing Platform(KAMP). <https://www.kamp-ai.kr/>

제조AI데이터셋 개요

제조AI데이터셋명 : 장비이상 조기탐지 AI 데이터셋

업종 : 뿌리(표면처리) | **목적** : 예지보전 | **유형** : csv

알고리즘 : Sound Data와 numerical Data를 특성을 활용하여 머신러닝의 Decision Tree 딥러닝의 Long-Short Term Memory(LSTM) 적용해 설비 예지 보전 및 이상 탐지를 진행하고자 한다.

사용조건 : 콘텐츠 변경허용

제공기관 : KAIST (수행기관: (주)에이비에이치/(주)임픽스)

등록일 : 2021-12-27

이용자 정보

회원 ID : shlfyd84562

다운로드일 : 2026-03-17 16:27:22

활용목적 : 제조AI 교육 및 강의

인공지능 제조 플랫폼(KAMP, Korea AI Manufacturing Platform)에서 다운로드한 제조AI데이터셋의 저작권을 위하여 문서 고유 번호와 워터마크를 부여하여, 해당 문서의 표준을 준수하였음을 증명합니다.

운영기관 : KAIST 제조AI빅데이터센터

주소 : 대전광역시 유성구 문지로 193 KAIST 문지캠퍼스 행정동

전화번호 : 042-350-1331 | 이메일 : kamp@kaist.ac.kr

「장비이상 조기탐지 AI 데이터셋」 분석실습 가이드북



「장비이상 조기탐지 AI 데이터셋」 분석실습 가이드북



Contents

1 분석요약	04
2 분석 실습	
1. 분석 개요	05
1.1 분석 배경	05
1) 공정(설비) 개요	
2) 이슈사항(Pain point)	
1.2 분석 목표	06
1) 분석 목표	
2) 데이터 정의 및 소개	
3) 데이터 분석 기대효과	
4) 시사점(implication) 요약기술	
2. 분석실습	08
2.1 제조데이터 소개	08
1) 데이터 수집 방법	
2) 데이터 유형/구조	
3) 데이터 (품질) 전처리	
2.2 분석 모델 소개	12
1) 데이터 흐름 및 인공지능 모델 적용 흐름도	
2) AI 분석모델	
2.3 분석 체험	44
1) 필요 SW, 패키지 설치 방법 및 절차 가이드	
2) 분석 단계별 프로세스 - LSTM-AE	
3) 분석 단계별 프로세스 - Decision Tree	
[단계 ①] 데이터 준비	
[단계 ②] 데이터 탐색	
[단계 ③] 데이터 정제(전처리)	
[단계 ④] 알고리즘 선택	
[단계 ⑤] 학습, 평가 데이터 준비	
[단계 ⑥] 모델링	
[단계 ⑦] 모델 훈련	
[단계 ⑧] 모델 튜닝	
[단계 ⑨] 모델 평가 및 해석	
3. 유사 타현장의 「장비이상 조기탐지 AI 데이터셋」 분석 적용	90
부 록	
분석환경 구축을 위한 설치 가이드	91
데이터 품질 전처리 [실습 코드]	111

「장비이상 조기탐지 AI 데이터셋」 분석실습 가이드북

- 필요 SW : Python, Anaconda, Jupyter Notebook
- 필요 패키지 : keras, pandas, sklearn, matplotlib, numpy, librosa
- 분석 환경 : [CPU] AMD Ryzen 7 5800X, [Memory] 64GB,
[GPU] NVIDIA GeForce RTX 3070
- 필요 데이터 : kemp-abh-sensor-2021*.csv, Error Lot list.csv, FAN_sound*.wav

1 분석요약

No	구분		내용
1	분석 목적 (현장 이슈, 목적)		열풍건조의 공정은 제품의 수명에 장기적으로 영향을 끼치는 공정이며 공정으로 발생한 열기로 인해 작업자가 설비 이상 감사를 진행하기가 어렵다. 데이터 간의 특성을 활용해 설비 예지 보전 및 이상 탐지를 진행하고자 한다.
2	데이터셋 형태 및 수집방법		1) 데이터 수집 방법 : PLC 및 센서를 통해 실시간 데이터를 수신하여 수집된 시간을 기준으로 CSV 파일 형태로 저장한다. 열풍건조 공정 팬에 설치된 Sound Sensor를 통해 해당 공정 음성데이터를 WAV 파일 형태로 저장한다. 2) 데이터셋 파일 확장자 : CSV, WAV
3	데이터 개수 데이터셋 총량		- 데이터 개수 : 307,083개 - 데이터셋 총량 : 1.8GB
4	분석적용 알고리즘	알고리즘	- Sound Data와 numerical Data를 특성을 활용하여 머신러닝의 Decision Tree와 딥러닝의 Long-Short Term Memory(LSTM) 적용해 설비 예지 보전 및 이상 탐지를 진행하고자 한다.
		알고리즘 간략소개	- LSTM : 시퀀스 예측 문제에서 순서 의존성을 학습하면서 기존 RNN이 갖는 장기 의존성 문제를 해결할 수 있는 RNN의 개선된 알고리즘이다. - Decision Tree : 머신러닝에서 분류 또는 예측 모형으로 사용되는 지도학습 방법론으로써, 중요 변수 중심의 의사 결정 규칙을 Tree 구조로 도표화 하여 분석을 수행하며, 목표변수가 범주형인 경우와 연속형인 경우로 나눌 수 있다.
5	분석결과 및 시사점		- LSTM을 통한 시계열 분석으로 미래 데이터를 예측하여 설비 예지 보전, 설비 이상 탐지에 사용한다. - Decision Tree를 통한 데이터 분석으로 이상 탐지에 사용한다.

1. 분석 개요

1.1 분석 배경

1) 공정(설비) 개요

• 열풍건조 정의

- 도금처리 이후 피막과 안정제가 금속 표면에 안정적으로 달라붙을 수 있게 하는 공정이다.
- 탈수로 물기가 제거된 제품을 열풍건조 공정을 통과시킴으로써 표면의 물기를 완전 건조 시킬 수 있다.



[그림 1] 열풍건조

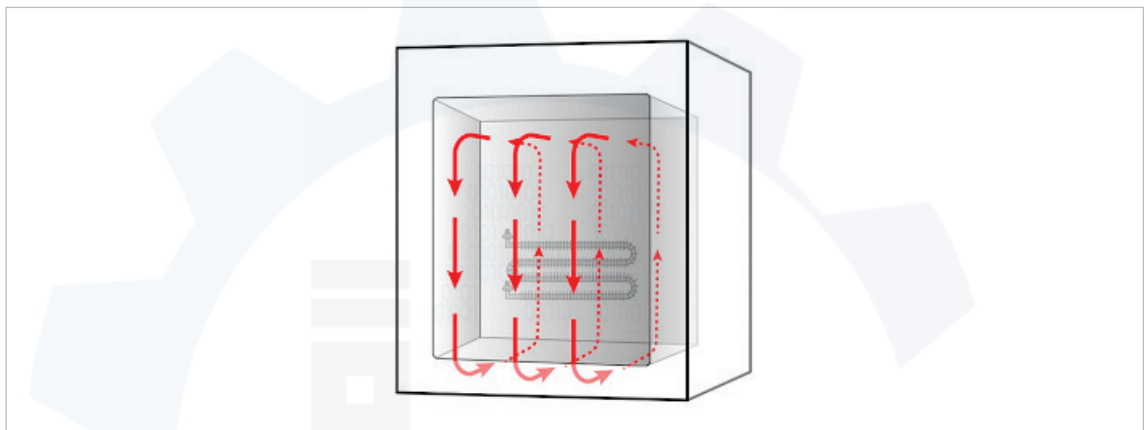
2) 이슈사항(Pain point)

• 공정(설비)상의 문제 현황

- 현재 다양한 데이터가 수집되고 있으나 활용을 못 하는 상황으로, 이는 대부분의 중소기업 제조 현장에서 공통으로 겪고 있는 것으로 판단된다.
- 열풍건조의 경우, 해당 공정 자체로 제품 품질에 경미한 수준 외의 큰 영향을 주지 않는다. 그러나 해당 설비를 가동하기 위해 상당한 전력이 소모되며, 높은 전력 소모 구조로 인해서 설비의 고장이 드물지 않게 발생한다. 설비의 고장은 생산 일정 및 품질 관리에 영향을 줄 뿐만 아니라, 고장으로 인한 과도한 전력 낭비와 함께 사고의 위험성을 내포하고 있으므로 특히 집중 관리가 되어야 하는 부분이다.
- 내부 온도 유지 또는 관리를 위해 동작하는 설비들에 있어서, 코일은 열풍 발생 및 과열 방지를 위한 모터에 동력을 공급하는 부분이다. 해당 부분이 이상으로 인해 열 저항 변화가 발생하면, 가동을 위한 표준 전력 범위를 벗어나면서 열풍 및 모터 가동이 불가능해진다. 또한, 과열 방지를 위한 모터는 사전 설정된 적정값으로 회전하면서 그 고유 진동수 및 진동 패턴을 보유하고 있다. 그러나, 위 언급된 코일 문제나 기타 문제가 발생할 경우, 해당 고유 상태가 깨지면서 이상 현상을 일으킨다.

• 문제해결 장애 요인

- 열풍건조를 진행하면서 열풍의 적정 온도를 유지할 필요가 있는데, 이는 열풍의 온도 변화에서 상대습도, 비열, 점도, 열전도도 및 열전달 계수 변화 등으로 인해서 건조 과정의 효율과 안정성이 달라지기 때문이다. 설비 내부에서는 적정 온도를 유지하기 위하여 코일 등 여러 가지 조절 장치들을 복합적으로 내재하고 있다. 그러나, 이런 조절 장치들이 제 기능을 내지 못하게 되면 일차적으로 불필요한 재화 및 에너지 낭비가 발생하며, 심층적으로는 내부 온도가 급격히 상승하여 열풍건조의 조건이 변조되며, 궁극적으로는 급격히 상승한 온도로 인해 추가적인 설비 고장 및 사고를 유발할 수 있다.



[그림 2] 열풍건조

• 극복 방안

- 다양한 데이터 분석을 통해 공정 운영 최적화, 품질 예측, 설비 예지 보전 등에 적용 가능한 것으로 판단된다. 공정에서 수집되는 데이터는 공정에서 발생하는 문제를 파악하는 데 사용될 수 있다. 또한, 해당 장치들의 주기적인 점검을 넘어서 현재 상태를 적절하게 점검하고 추후 상태를 예측할 수 있어야 한다.

1.2 분석 목표

1) 분석 목표

- 열풍건조 설비는 요구하는 가동 에너지 밀도가 높은 편으로 이로 인한 여러 고장 요인이 적지 않은 편이다. 따라서, 제품 생산 일정 및 품질 보존의 측면과 에너지 낭비 및 사고 방지의 측면에서도 열풍건조 설비에 대한 체계적인 설비 예지 보전 모델 구축은 중요하다. 이를 위해, 열풍건조 설비의 내부 온도를 비롯하여 코일의 전류, 모터의 진동 패턴 등을 파악하여 기존의 정상 상태와의 비교 분석 및 향후 설비 상태에 대한 예측 모델 구축을 목표로 한다.

2) 데이터 정의 및 소개

- 공정 데이터(온도, 전류, 진동, 시간, 이미지, 소리)를 이용한다. 또한, AI를 통해 새로운 통찰력을 찾고 설비 예지 보전에 사용할 수 있도록 분석 진행한다.

3) 데이터 분석 기대효과

- 열풍건조 데이터 분석의 경우 분체도장 및 세라믹 공정 등 다양한 공정에서도 온도, 히터, 전류 및 진동과 같은 다양한 열풍건조 공정에 데이터를 적용할 수 있을 것으로 판단된다.
- 열풍건조 공정에서 발생하는 공정 데이터와 품질 데이터 간의 상관관계 분석을 진행하고 주요 원인 인자를 분석한다. 기계 학습 알고리즘을 활용하여 열풍건조 공정에서의 설비 예지 보전 모델을 만들고자 한다.

4) 시사점(implication) 요약기술

- 공정 중 최적 처리 공정으로 공정 직후에는 육안으로 불량률 검사하기 힘든 것으로 생각된다.
- 주식회사 캠프와의 회의를 통해 진행된 회의로 공정이 장기적인 수명 관점에서 가장 큰 영향을 미치는 공정으로 판단하고 있다.
- 현재 캠프 공장에는 시각적인 품질검사를 위한 양품과 불량률 판별할 수 있는 AI 기법의 적용이 필요한 상황이다. 이에 AI로 양품과 불량률 확인할 수 있는 분석을 수행하고자 한다. 설비 이상에 영향을 줄 것으로 생각되는 공정변수를 통해 설비 이상 탐지를 위한 데이터 분석을 바탕으로 AI 분석모델을 구축하고 AI로 설비 이상 유무를 확인할 수 있는 분석을 수행하고자 한다.

• 시사점 분석

- 캠프에서 생성하는 제품은 육안으로 제품의 양품선별을 진행하지만, 사람에 의한 판단으로 품질의 균일성을 확보하기 힘들었다. 또한, 공정 데이터 수집은 되지만 데이터 분석이 되지 않고 있었으며 이를 활용할 수 있는 AI 모델을 구축하여 각 공정에서 목적한 분석을 수행한다.
- 본 데이터 분석은 현재 열악한 중소기업에 AI를 적용하여 생산성 증대, 품질 향상, 비용 절감 등의 부분에 기여할 수 있다는 점에서 시사하는 바가 크다고 판단된다.

2. 분석실습

2.1 제조데이터 소개

1) 데이터 수집 방법

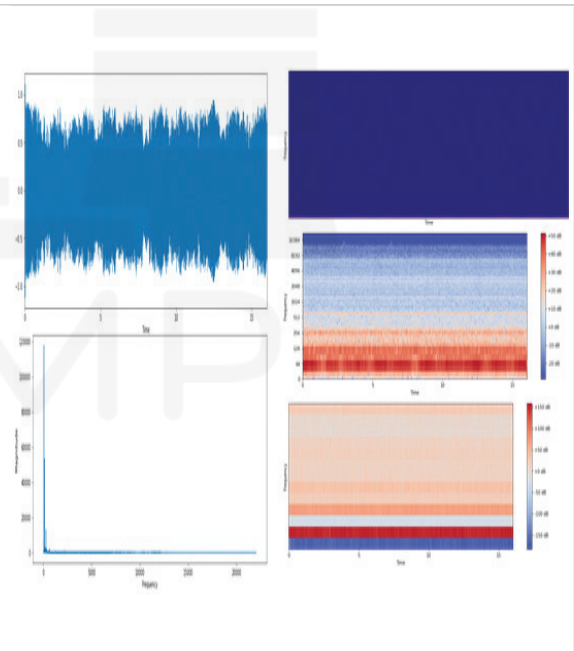
- 제조 분야 : 이노징크 전기도금
- 제조 공정명 : 베이킹
- 수집 장비 : 열풍건조 설비 내 PLC Data, HDH Data, Sound Sensor
- 수집 기간 : 2021년 09월 06일 ~ 2021년 10월 27일
- 수집 주기 : 5 sec

2) 데이터 유형/구조

- 데이터셋 구조 : 테이블 형식(Tabular), WAV
- 데이터 개수
 - Numerical Data : column수 6개, row수 51,084개, 데이터 수 306,504개
 - Sound Data : track 당 30초, 데이터 수 183개
 - Error Lot list Data : column수 12개, row수 33개, 데이터 수 396개

Index	Process	Time	Temp	Current
1	1	오후 4:23:	68.43	1.52
2	1	오후 4:23:	67.31	1.73
3	1	오후 4:23:	72.46	1.72
4	1	오후 4:23:	65.75	1.5
5	1	오후 4:23:	71.2	1.52
6	1	오후 4:23:	66.3	1.55
7	1	오후 4:23:	70.34	1.69
8	1	오후 4:23:	74.51	1.63
9	1	오후 4:23:	66.84	1.58
10	1	오후 4:23:	66.87	1.62
11	1	오후 4:24:	67.66	1.57
12	1	오후 4:24:	67.78	1.51
13	1	오후 4:24:	69.77	1.62
14	1	오후 4:24:	70.11	1.32
15	1	오후 4:24:	71.44	1.67
16	1	오후 4:24:	74.23	1.59
17	1	오후 4:24:	72.92	1.7
18	1	오후 4:24:	66.9	1.67
19	1	오후 4:24:	71.8	1.64
20	1	오후 4:24:	74.69	1.27
21	1	오후 4:24:	69.57	1.54
22	1	오후 4:24:	73.4	1.57
23	1	오후 4:25:	73.56	1.63
24	1	오후 4:25:	68.05	1.46

[그림 3] Numerical Input Data 예시



[그림 4] Sound Data 시각화 파형

• AI 데이터셋 주요 변수 정의

- Numerical Data

속성	설명	비고
Index	데이터 수집 시 자동 생성 값	int
Process	process 추적을 위해, 동일 process에 동일 숫자 부여	int
Time	측정 시 시간을 초 단위까지 기록(H:MM:SS.S)	timestamps
Temp	열풍건조 설비 내 공정 온도 측정 값	float
Current	열풍건조 설비 내 공정 전압 측정 값	float

- Error list Data

속성	설명	비고
0	0: 에러가 발생한 작업단위(lots)의 날짜	timestamps
1~11	Error Lots Number	int

• 주요 변수 기술 통계

	속성	데이터형	개수	평균	표준편차	최소값	중앙값	최대값
Input	Index	int	51,084	774.5	446.87	1	774.5	1548.0
	Process	int	51,084	22.0	12.4	1	22.0	43.0
	Time	timestamps	NaN	NaN	NaN	NaN	NaN	NaN
	Temp	float	51,084	74.259	12.291	64.007	71.19	123.094
	Current	float	51,084	1.552	0.193	0.674	1.596	1.7675
error	0	timestamps	NaN	NaN	NaN	NaN	NaN	NaN
	1	int	23	25.2	12.5	2.0	32	41
	2	int	23	25.7	12.3	1.0	28	42
	3	int	21	25.4	13.4	2.0	29	43
	4	int	13	22	10.9	3.0	21	39
	5	int	12	25.8	8.4	16.0	22	40
	6	int	9	25.5	7.6	17.0	23	41
	7	int	3	27	2.6	24.0	28	29
	8	int	3	29	1.0	28.0	29	30
	9	int	2	30	1.4	29.0	30	31
	10	int	1	30	NaN	30.0	30.0	30
	11	int	1	31	NaN	31.0	31.0	31

• 독립변수, 종속 변수 설명

구분	명칭
독립변수	Process
	Time
	Temp
	Current
종속변수	양품/불량

3) 데이터 (품질) 전처리

• 데이터 품질 전처리 목적

- 실제 공정에서 발생하는 데이터들은 값이 의미 없거나 누락 및 오타가 발생하여 품질이 떨어진다.
- 아무리 좋은 분석 방법론을 가지고 있더라도 품질이 낮은 데이터를 이용하면 좋은 결과를 얻을 수 없다.
- 그렇기에 데이터 (품질) 전처리는 데이터 분석에 있어서 필수적인 단계이다,
- 따라서 데이터들의 6가지 품질 지수를 파악하고, 데이터 전처리를 통해 품질 지수를 향상한다.

• 데이터 품질 지수

- 완전성(Completeness) : 필수항목에 누락이 없어야 한다.
- 유일성(Uniqueness) : 데이터 항목은 유일해야 하며 중복되어서는 안 된다.
- 유효성(Validity) : 데이터 항목은 정해진 데이터 유효범위 및 도메인을 충족해야 한다.
- 일관성(Consistency) : 데이터가 지켜야 할 구조, 값, 표현되는 형태가 일관되게 정의되고, 서로 일치해야 한다.
- 정확성(Accuracy) : 실제 존재하는 객체의 표현 값이 정확하게 반영이 되어야 한다.
- 무결성(Integrity) : 데이터베이스 자료의 오류 없이 변화에 영향을 받지 않고 데이터의 정확성과 일관성, 유효성이 보호되어야 한다.

• 데이터 품질 지수 세부 설명

- 완전성 품질 지수 = $(1 - (\text{결측치} / \text{전체 데이터 수})) * 100$

- ① 결측치 값이 30% 이상인 데이터들은 데이터의 완전성이 떨어지기 때문에 열별 결측치 값의 비율을 확인하여 삭제한다.

- ② 데이터의 결측치를 확인하기 위해 'isnull()' 함수를 사용한 뒤 'sum()' 함수를 이용하여 총 결측치 개수를 구한다.
- ③ 구한 결측치의 개수를 이용하여 완전성 품질 지수를 구한다.

- 유일성 품질 지수 = ((유일한 데이터 수)/전체 데이터 수) * 100

- ① 이 데이터는 유일한 값을 가지는 열이 존재하지 않으므로 유일성을 판단하지 않는다.
- ② 유일성을 판단하고 싶다면 날짜와 시간 두 열을 이용하여 합성키를 만든 뒤 기본키로 설정하여 중복을 판단하면 된다.

- 유효성 품질 지수 = (유효성 만족 데이터 수/전체 데이터 수) * 100

- ① 데이터가 유효범위 내에 있는가?
- ② 데이터 형식에 맞는가?
- ③ 수집된 날짜 안에 들어가 있는가? 등을 검증하는 것이다.

- 일관성 품질 지수 = (일관성 만족 데이터 수/전체 데이터 수) * 100

- ① 데이터 테이블 간 종속관계 확인
- ② 데이터 형식에 맞는가?
- ③ 수집된 날짜 안에 들어가 있는가? 등을 검증하는 것이다.
데이터가 지켜야 할 구조, 값, 표현되는 형태가 일관되게 정의되고, 서로 일치해야 한다.

- 정확성 품질 지수 = (1-(정확성 위배 데이터 수/전체 데이터 수)) * 100

해당 최대수요전력 예측과 관련하여 중복되는 예측값은 존재할 수 있으나 누락이 발생해서는 안된다.

- 무결성 품질 지수 = (1-(유일성, 유효성, 일관성 지수 중 100%가 아닌 지수 개수/3)) * 100

무결성 품질 지수는 고려되는 모든 데이터가 '유일성', '유효성', '일관성' 지수를 만족하는지 확인하는 지수이다. 본 가이드북에서는 세 가지 지수 중 100% 품질을 담당하는 지수의 비율을 무결성 품질 지수로 정의한다.

2.2 분석 모델 소개

1) 데이터 흐름 및 인공지능 모델 적용 흐름도



[그림 5] 데이터 흐름 및 인공지능 모델 적용 흐름도

- 결측치, 이상치 제거

*[3] 데이터 정제 - 데이터 특성을 기반으로 이상치를 수정 혹은 제거 파트에서 확인에서 자세하게 설명 진행

- 고품질 이미지 데이터 세트를 만들기 위한 변환 및 특징(feature) 추출 방안

- Sound 데이터 같은 경우 음성데이터를 시각화하여 형태 분포 파형을 확인한 후 특징을 추출하는 작업을 진행한다.
- 추가로 음성에서 추출한 데이터의 특징을 데이터프레임에 쌓아 Decision Tree를 이용해서 이상을 탐지할 것이다.

2) AI 분석모델

- AI 분석모델로 사용하게 될 모델은 LSTM(Long Short Term Memory)이다. 간단하게 설명하자면 RNN이 시간에 따른 학습이 길어질수록 입력으로 받은 정보가 학습에 끼치는 영향이 점점 감소하다가 사라지는 이슈를 해결하기 위해 고안된 모델이다. LSTM은 시계열 데이터 영역에서 높은 정밀도를 보임에 따라 다양한 분야에서 활용되고 있으며 더 오래 시간 정보를 기억하고 다양한 시계열 예측 작업에서 놀라운 성과를 띄고 있다.

- 해당 AI 방법론(알고리즘) 선정 이유 기술

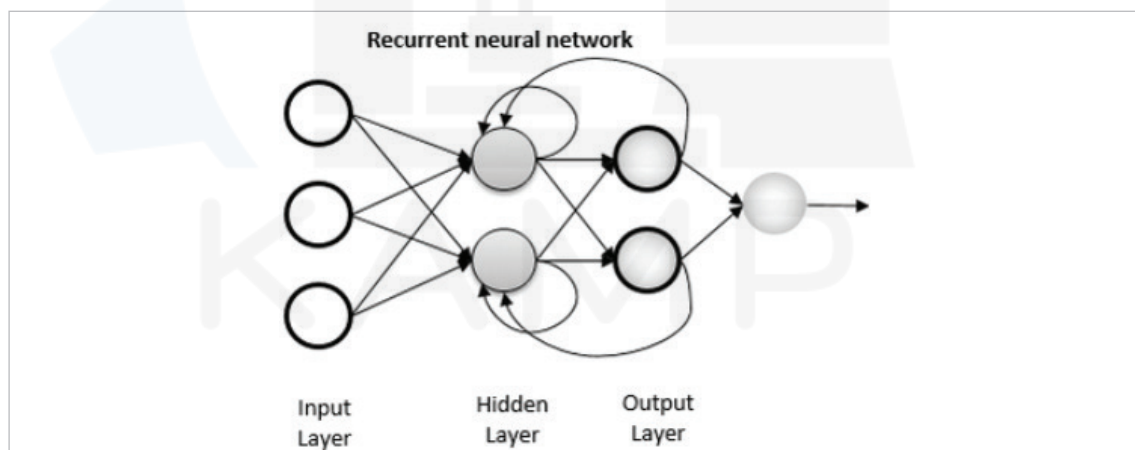
- 수집된 데이터는 시계열성을 띄고 있다. 그러므로 RNN보다는 LSTM을 채택하였다.
- RNN을 이용하지 않은 이유는 아래와 같다.
 - 먼저 RNN은 글, 계측데이터, 로그 데이터, 음성신호 등 배열 데이터, 즉, 시간의 흐름에 따라 변하는 데이터에서 패턴을 인식하는 모델이다.
 - RNN의 경우 과거 시간 단계(Time step)의 은닉 상태(Hidden state)와 현 시간 단계의 시스템

입력 값을 새로운 은닉 상태를 계산하는데 사용하는 구조적 특징을 가지고 있다. 그러므로 RNN은 비교적 짧은 Sequence(data 특징)에 대해서만 효과를 보인다. 그 말은 데이터가 Time step 길어질수록 데이터가 올바르게 전달 되지 못하는 현상이 발생한다는 것이다. 학습 과정에서 입력의 길이가 길어질수록 오래된 입력값을 잊어버리게 된다. 이러한 단점 때문에 RNN보다는 LSTM을 채택하게 되었다.

- 데이터가 추출되는 환경이나 현장 상태 따라 영향을 받을 것으로 판단이 되어 유사한 환경인 과거 데이터를 기반으로 예측 성능을 높이는데 긍정적인 영향을 미칠 것으로 판단하였고 LSTM은 앞서 말한 것처럼 RNN의 단점을 보완한 모델로 정보가 소실되는 현상을 해결한 모델이기 때문이다.
- Sound Data의 AI 분석모델로 사용하게 될 모델은 Decision Tree이다.

• 적용하고자 하는 AI 분석 방법론(알고리즘)의 구체적 소개

- Long-Short Term Memory(LSTM)는 실제 뇌의 기억 구조인 Short-term memory plasticity와 Long-term memory Transition에서 그 기본적인 구조를 차용했으며, Algorithm 측면에서는 Recurrent Neural Network(RNN)의 확장 및 개선된 형식이다.
- 우선 기존의 Neural Network가 Feed-forward로 단일 방향으로 전파되는 것과 달리, Recurrent Neural Network(RNN)는 계산 결과가 다시 스스로에게 반영되는 회귀(Recurrent) 방향도 보유하고 있는 Network 구조이다.



[그림 6] Recurrent Neural Network

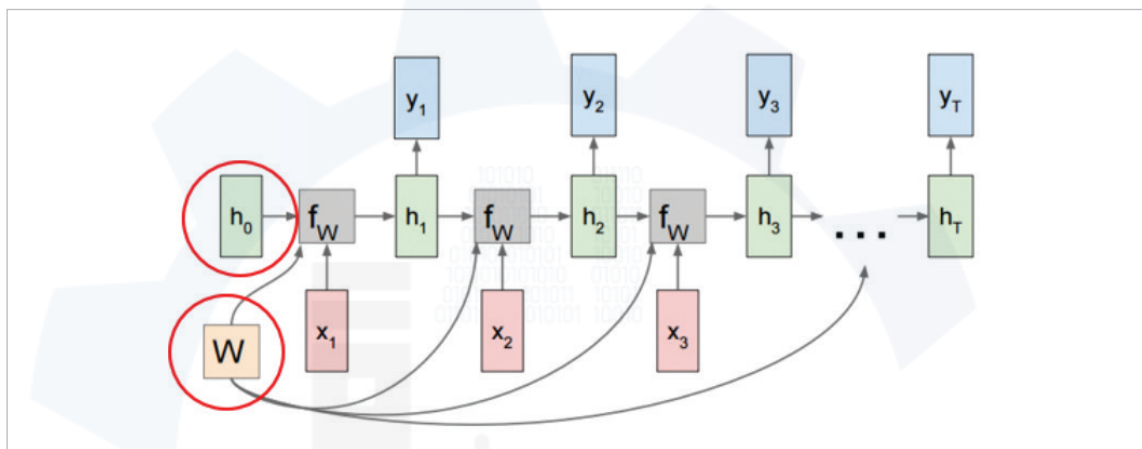
- 따라서, RNN은 기존의 DNN과 달리 layer에 입력되는 input value가 2종류이다. l -th layer를 기준으로, $(l-1)$ -th layer의 정보 또는 1st부터 $(l-1)$ -th layer까지의 모든 정보를 보유하고 있는(임시 저장) 부분을 'cell'이라고 지칭하며, 현재 l -th layer의 정보까지 포함하여 $(l+1)$ -th layer에 전송될 정보 전체를 'hidden state'라고 지칭한다.

- 이전 layer(cell)에서 넘어온 hidden state, 현재 layer에 입력되는 정보, 현재 layer에서의 affine operation까지 여러 차원의 값들이 움직이므로, RNN에서 해당 연산을 수행하기 위한 weight도 여러 종류로 분리되어 관리된다. RNN의 기본적인 연산 구조는 다음과 같다.

$$\text{Hidden state } h^{(l)} = f(h^{(l-1)}, x^{(l)}) = W_{hh}^{(l)} h^{(l-1)} + W_{xh}^{(l)} x^{(l)} + b^{(l)}$$

$$\text{Activation } z^{(l)} = \tanh(h^{(l)}) = \tanh(W_{hh}^{(l)} h^{(l-1)} + W_{xh}^{(l)} x^{(l)} + b^{(l)})$$

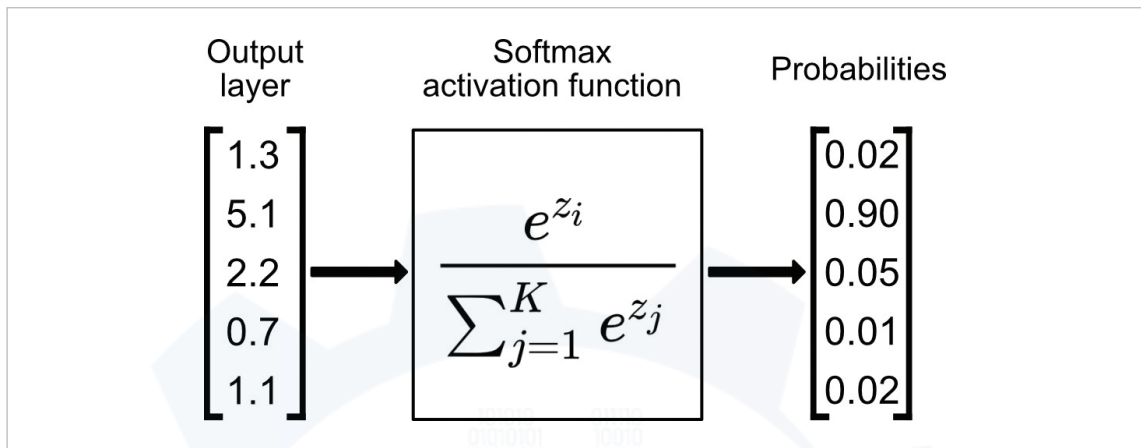
$$\text{Output value } y^{(l)} = W_{zy}^{(l)} z^{(l)} = W_{zy}^{(l)} \tanh(h^{(l)})$$



[그림 7] RNN cell & hidden state

- 여기서 Activation Function이란 생물학적 신경 구조에서 ‘역치’에 대응되는 개념으로, 사전 지정된 임계값(Threshold)을 기준으로 다음 Layer에 연산 결과를 전파할지 또는 얼마나 가중되게 전파할지를 결정한다. Activation Function으로는 sigmoid, hyperbolic tangent(tanh), ReLu 등이 가장 보편적으로 사용되며, RNN에서는 특별한 이유 없이 관례적으로 tanh를 가장 많이 활용한다.
- Output value를 연산 및 출력함에 있어서, Matrix operation을 수행하기도 하지만, soft-max 연산을 수행하기도 한다. Soft-Max 연산은 기존의 DNN이나 CNN 구조에서는 모든 layer를 지난 후 최종 출력값을 산출하기 직전에 적용되지만, RNN에서는 각 단계에서의 출력값을 산출할 때마다 적용된다. 이는 기존 구조에서는 data 전체가 input-layer에서 한 번에 입력되고, 최종 결과값이 가장 마지막 layer에서 유일하게 도출되는 것과 달리, RNN은 data의 입력부터 한 번에 입력되는 것이 아니라 각 layer 별로 입력되고 각 layer의 연산은 이전 layer의 연산 값과 새로 입력된 input value를 둘 다 활용하면서 진행된다. 따라서, output 역시 단순히 어떤 affine operation의 연산 값이 아니라, input value에 대응되는 유의미한 정보

를 산출할 필요가 있으므로 soft-max가 개별적으로 적용된다. soft-max를 거친 출력값은 Logit이라고 부르기도 하며, 각 class별 출력값을 전체 출력값으로 나누어서 Normalization을 수행한 값이다. 즉, 전체 logit의 합을 1로 조절하여 각 class별 확률(probability)을 구현하는 기능을 담당하고 있다.



[그림 8] Soft-Max

- 각 layer별 output이 모두 도출되고 나면, Training Data에 사전 입력된 Labeled Data와 비교하여 Error를 계산할 수 있다. 이렇게 Error를 계산하는 함수를 Loss Function이라고도 하며, Deep Learning 분야에서는 주로 Square loss와 Entropy loss를 사용한다. 총 n개의 Training Output Data y_k 와 Labeled Data t_k 에 대하여, Square loss와 Entropy loss는 각각 다음과 같다.

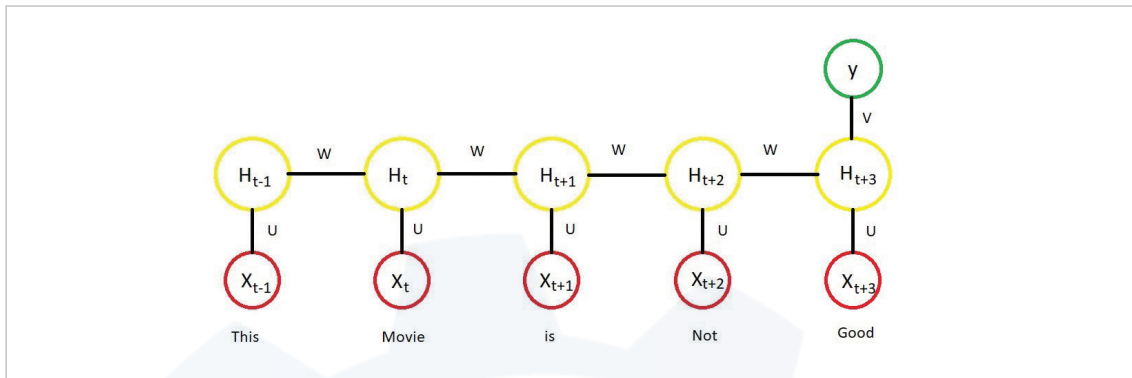
$$\text{Square loss } E(y, t) = \frac{1}{2} \sum_{k=1}^n |t_k - y_k| \text{ with } \sum_{k=1}^n y_k = 1$$

$$\text{Entropy loss } E(y, t) = - \sum_{k=1}^n t_k \log \left(\frac{\exp[y_k]}{\sum_{j=1}^n \exp[y_j]} \right)$$

Recurrent Neural Network는 위에 제시된 Loss Function을 최소화하기 위해서 내부 parameter를 조율하며, 여기서 조율 대상은 각 연산에 적용되는 matrix set $W_{hh}^{(l)}, W_{xh}^{(l)}, W_{zy}^{(l)}$ 와 bias vector $b^{(l)}$ 이다.

- 순환 구조(Recurrent Structure)를 갖는다는 점, 각 layer에서 input, output이 개별적으로 존재한다는 점, 앞서의 누적된 정보가 현재 layer 연산에 개입한다는 점은 기존의 DNN, CNN과는 명확히 다른 특성이다. 즉, RNN은 독특한 구조만큼 처리하는 데이터의 특성도 명확히 구분된다. DNN이 numerical chart data나 간단한 이미지, CNN이 image processing에 특화된 것과 달리, RNN은 연속된 데이터나 인과 관계

가 분명한 데이터에 특화되어 있다. 즉, 언어나 음악 등과 같이 특정 시점의 근방에서 데이터끼리 서로 연관성이나 pattern이 있거나, 오랜 시간에 걸쳐서 누적된 데이터(시계열 데이터)에 매우 유리하다.



[그림 9] Natural Language Process(NLP)

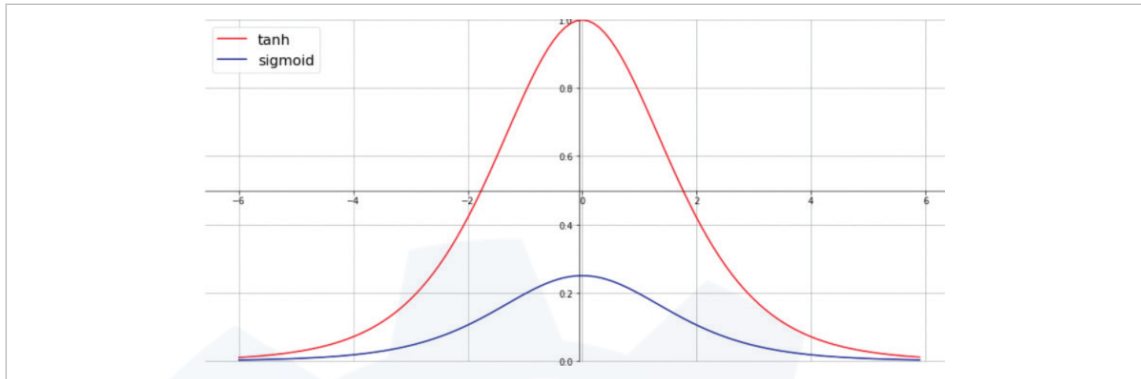
- 그러나 RNN은 그 자체로 치명적인 구조적 결함을 내포하고 있다. Matrix set $w_{hh}^{(l)}, w_{xh}^{(l)}, w_{zy}^{(l)}$ 와 bias vector $b^{(l)}$ 를 조율하기 위해서는 Jaccobian Matrix(Partial Derivate system)을 기반으로 한 최적화 과정, 즉, 역전파(Back-propagation) 과정을 수행해야 한다.

$$\begin{aligned} & \frac{\partial}{\partial W^{(l)}} F((w^{(L)}, b^{(L)}), \dots, (w^{(1)}, b^{(1)})) \\ &= [h^{(k)} \circ A^{(k)} \circ \dots \circ h^{(1)} \circ A^{(1)}]^T \cdot [DS(W^{(L+1)})^T Dh^{(L)}(W^{(L)})^T \dots Dh^{(k)}(x^{(k)})] \\ & \frac{\partial}{\partial b^{(l)}} F((w^{(L)}, b^{(L)}), \dots, (w^{(1)}, b^{(1)})) = [DS(W^{(L+1)})^T Dh^{(L)}(W^{(L)})^T \dots Dh^{(k)}(x^{(k)})] \end{aligned}$$

RNN에서 주로 활용되는 Activation Function은 hyperbolic tangent이며, (-1,1)에 유계(bounded)되어 있다. 또한, 그 미분형은 (0,1)에 유계되어 있다. 따라서, 기본형과 미분형 둘 다 연산이 반복될수록 값이 0으로 수렴(convergence)하게 된다. 즉, cell에 저장된 hidden state는 뒤쪽 layer로 갈수록 영향력이 작아지며(정보 손실 발생), 반대로 gradient descent를 활용한 역전파에 있어서도 제대로 최적화가 이루어지지 않아서, parameter가 개선되지 못한다. 이러한 현상을 Vanishing Gradient Problem이라고 하며, computing power등의 문제가 아니라 순수히 여러 방향에서 input을 유기적으로 활용하기 위한 RNN 그 자체의 mathematical structure에서 발생하는 결함(defect)으로 본질적 결함(essential defect)에 해당한다. 또한, hyperbolic tangent가 아닌 다른 activation function을 차용해도 문제의 본질은 달라지지 않는다.

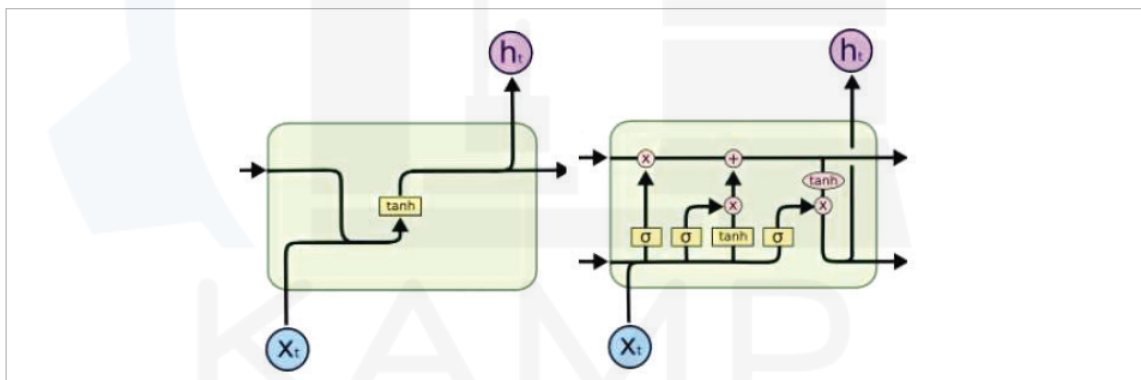
$$\tanh: h(x) = \tanh x = \frac{\sinh x}{\cosh x} = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

$$\rightarrow \frac{dh(x)}{dx} = 1 - \left(\frac{e^x - e^{-x}}{e^x + e^{-x}} \right)^2 = 1 - [h(x)]^2 = (1 - h(x))(1 + h(x))$$



[그림 10] Sigmoid & tanh Derivate

- 이러한 RNN의 한계성을 극복하기 위해 개선된 모델이 Long-Short Term Memory(LSTM)이다. 앞에서 언급한 대로, LSTM은 단기 기억 특성인 short-term memory plasticity와 장기 기억 특성인 long-term memory transition을 모두 활용한다.

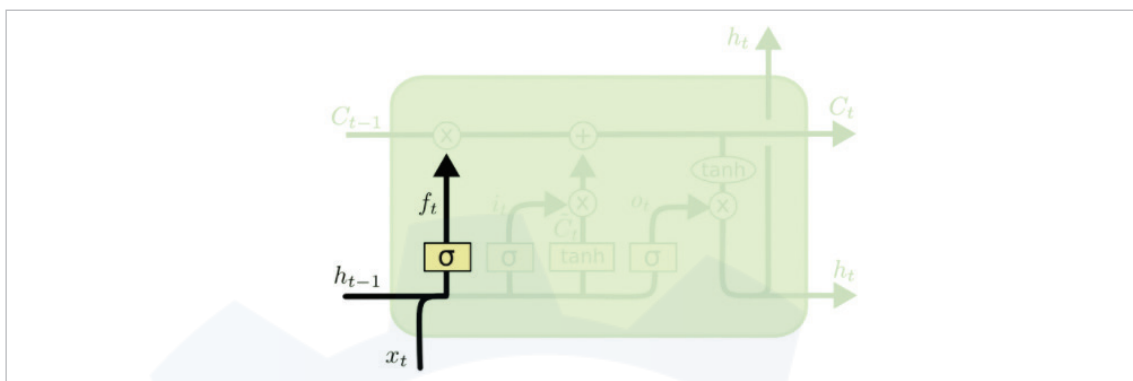


[그림 11] RNN vs LSTM

- LSTM 구조에서 위쪽의 선은 모든 layer를 관통하며 cell state 또는 belt라고 부른다. 해당 구조체는 전체 흐름을 관리한다. 각 layer에서는 gate들을 통해서 cell state에 일부 정보가 추가될 수 있다. cell state 아래쪽에서 cell state에 연결된 부분들이 gate로 각 layer 별로 3개씩 포함되어 있다.

- 첫 번째 gate는 단순한 affine operation과 sigmoid(activation function)로 구성된다. 해당 gate는 이전 layer들에서 들어온 정보 중 폐기할 정보를 결정하는 곳으로 'forget gate layer'라고 불리기도 한다.

$$f^{(l)} = \sigma(W_f^{(l)} \cdot [h^{(l-1)}, x^{(l)}] + b_f^{(l)})$$



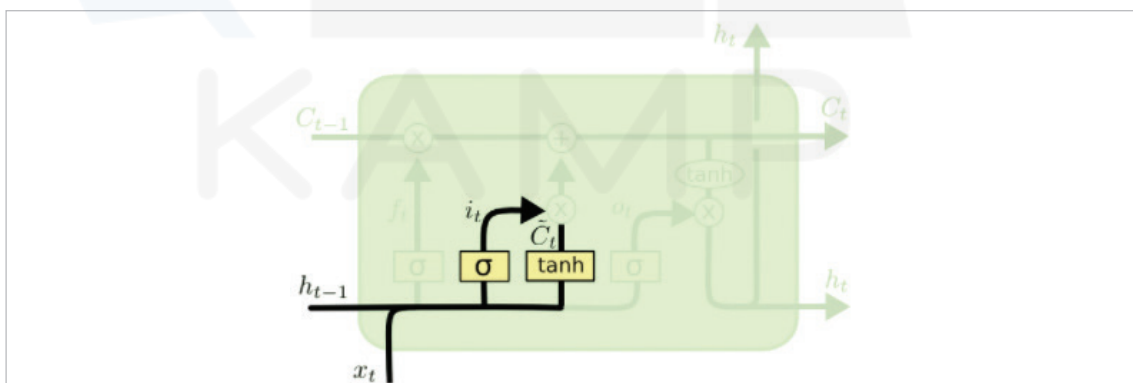
[그림 12] Forget gate Layer

- 두 번째 gate는 이전 layer에서 남은 정보들과 해당 layer에서 새로 input이 되는 정보들을 조합하여 어떤 정보를 cell state에 저장할 것인지를 결정하는 'input gate layer'이다. 즉, 해당 단계까지 진행되면 cell state update가 발생한다.

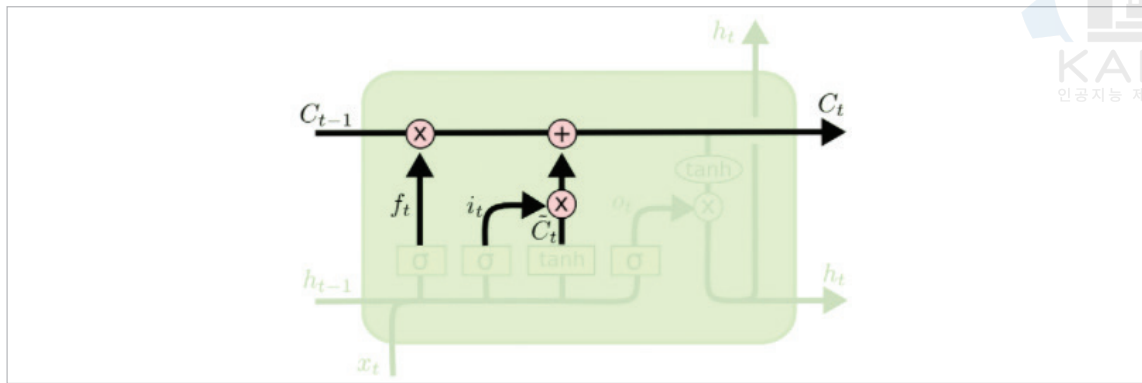
$$i^{(l)} = \sigma(W_i^{(l)} \cdot [h^{(l-1)}, x^{(l)}] + b_i^{(l)})$$

$$\tilde{C}_t = \tanh(W_C^{(l)} \cdot [h^{(l-1)}, x^{(l)}] + b_C^{(l)})$$

$$\text{cell state update: } C^{(l)} = f^{(l)} \cdot C^{(l-1)} + i^{(l)} \cdot \tilde{C}^{(l)}$$



[그림 13] Input gate Layer

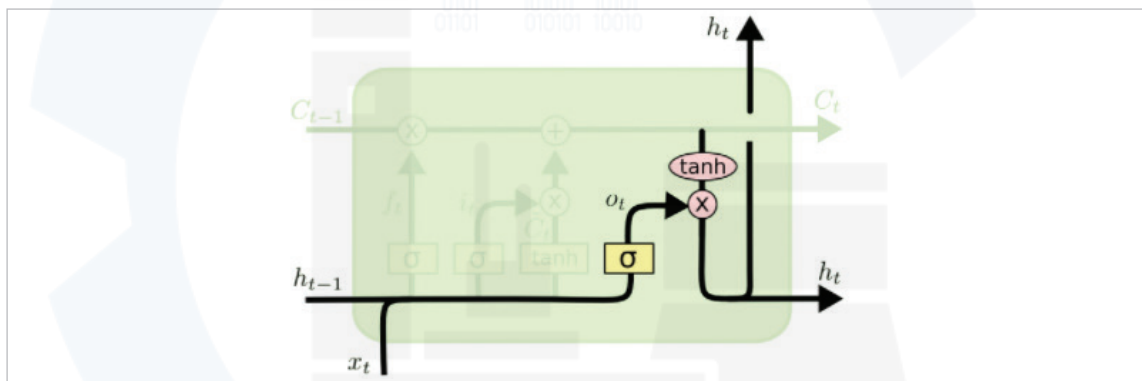


[그림 14] Cell state update

- 마지막 세 번째 gate는 진행된 연산 결과를 output으로 내보내고, 다음 layer로 전파하는 ‘output gate layer’이다. 해당 부분은 기존의 RNN과 크게 다르지 않다.

$$out^{(l)} = \sigma(W_o^{(l)} \cdot [h^{(l-1)}, x^{(l)}] + b_o^{(l)})$$

$$h^{(l)} = out^{(l)} \cdot \tanh(C^{(l)})$$

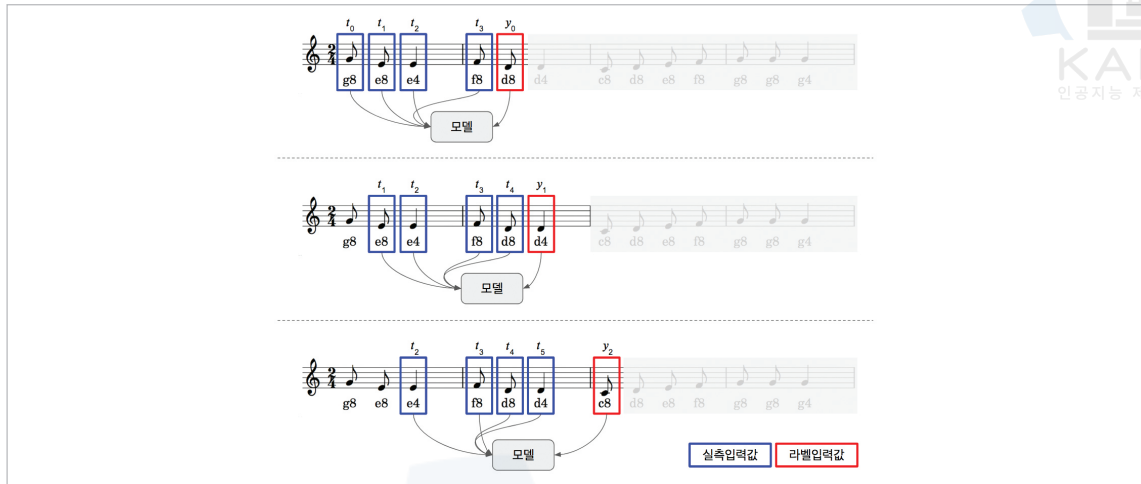


[그림 15] Output gate Layer

- LSTM이 개발된 이후, 좀 더 개선된 모델들이 지속적으로 제시되고 있으며 가장 대표적으로 ‘peephole connection’이나 ‘Gated Recurrent Unit(GRU)’등이 있다.

• LSTM의 장점 그리고 단점

- LSTM의 장점은 기본적으로 RNN의 장점을 기본적으로 내포한다. RNN과 동일하게 LSTM은 시계열 데이터, 인과 관계 분석에 매우 유리하다. 따라서, 악보 패턴 인식 및 예측, 자연어 처리(NLP), 데이터 이상 패턴 예측 및 감지 등에 매우 유용하며, 개선 모델들을 모두 LSTM으로 크게 묶으면 사실상 상기 언급된 분야의 유일한 학습법이기도 하다. 또한, RNN의 치명적인 단점을 극복하기 위해 제시된 모델인만큼, RNN과 달리 데이터의 길이나 layer의 길이로 인한 정보 손실이나 Gradient Vanishing이 없다.

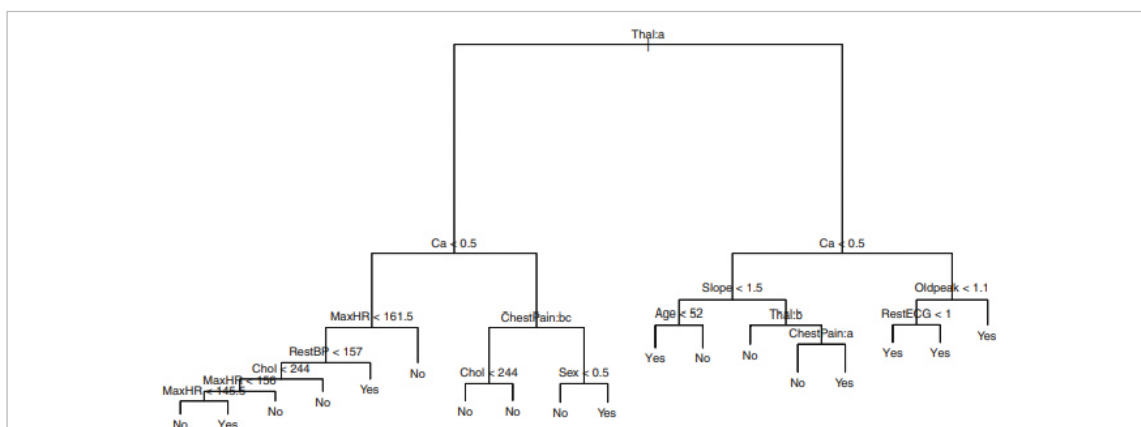


[그림 16] LSTM 동작 원리

- 그러나 기본적으로 LSTM은 실제 신경 반응의 아주 복잡한 반응을 구현한만큼 기존 DNN, CNN, RNN 등과 비교하여 수학적으로도 코드상으로도 그 구조가 매우 어려운 편이다. 따라서, 모델을 다루기가 다소 까다로우며 그 구조를 개선하기 위해 분석하고 변형하여 적용하기가 매우 어렵다. 또한, 복잡한 구조를 가지고 있는만큼 상당히 높은 computational cost와 computing power를 요구하며, 당연히 process time이 긴 편에 속한다.
- 음성 데이터의 경우 의사결정나무(Decision Tree)를 사용하는데 의사결정나무를 구체적으로 설명하자면 아래와 같다.

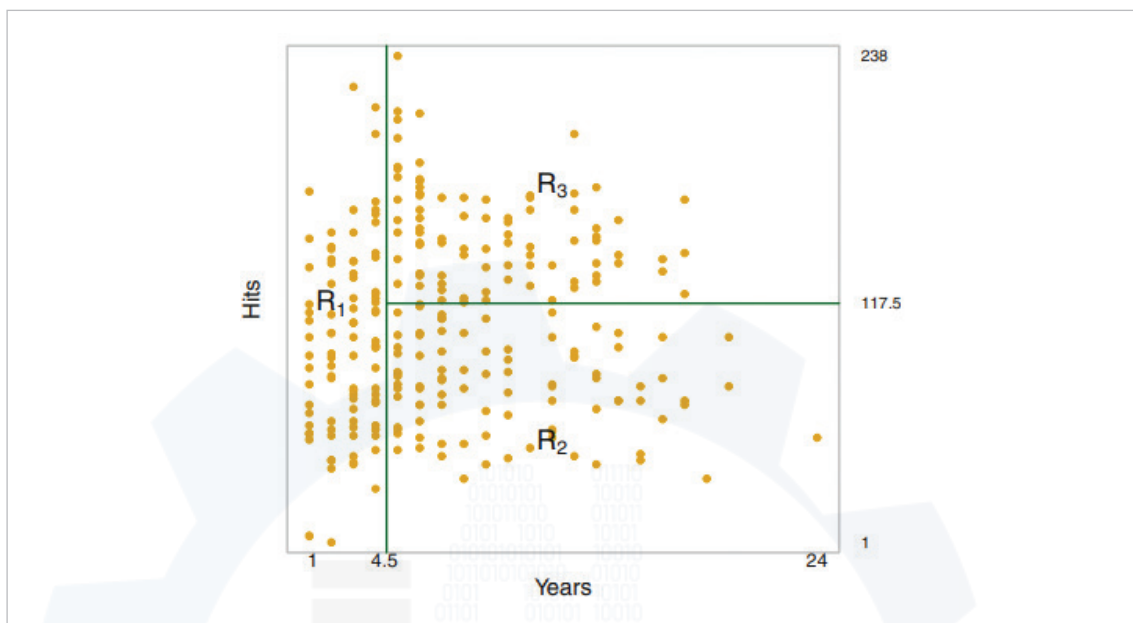
• 의사결정나무(Decision Tree)

- Decision Tree는 학습이 전개되었을 때, 그 모양이 나무가 뒤집힌 형태와 유사하다고 하여 Tree Based Method 또는 Decision Tree라 불린다. 또한, 이렇게 나무의 형태로 가시화가 된다는 점은 데이터 구조를 직관적으로 살펴볼 수 있다는 것으로 Decision Tree의 가장 큰 장점 중 하나이다.



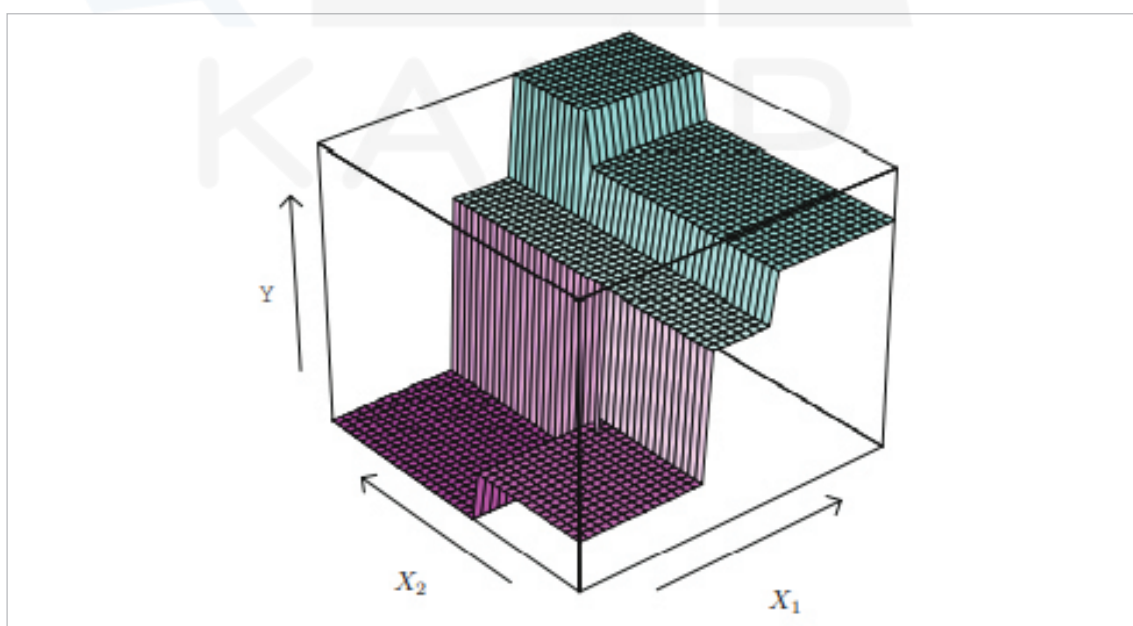
[그림 17] Decision Tree Visualization

- 가지의 맨 끝에 해당하는 부분은 Terminal node 또는 leaf라고 불리며, 중간에 있는 분기점은 Internal node 또는 branch라고 불린다. 또한, Decision Tree는 전체에서 세분화시키면서 전개가 되므로 Top-down 방식에 해당한다.



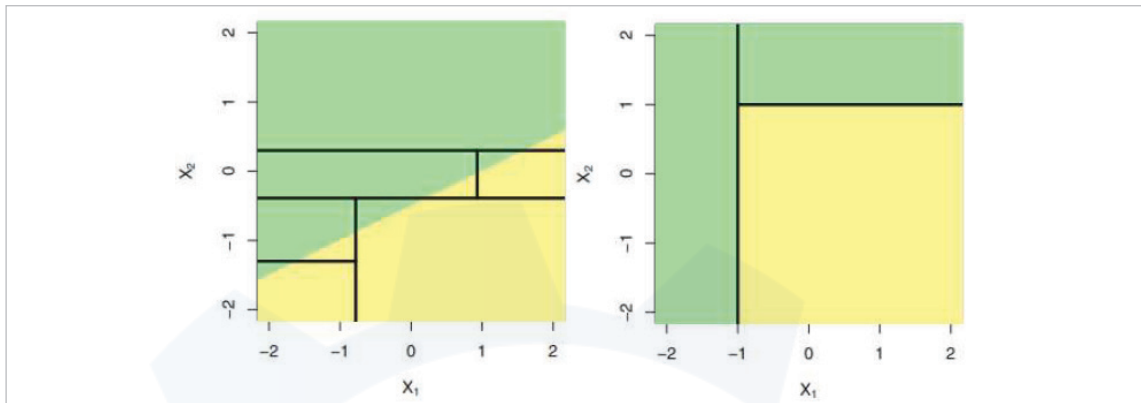
[그림 18] Domain Separation

- Data가 분포한 Feature space에서 살펴보면, 각각의 branch는 주어진 space를 분할(seperation)하는 경계(boundary)에 해당하며, 각각의 leaf는 분할된 영역(Region)에 해당한다. 또한, 이를 고차원에서 보면 아래 그림과 같이 곡면(Surface)의 계단형 근사(Step function Approximation)에 해당한다.



[그림 19] Surface Approximation

- Decision Tree는 Regression과 Classification 양쪽 모두에 사용 가능하지만, 분기점을 통해 영역을 나눈다는 특성에 주목하여 classification 방향으로 사용되는 쪽이 더 선호된다. 그러나, 어느 측면으로 활용되든 True-Value의 분포가 직교 구조(Orthogonal Structure)에 근접할수록 성능이 잘 나온다는 한계점은 존재한다.



[그림 20] Incline Domain vs Orthogonal Domain

- Decision Tree는 영역을 분할한 후 각 영역 별로 데이터가 분포하므로, 검증에 있어서도 각 영역별로 집계 후 전체 영역이 합산된다. Regression Tree의 경우, 각 영역을 R_j ($j=1,2,\dots,J$), True-value를 y_i , Prediction-value를 y_{R_j} 라 하면 전체 성능 평가는 다음과 같다.

$$\min \sum_{j=1}^J \sum_{i \in R_j} (y_i - \hat{y}_{R_j})^2$$

- Classification Tree의 경우에도 분할된 영역에 대해서, 각각 분류해야 할 Category Error를 고려해야 한다. $\hat{p}_{mk}$ 를 m 번 영역에서의 k -th class가 차지하는 Training Data 비율이라고 하면, 가장 기본적으로 사용되는 Classification Tree Error는 다음과 같다.

$$E = 1 - \max_k [\hat{p}_{mk}]$$

해당 표현을 기반으로, Tree의 크기에 대한 민감성(Sensitivity)를 높인 형태로는 Gini Index와 Entropy가 있다.

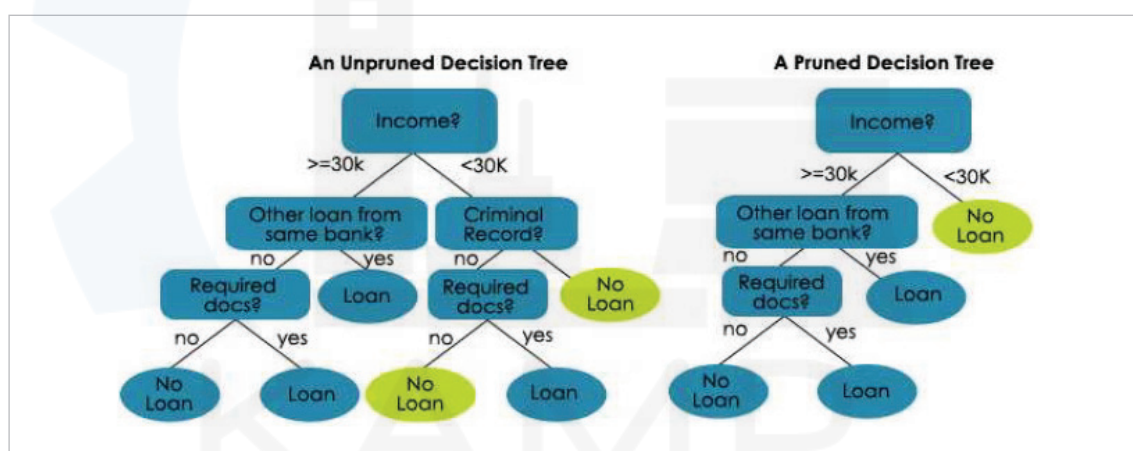
$$\text{Gini Index } G = \sum_{k=1}^K \hat{p}_{mk} (1 - \hat{p}_{mk})$$

$$\text{Entropy } D = - \sum_{k=1}^K \hat{p}_{mk} \log \hat{p}_{mk}$$

- 모델의 성능을 높이기 위해서는, 분기점의 수가 많아져야 한다. 그러나 분기점을 최대한으로 높이면 각 데이터가 하나의 Terminal Node를 형성하게 되며, 이는 궁극적으로 과적합(Over-fitting) 오류를 일으키게 된다. 따라서, 나무의 크기가 무한정 커지지 않도록 그 크기에 대한 상계 또는 penalty term이 있어야 한다. Decision Tree에서 해당 과정을 Tree Pruning이라고 하며, Terminal node의 수를 라 할 때, 최소화해야 하는 목표 함수(objective funtion)는 다음과 같다.

$$\min [\sum_{j=1}^{|T|} \sum_{i \in R_j} (y_i - \hat{y}_{R_j})^2 + \alpha |T|]$$

- 위에 제시된 계산 식에서, 앞쪽의 기준에 제시되었던 항은 Loss Term 또는 Fidelity Term이라고 하며 오류를 최소화하기 위해서 Terminal node $|T|$ 를 늘리고자 한다. 뒤쪽에 추가된 항은 Penalty Term 또는 Regularization Term이라고 하며, 전체 목표 함수 최소화를 위해서 Terminal node $|T|$ 를 줄이고자 한다. α 는 Tuning parameter로 Cross-Validation을 통해서 결정한다.



[그림 21] Decision Tree Pruning

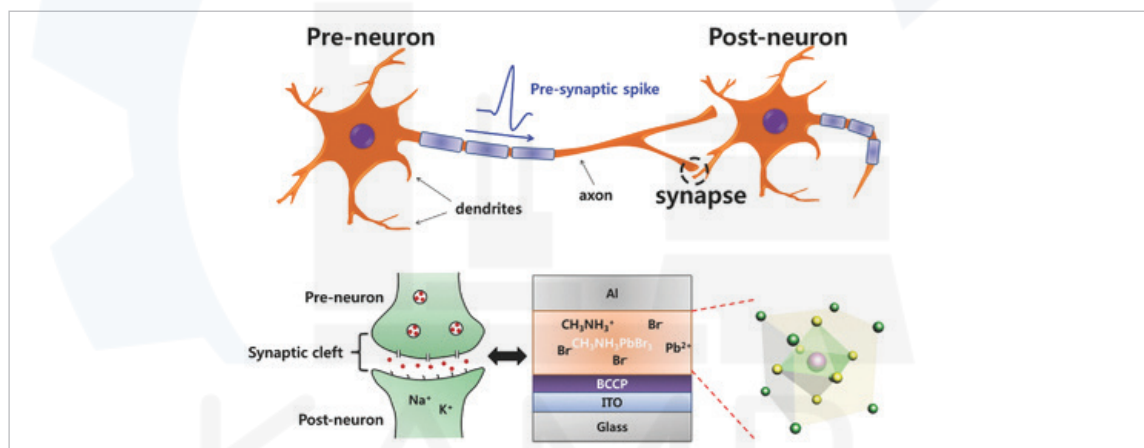
• Decision Tree의 장/단점

- Decision Tree는 가시성이 매우 뛰어나기 때문에, Tree 구조를 직접 보여주면서 설명하기가 매우 좋다. 또한, 분기점을 두고 기준에 맞추어 데이터를 나누어가는 과정은 인간의 인지구조 및 결정과정과 매우 유사하기 때문에, Decision Tree는 직관적으로 이해하고 받아들이기 매우 유리하다. 끝으로, 분기점을 가지고 있기 때문에 Network 방식을 제외한 기존의 학습법들과 달리 추가적인 Dummy Variable없이 바로 학습을 수행할 수 있다.

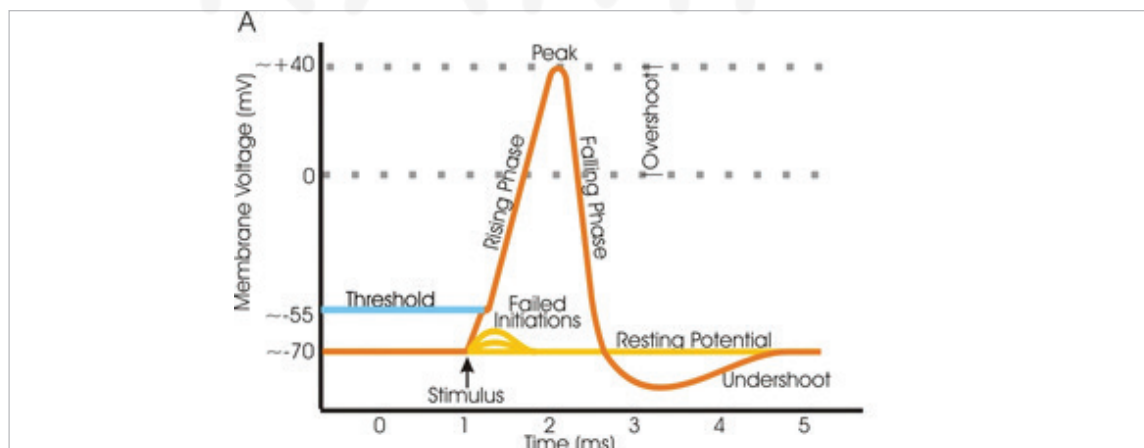
- 그러나, 단일 Decision Tree는 그 성능 자체에는 상당한 한계가 존재한다. 단일 Tree는 Regression 및 Classification에 있어서 기존에 존재하는 거의 모든 학습 방식보다 그 성능이 뒤쳐진다(즉, 오류율이 높다). 또한, 그 구조가 매우 취약해서 데이터의 작은 변화에도 전체 Tree가 상당히 흔들리기 쉬운 High-variance Structure이다. 따라서 이를 극복하기 위해 가시성을 일부 포기하고 성능을 높이는 기법들이 있으며, 각각 Bagging, Random Forest, Boosting이라고 불린다.

• Activation Function

- Activation Function은 기본적으로 생물의 Neural Network에 있어서 ‘역치 (Threshold Potential)’의 개념을 그대로 차용한 개념이다. Pre-neuron의 여러 dendrites에서 입력된 signal들이 신호 처리(Affine matrix operation)가 이루어진 후, Axon을 지나갈 때 Threshold potential을 넘지 못하는 signal은 삭제되고, 그 이상의 signal만 firing(activation)된다. 이후, activation된 signal은 post-neuron의 dendrite에 입력된다.



[그림 22] Neuron & Synapse Structure



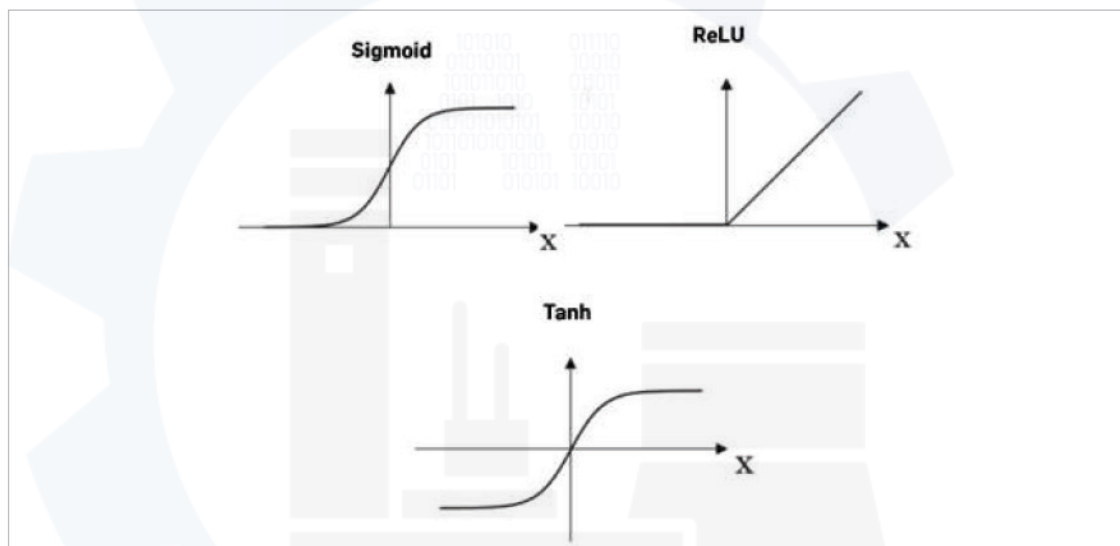
[그림 23] Threshold potential & Firing

- Activation Function은 이러한 생물학적 Neural system(synapse structure)를 그대로 따르며, 보통 0을 기준 임계값(Threshold)로 하고 그 미만의 값은 제거하여 다음 Layer 층으로 전달하지 않는다. Activation Function으로 활용되는 함수는 매우 다양하며, 가장 대표적인 것들로는 Sigmoid function, Rectified Linear Unit(ReLU), Tangent Hyperbolic function(tanh) 등이 있다.

$$\text{Sigmoid: } h(x) = \frac{1}{1+e^{-x}}$$

$$\text{LeRU: } h(x) = \begin{cases} x & \cdots \text{ if } x > 0 \\ 0 & \cdots \text{ if } x \leq 0 \end{cases}$$

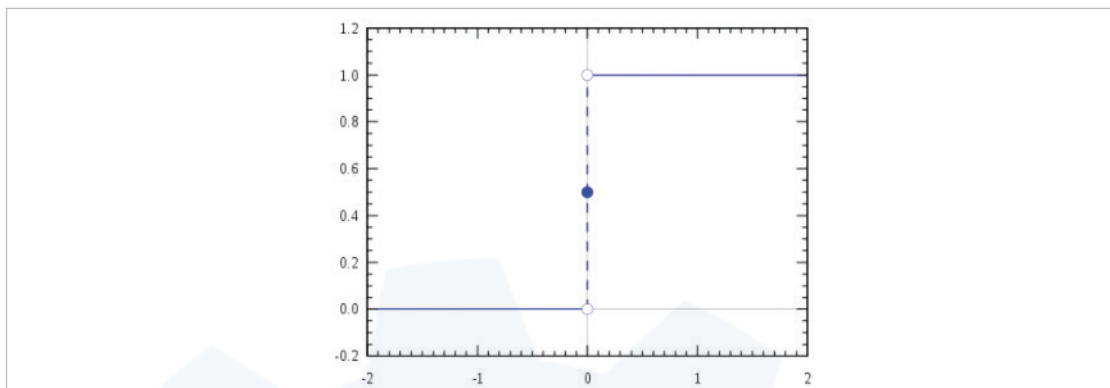
$$\text{tanh: } h(x) = \tanh x = \frac{\sinh x}{\cosh x} = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$



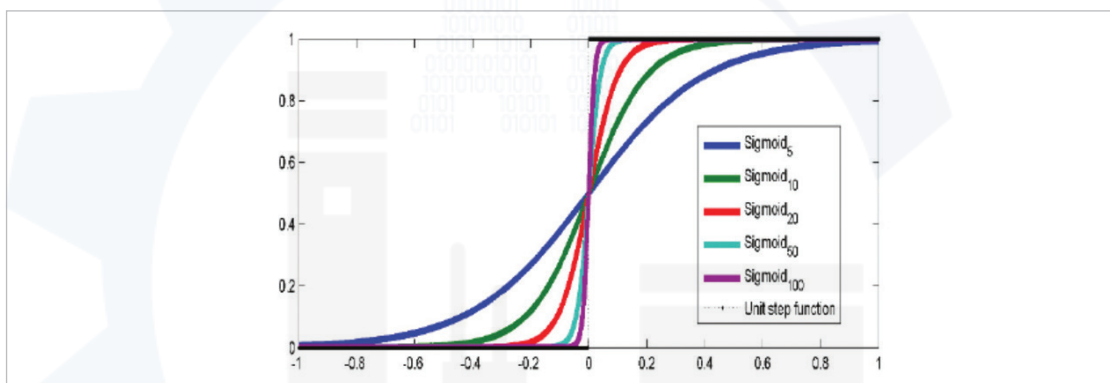
[그림 24] Activation Function

- 제시된 함수들 이외에도 Leaky ReLU, ELU, Maxout 등 다양한 함수를 사용할 수 있다. Activation Function은 임계점(Threshold) 근방(neighborhood)에서 함수값이 크게 차이가 나는 것이 바람직하다. 또한, 보다 바람직하게는 그 함수의 범위가 유계(bounded)되어 있는 것이 좋다. 예를 들어, sigmoid는 (0,1)에 유계되어 있으며, Tanh은 (-1,1)에 유계되어 있다.
- 임계점 근방에서의 특성이나, 유계적 성질을 고려했을 때 가장 우수한 성능을 보이는 함수는 step-function이다.
- 그러나, step-function은 측도 공간(Measurement space)이 이산 공간(Discrete space)이기 때문에, 미분 불능 함수(indifferentiable function)이다. 따라서, step-function을 연속 공간(Continuous space)로 근사한 Sigmoid function이 가장

먼저 Activation Function으로 정립되었다. 또한, step-function의 적분형이 곧 ReLU(즉, ReLU의 미분형은 step-function)이기 때문에, ReLU가 정립된 후로는 가장 많이 사용되는 Activation Function이다.



[그림 25] Step Function



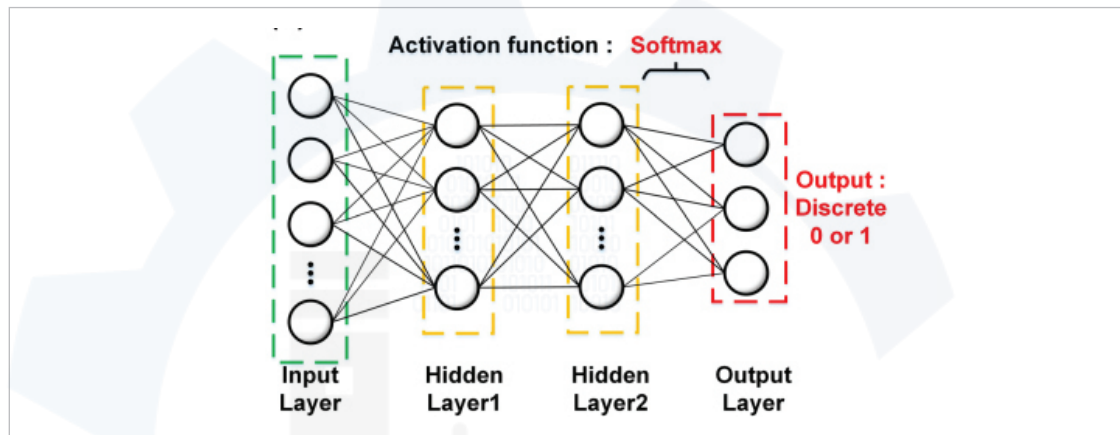
[그림 26] Sigmoid Function & Step Function

- Activation Function은 pre-layer에서 post-layer로 전파될 때 동작한다. 따라서, layer가 1개인 단일 구조(mono-layer structure)에서는 Activation Function이 사용되지 않는다. 즉, 입력 값 x 에 대하여 Affine operation $z = xW + b$ (W : weight matrix, b : bias vector)를 수행한 후, 바로 Soft-max layer로 연결되어 $output = S(z) = S(xW + b)$ 로 출력된다.
- 그러나, 2개 이상의 layer로 이루어진 구조에서는(multi-layer structure) 입력 값 x 에 대하여 Affine operation $z = xW + b$ 를 수행한 후, Activation Function $h(x)$ 를 거친다. 즉, l -th layer의 input value $z^{(l)}$ 에 대하여, Affine operation $y^{(l)} = z^{(l)}W^{(l)} + b^{(l)}$ 을 수행한 이후, output $z^{(l+1)} = h(y^{(l)}) = h(z^{(l)}W^{(l)} + b^{(l)})$ 을 출력한다. 해당 출력 값은 이후 $(l+1)$ -th layer의 input value가 된다.
- 전체 Layer의 수가 이라 할 때, 가장 마지막 L -th layer는 Activation을 하지 않는다. 이는 Mono-layer Structure에서 출력을 Activation 하지 않는 것과 동일하며, 곧 바

로 Soft-Max layer로 연결되어 최종 출력값을 산출한다. 즉, 이를 합성 함수의 형태로 전체를 표현한다면, 다음과 같다.

$$\begin{aligned} output &= S \circ A^{(L+1)} \circ h^{(L)} \circ A^{(L)} \circ \dots \circ h^{(1)} \circ A^{(1)}(x) \\ &= S(A^{(L+1)}(h^{(L)}(A^{(L)} \dots (h^{(1)}(A^{(1)}(x)))))) \quad A^{(l)}(x) \quad l-th \end{aligned}$$

여기서 x 는 초기 Data 값(1st-layer input value), $A^{(l)}(x)$ 는 l -th layer의 Affine Operator $A^{(l)}(x) = xW^{(l)} + b^{(l)}$, $h^{(l)}(x)$ 는 l -th layer의 Activation Function이다.



[그림 27] Feed-forward propagation Structure

- 역전파(Back-propagation)를 위해 순차적으로 미분을 수행하면, 다음과 같은 partial derivative 및 Jaccobian Matrix 구조를 따른다.

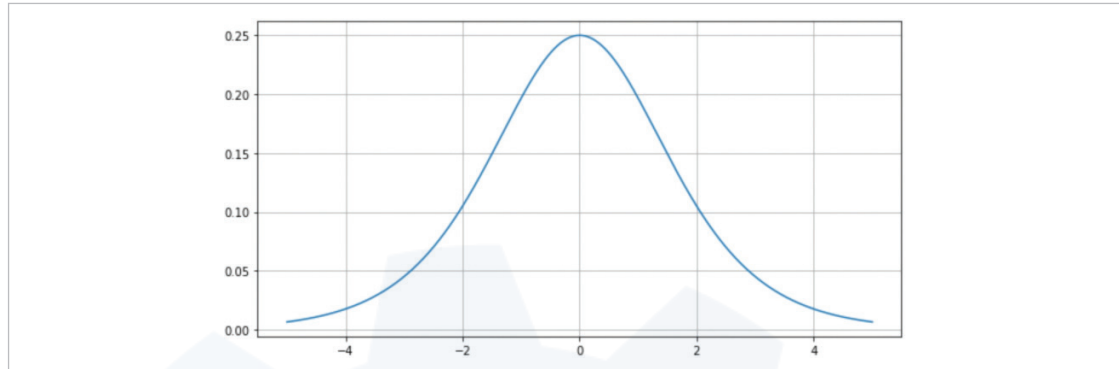
$$\begin{aligned} &\frac{\partial}{\partial W^{(l)}} F((w^{(L)}, b^{(L)}), \dots, (w^{(1)}, b^{(1)})) \\ &= [h^{(k)} \circ A^{(k)} \circ \dots \circ h^{(1)} \circ A^{(1)}]^T \cdot [DS(W^{(L+1)})^T Dh^{(L)}(W^{(L)})^T \dots Dh^{(k)}(x^{(k)})] \end{aligned}$$

$$\frac{\partial}{\partial b^{(l)}} F((w^{(L)}, b^{(L)}), \dots, (w^{(1)}, b^{(1)})) = [DS(W^{(L+1)})^T Dh^{(L)}(W^{(L)})^T \dots Dh^{(k)}(x^{(k)})]$$

여기서 Affine operator $A^{(l)}(x) = xW^{(l)} + b^{(l)}$ 는 조율 대상인 parameter들(W : weight matrix, b : bias vector)에 의존하지만, Activation Function은 그 자체로는 의존하지 않는다. 따라서, 미분 구조(전미분; Total Derivative)를 찾는데 있어서 오직 input value x 에 대한 미분 구조만 구하면 된다.

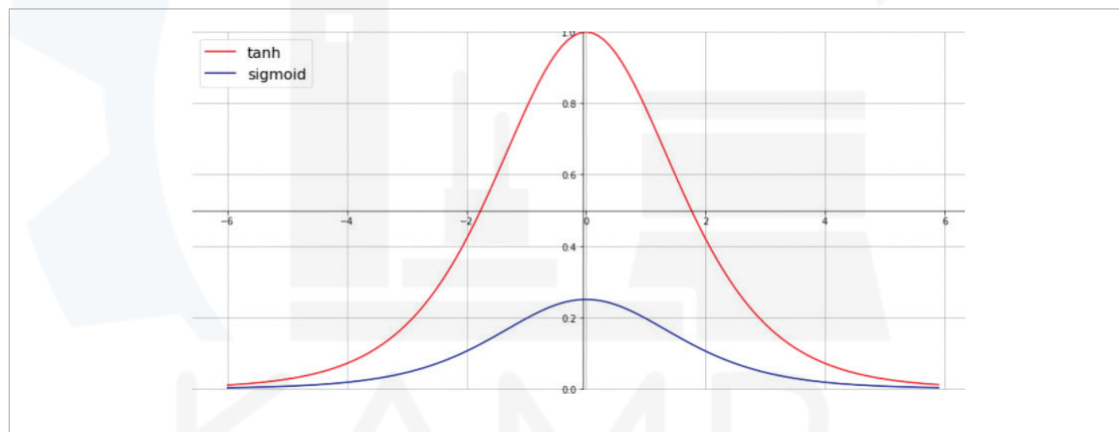
예시로는 다음과 같다.

$$\text{Sigmoid: } h(x) = \frac{1}{1+e^{-x}} \rightarrow \frac{dh(x)}{dx} = \frac{1}{1+e^{-x}} \left(1 - \frac{1}{1+e^{-x}}\right) = h(x)(1-h(x))$$



[그림 28] Sigmoid Derivate

$$\begin{aligned} \tanh: h(x) &= \tanh x = \frac{\sinh x}{\cosh x} = \frac{e^x - e^{-x}}{e^x + e^{-x}} \\ \frac{dh(x)}{dx} &= 1 - \left(\frac{e^x - e^{-x}}{e^x + e^{-x}}\right)^2 = 1 - [h(x)]^2 = (1-h(x))(1+h(x)) \end{aligned}$$

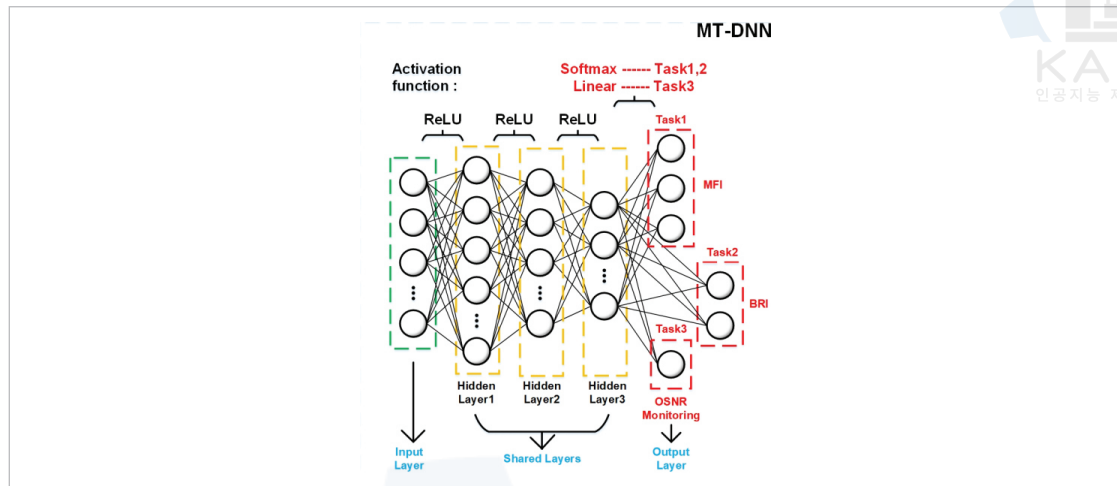


[그림 29] Sigmoid & tanh Derivate

$$\text{LeRU: } h(x) = \begin{cases} x & \cdots & \text{if } x > 0 \\ 0 & \cdots & \text{if } x \leq 0 \end{cases} \rightarrow \frac{dh(x)}{dx} = \begin{cases} 1 & \cdots & x > 0 \\ 0 & \cdots & x \leq 0 \end{cases}$$

- 또한, 활용 가능한 Activation Function은 여러 종류가 있지만, 보통 하나의 Neural Network에서는 사전에 결정된 하나의 Activation Function을 모든 Layer에서 사용하며, 여러 종류의 Activation Function을 교차 사용하지 않는다. 그러나 Multi-Task simulation 등 보다 특수한 상황에서는 가장 마지막 Layer 층에서 교차 사용을 하는 경우도 존재한다.¹⁾

1) YIJUN CHENG, SONGNIAN FU, MING TANG, AND DEMING LIU (2019). "Multi-task deep neural network (MT-DNN) enabled optical performance monitoring from directly detected PDM-QAM signals." OPTICS EXPRESS, Vol. 27, No. 13, 24 Jun 2019.



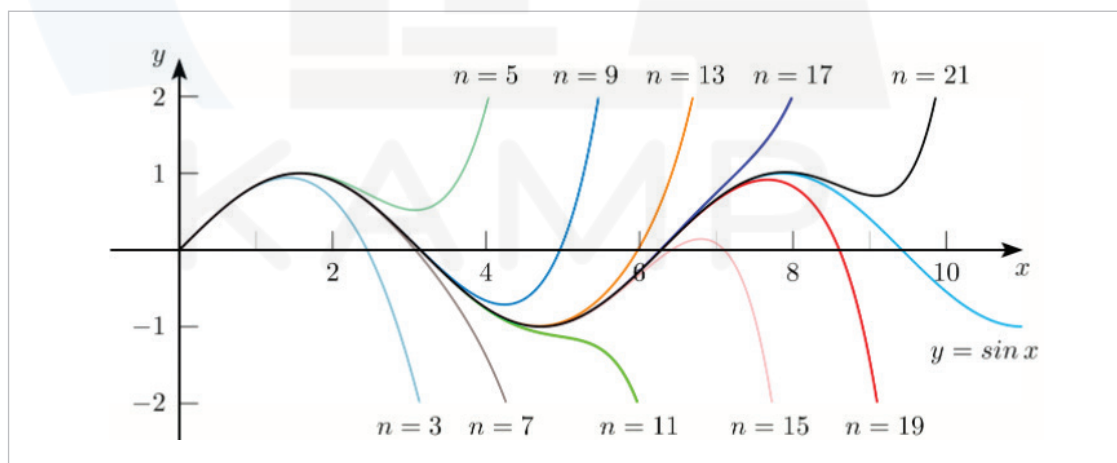
[그림 30] mixed Activation Function structure

• SGD

- Stochastic Gradient Descent는 가장 기본적인 Convex Optimization Theory인 Gradient Descent에서 발전된 최적화 기법이다. Gradient Descent는 다시 해석학 (Analysis)의 Taylor Series Expansion에 기반을 두고 있다.

$$f(x + \Delta x) = f(x) + \frac{f'(x)}{1!} \Delta x + \frac{f''(x)}{2!} (\Delta x)^2 + \dots = \sum_{n=0}^{\infty} \frac{f^{(n)}(x)}{n!} (\Delta x)^n$$

$$\Rightarrow f(x + \Delta x) \simeq f(x) + f'(x) \Delta x, \quad f(x + \Delta x) - f(x) = f'(x) \Delta x$$

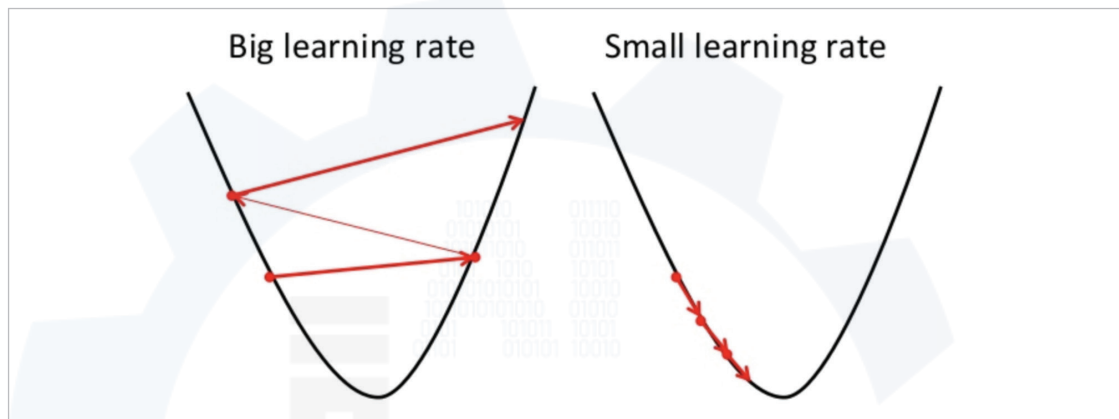


[그림 31] Taylor Expansion

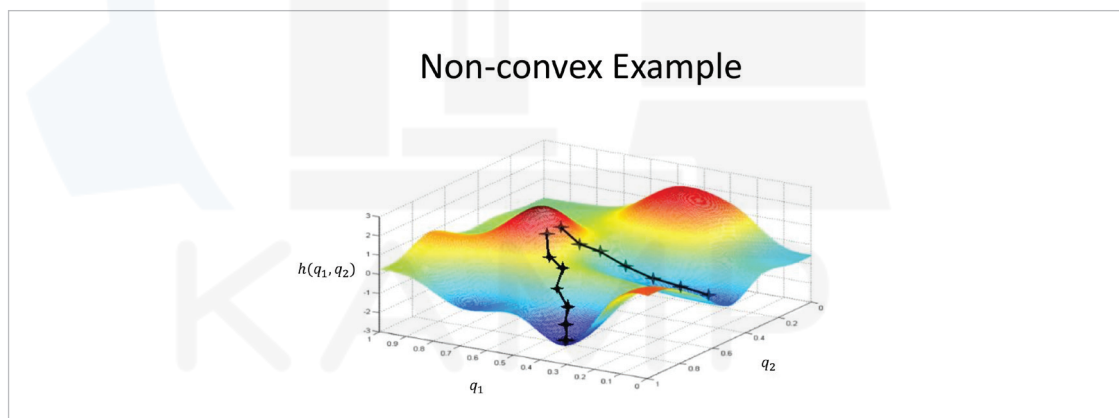
해당 근사 계산에 근거하여, Gradient Descent는 다음과 같은 수열 구조를 갖는다.

$$x_{n+1} = x_n - \eta \frac{df}{dx_n}$$

여기서 $f(x)$ 는 최적화 대상인 목표 함수(Objective Function)이고, η 는 학습 비율/속도(Learning Rate), x_n 은 n -th iteration에서의 parameter 계산 값이다. 여기서 Learning Rate η 를 크게 하면, iteration speed는 높아지지만 계산 값이 부정확하고, η 를 작게 하면, 계산 값이 정확해지지만 iteration speed가 낮아진다. 특히 너무 작은 η 값은 과적합(over-fitting)의 위험성을 가지고 있다.



[그림 32] Learning Rate



[그림 33] Gradient Descent

- Gradient Descent Sequence의 구조는 전적으로 objective function f 의 성질에 따라 달라진다. objective function f 의 형태에 따라서 미분 형태가 달라지며, 의존하는 parameter 수에 따라서 Jacobian Matrix의 차원 및 구조가 달라진다. 가장 보편적으로 Affine Operation $A(x) = xW + b$ (W : weight matrix, b : bias vector)과 Square loss $E(y, t) = \frac{1}{2} \sum_{k=1}^n |t_k - y_k|^2$ (with $\sum_{k=1}^n y_k = 1$) (with)에 대해서는 Jacobian Matrix가 다음과 같다.

Square loss Jacobian Matrix

$$\frac{\partial}{\partial W} E(y, t) = x^T \cdot DF(xW + b) = x^T \cdot (y - t)$$

$$\frac{\partial}{\partial b} E(y, t) = DF(xW + b) = y - t$$

따라서, Gradient Descent Sequence는 다음과 같다.

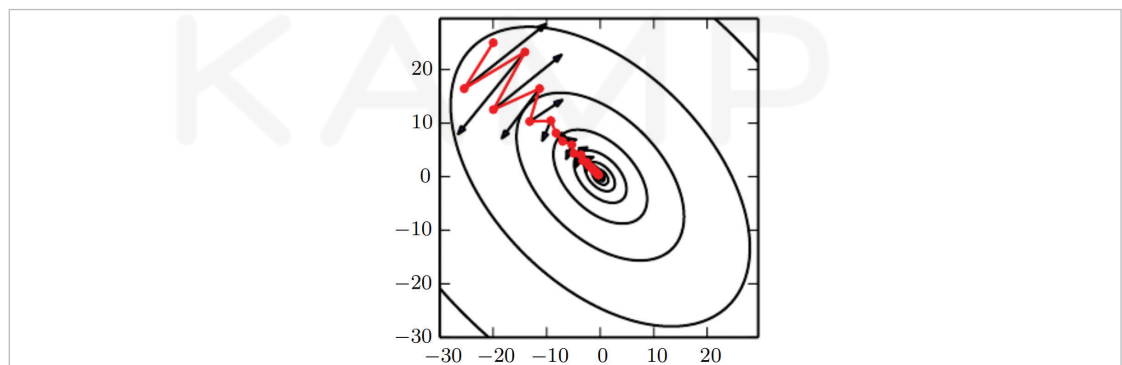
$$W_{n+1} = W_n - \eta \frac{\partial E}{\partial W_n} = W - \eta x^T \cdot (y - t)$$

$$b_{n+1} = b_n - \eta \frac{\partial E}{\partial b_n} = W - \eta(y - t)$$

- Gradient Descent를 개선한 여러 가지 확장 방식들이 존재하는데, 대표적으로 Momentum Gradient Descent(MGD), Stochastic Gradient Descent(SGD)가 있다.
- Momentum Gradient Descent는 물리학의 운동량(Momentum) 개념을 그대로 받아들여서, Gradient가 개선되는 방향이 이전의 개선 방향(마치 역학의 운동 방향)에 의존적이라는 관점이다. 따라서, 다음과 같은 수열 구조를 갖는다.

$$x_{n+1} = x_n - \alpha \frac{df}{dx_n} + \beta(x_n - x_{n-1})$$

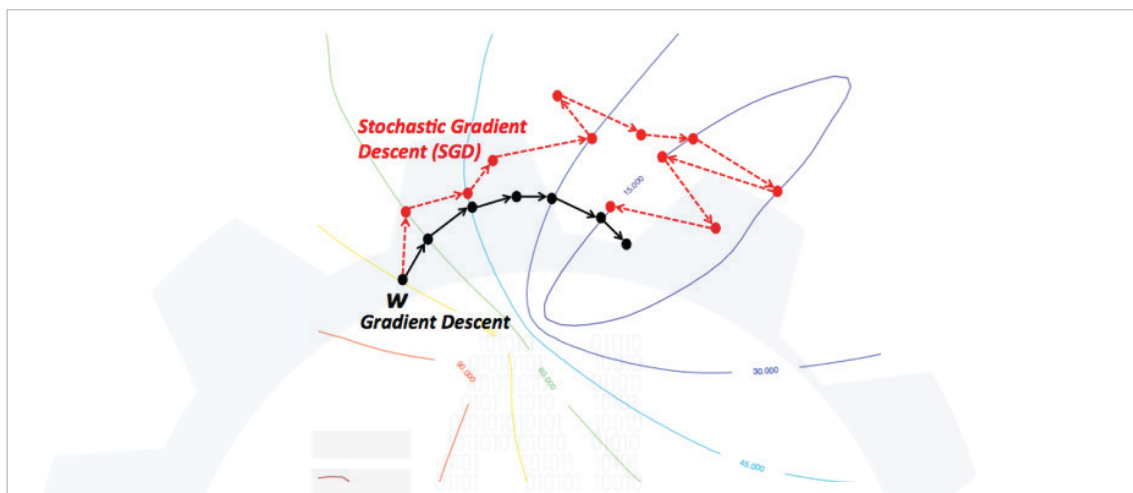
여기서 α 는 η 앞서 와 동일한 학습 비율/속도(Learning Rate)이며, β 는 Momentum에 대한 의존성을 조절하는 Tuning Parameter이다.



[그림 34] Momentum Gradient Descent

- 앞서 Momentum Gradient Descent가 Gradient Descent의 성능을 개선하기 위해서 수열 구조를 보완하였다면, Stochastic Gradient Descent는 iteration speed를 개선한 방식이다. 기본적으로 Gradient Descent는 주어진 '모든'데이터셋에 대해서

적용한다. 그러나 데이터셋의 규모가 매우 커질 경우(수 억~수 천억), 해당 규모에 대응되는 iteration과 computational cost는 급격하게 증가하며, 심각한 경우, 학습을 진행할 수 없다. 따라서, 이러한 문제점을 극복하기 위해 모든 데이터셋이 아니라 일정 데이터셋을 ‘확률적’으로 Random Sampling하여 Gradient Descent를 적용하여 iteration speed를 보장하는 접근법이 Stochastic Gradient Descent(SGD)이다.

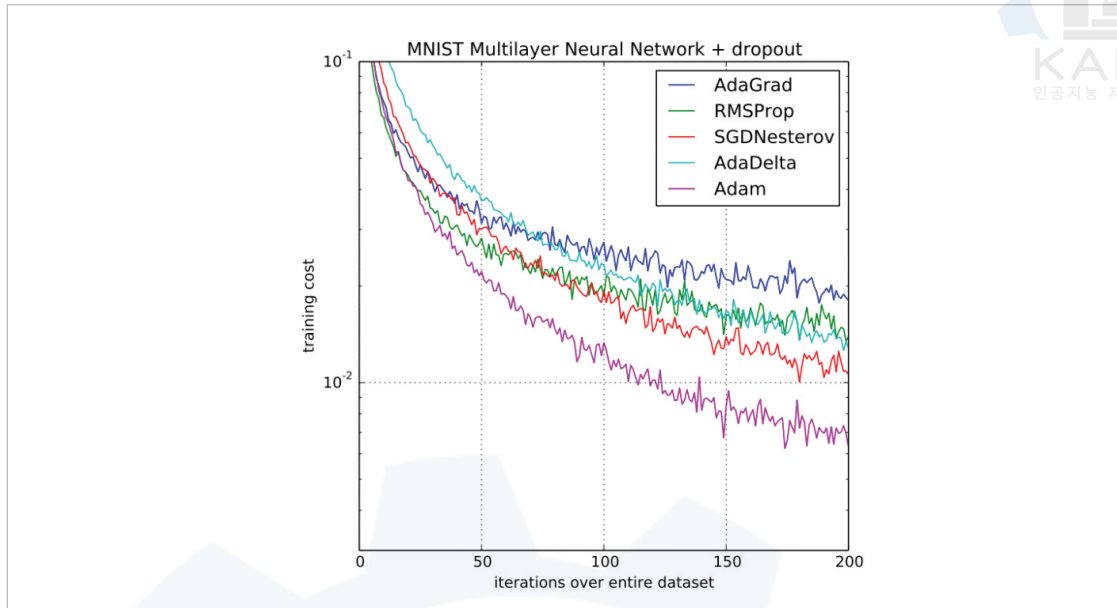


[그림 35] Stochastic Gradient Descent

그러나, Stochastic Gradient Descent는 기본적으로 데이터셋의 Random Sampling으로 Iteration Speed를 확보하는 방식이므로, 기본적인 Gradient Descent보다 다소 noise가 많은 편이다. 또한, 모든 데이터셋을 대상으로 하지 않기 때문에, local method이다. 따라서, 학습의 결과로서 극소점(local minimum)을 찾을 수는 있지만, 실제 목표 함수(objective function)의 최소점(global minimum)은 찾지 못할 수 있다. 반면, 일반적인 Gradient descent는 global method이므로 최소점(global minimum)을 구할 수 있다. 따라서, Stochastic Gradient Descent는 일반적인 상황에서는 사용되지 않으며, 최소 수 천만 ~ 수 억의 데이터셋이 있는 ‘빅 데이터’급 상황에서 사용하는 것이 보다 바람직하다.

• ADAM

- ADAM은 Adaptive Moment Estimation의 줄임말이며, 현재 Deep Learning에서 가장 보편적으로 많이 사용되는 기법이다. ADAM은 최적화 기법 중 Momentum Gradient Descent(MGD)와 Root Mean Square Propagation(RMS-Prop)를 동시에 적용한 최적화 기법이다. 따라서, ADAM을 이해하기 위해서는 Momentum Gradient Descent(MGD)와 Root Mean Square Propagation(RMS-Prop)을 이해할 필요가 있다.

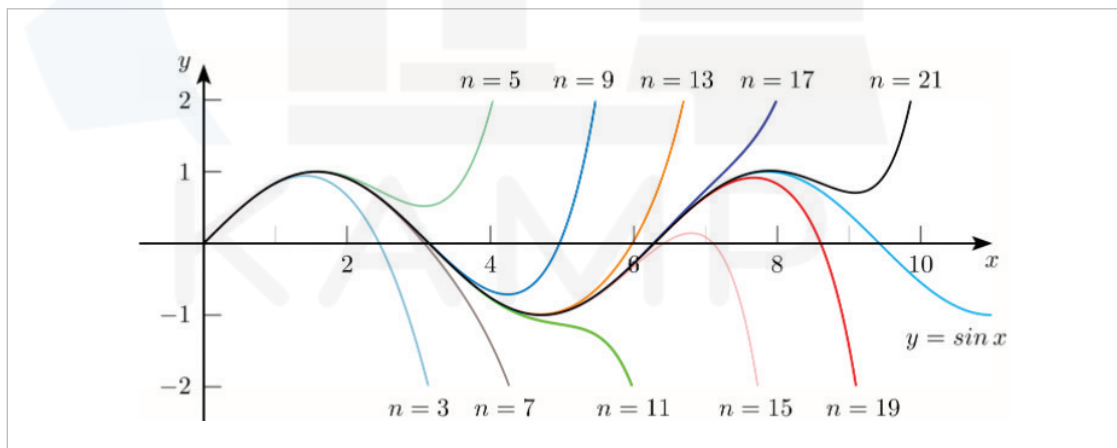


[그림 36] Cost Comparison

- 가장 기본적인 Convex Optimization Theory는 Gradient Descent로 다시 해석학 (Analysis)의 Taylor Series Expansion에 기반을 두고 있다.

$$f(x + \Delta x) = f(x) + \frac{f'(x)}{1!} \Delta x + \frac{f''(x)}{2!} (\Delta x)^2 + \dots = \sum_{n=0}^{\infty} \frac{f^{(n)}(x)}{n!} (\Delta x)^n$$

$$\Rightarrow f(x + \Delta x) \simeq f(x) + f'(x) \Delta x, \quad f(x + \Delta x) - f(x) = f'(x) \Delta x$$

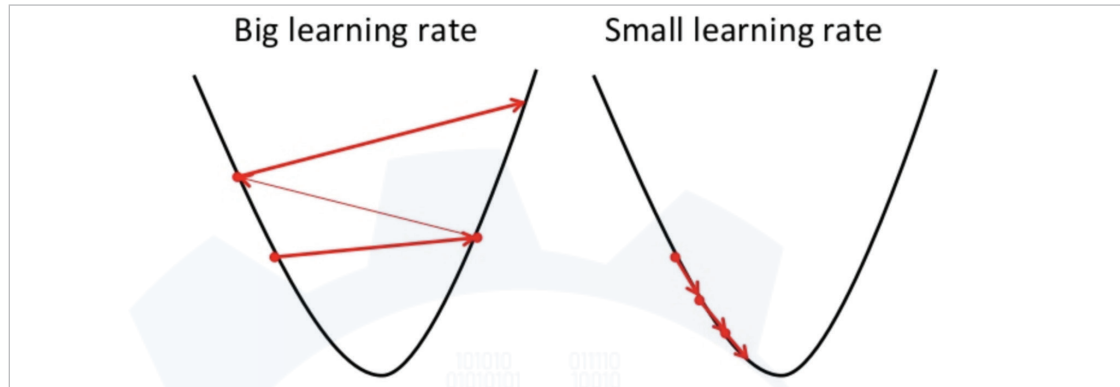


[그림 37] Taylor Expansion

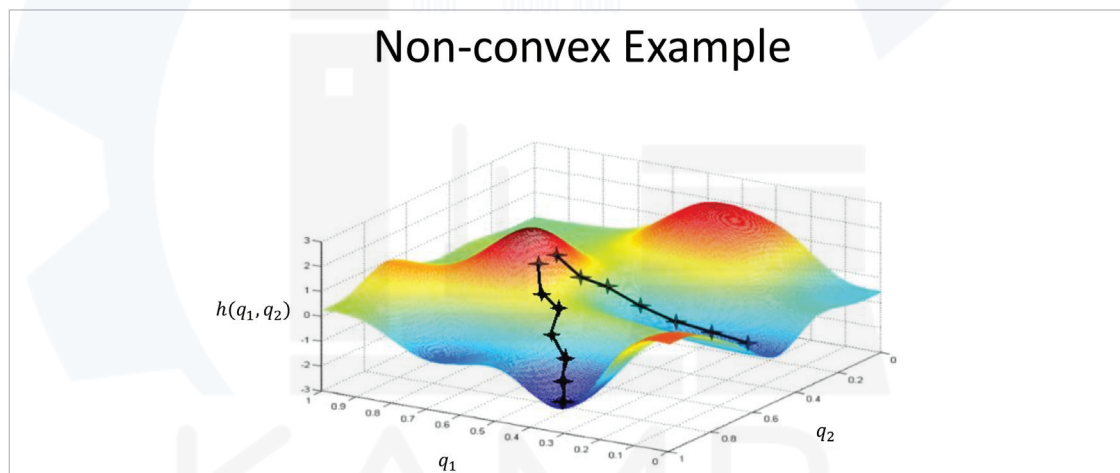
해당 근사 계산에 근거하여, Gradient Descent는 다음과 같은 수열 구조를 갖는다.

$$x_{n+1} = x_n - \eta \frac{df}{dx_n}$$

여기서 $f(x)$ 는 최적화 대상인 목표 함수(Objective Function)이고, η 는 학습 비율/속도(Learning Rate), x_n 은 n -th iteration에서의 parameter 계산 값이다. 여기서 Learning Rate η 를 크게 하면, iteration speed는 높아지지만 계산 값이 부정확고, η 를 작게 하면, 계산 값이 정확해지지만 iteration speed가 낮아진다. 특히 너무 작은 η 값은 과적합(over-fitting)의 위험성을 가지고 있다.



[그림 38] Learning Rate



[그림 39] Gradient Descent

- Gradient Descent Sequence의 구조는 전적으로 objective function f 의 성질에 따라 달라진다. objective function f 의 형태에 따라서 미분 형태가 달라지며, 의존하는 parameter 수에 따라서 Jacobian Matrix의 차원 및 구조가 달라진다. 가장 보편적으로 Affine Operation (weight matrix, bias vector)과 Square loss $E(y, t) = \frac{1}{2} \sum_{k=1}^n |t_k - y_k|^2$ (with $\sum_{k=1}^n y_k = 1$)에 대해서는 Jacobian Matrix가 다음과 같다.

Square loss Jacobbian Matrix

$$\frac{\partial}{\partial W} E(y, t) = x^T \cdot DF(xW + b) = x^T \cdot (y - t)$$

$$\frac{\partial}{\partial b} E(y, t) = DF(xW + b) = y - t$$

따라서, Gradient Descent Sequence는 다음과 같다.

$$W_{n+1} = W_n - \eta \frac{\partial E}{\partial W_n} = W_n - \eta x^T \cdot (y - t)$$

$$b_{n+1} = b_n - \eta \frac{\partial E}{\partial b_n} = b_n - \eta (y - t)$$

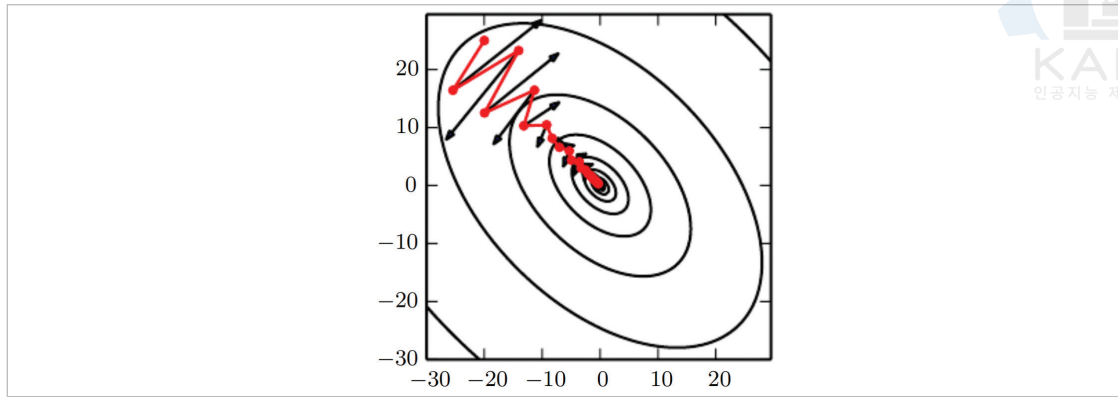
- Gradient Descent를 개선한 여러 가지 확장 방식들이 존재하는데, 대표적으로 Momentum Gradient Descent(MGD), Stochastic Gradient Descent(SGD)가 있다.
- Momentum Gradient Descent는 물리학의 운동량(Momentum) 개념을 그대로 받아들여서, Gradient가 개선되는 방향이 이전의 개선 방향(마치 역학의 운동 방향)에 의존적이라는 관점이다. 따라서, 다음과 같은 수열 구조를 갖는다.

$$x_{n+1} = x_n - \alpha \frac{df}{dx_n} + \beta(x_n - x_{n-1})$$

$$\begin{aligned} \rightarrow 1) \quad W_{n+1} &= W_n - \alpha \frac{\partial E}{\partial W_n} + \beta(W_n - W_{n-1}) \\ &= W_n - \alpha x^T \cdot (y - t) + \beta(W_n - W_{n-1}) \end{aligned}$$

$$\begin{aligned} 2) \quad b_{n+1} &= b_n - \alpha \frac{\partial E}{\partial b_n} + \beta(b_n - b_{n-1}) \\ &= b_n - \alpha (y - t) + \beta(b_n - b_{n-1}) \end{aligned}$$

여기서 α 는 앞서 η 와 동일한 학습 비율/속도(Learning Rate)이며, β 는 Momentum에 대한 의존성을 조절하는 Tuning Parameter이다.



[그림 40] Momentum Gradient Descent

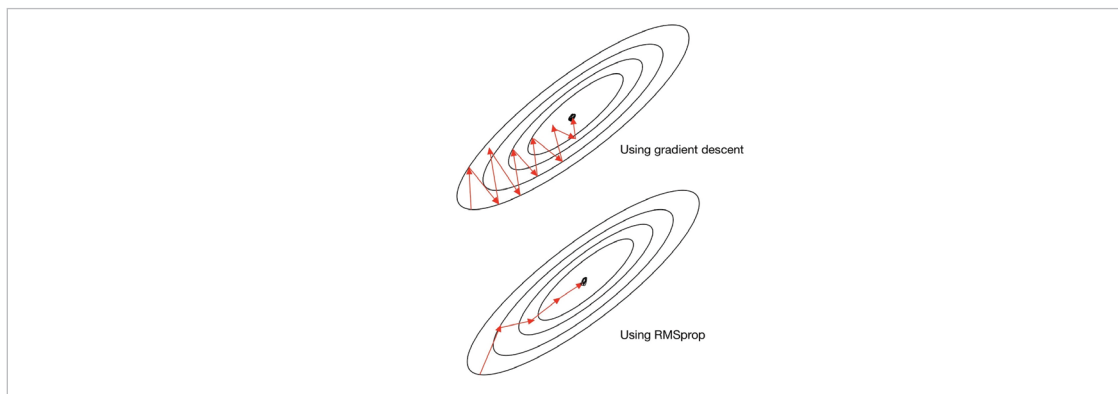
- Root Mean Square Propagation(RMS-Prop)는 기본 개념은 Momentum Gradient Descent와 매우 유사하다. 하지만, Momentum Gradient Descent와 달리 Gradient의 방향은 적용하지 않고 그 크기만 적용한다(물리학적으로는 속도(velocity)가 아닌 속력(speed)만을 차용하는 것). Root Mean Square Propagation의 계산 구조는 다음과 같다.

$$x_{n+1} = x_n - \alpha \frac{1}{\sqrt{\beta |x_n - x_{n-1}| + \epsilon}} \frac{df}{dx_n}$$

$$\rightarrow 1) W_{n+1} = W_n - \alpha \frac{1}{\sqrt{\beta |W_n - W_{n-1}| + \epsilon}} \frac{dE}{dW_n}$$

$$2) b_{n+1} = b_n - \alpha \frac{1}{\sqrt{\beta |b_n - b_{n-1}| + \epsilon}} \frac{dE}{db_n}$$

여기서 α , β 는 앞서 Momentum Gradient Descent와 동일하게 각각 학습 비율/속도(Learning Rate)와 Tuning Parameter이며, ϵ 은 $\sqrt{\beta |x_n - x_{n-1}|} \approx 0$ 이 되는 경우에 연산이 발산하는 것을 방지하기 위한 Regularization Term이다.



[그림 41] RMS-prop

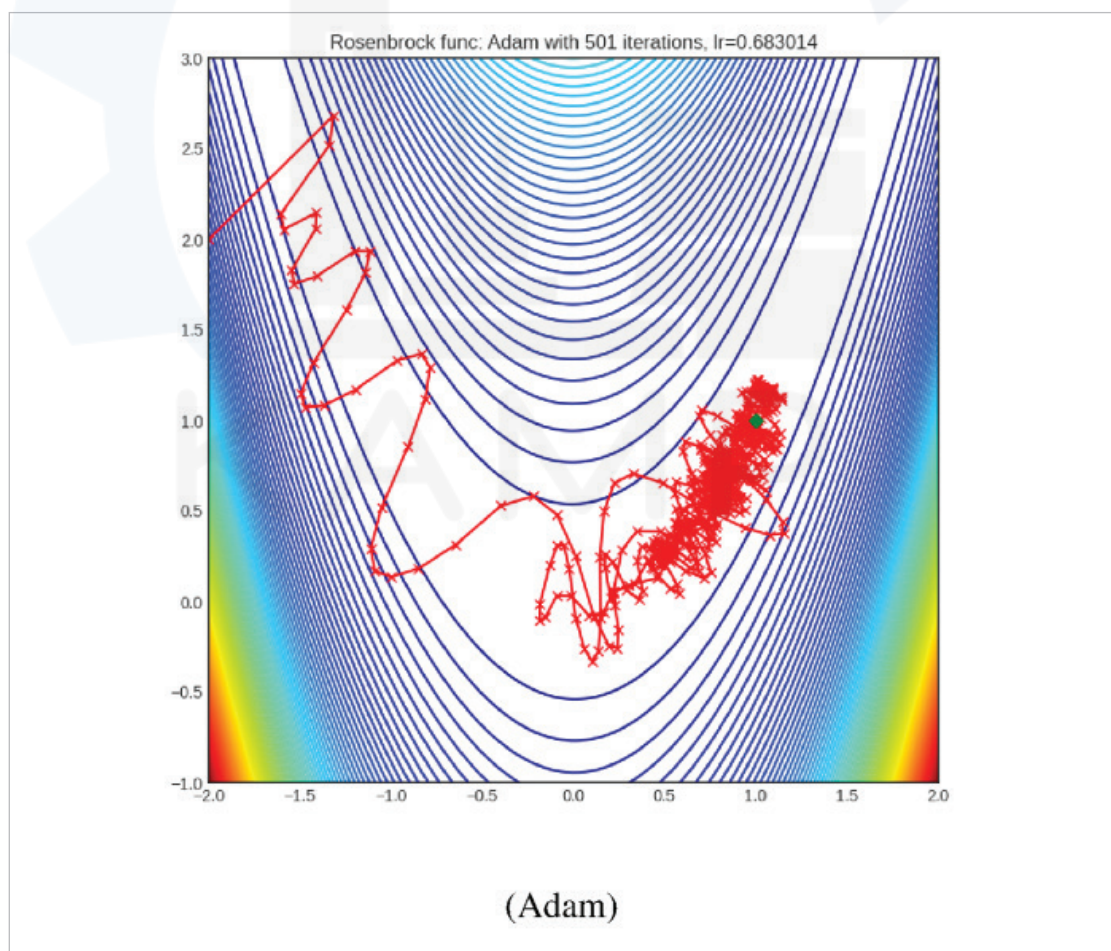
- 앞서 언급하였듯, ADAM은 Momentum Gradient Descent(MGD)와 Root Mean Square Propagation(RMS-Prop)를 동시에 활용하는 최적화 알고리즘이다. 따라서, 다음과 같은 수열 구조를 갖는다.

$$x_{n+1} = x_n - \alpha \frac{1}{\sqrt{\beta_2 |x_n - x_{n-1}| + \epsilon}} \left(\frac{df}{dx_n} + \beta_1 (x_n - x_{n-1}) \right) \rightarrow 1)$$

$$W_{n+1} = W_n - \alpha \frac{1}{\sqrt{\beta_2 |W_n - W_{n-1}| + \epsilon}} \left(\frac{dE}{dW_n} + \beta_1 (W_n - W_{n-1}) \right)$$

$$2) \quad b_{n+1} = b_n - \alpha \frac{1}{\sqrt{\beta_2 |b_n - b_{n-1}| + \epsilon}} \left(\frac{dE}{db_n} + \beta_1 (b_n - b_{n-1}) \right)$$

동일하게 α , β_1 , β_2 는 학습 비율/속도(Learning Rate)와 Tuning Parameter들이며, ϵ 은 발산을 방지하는 Regularization Term이다. 단, 앞서와 달리 momentum 전체를 받는 부분과, 크기만 받는 부분이 각각 다른 Tuning parameter에 의존한다.



[그림 42] Adam Optimizer

• Loss Function(손실함수)

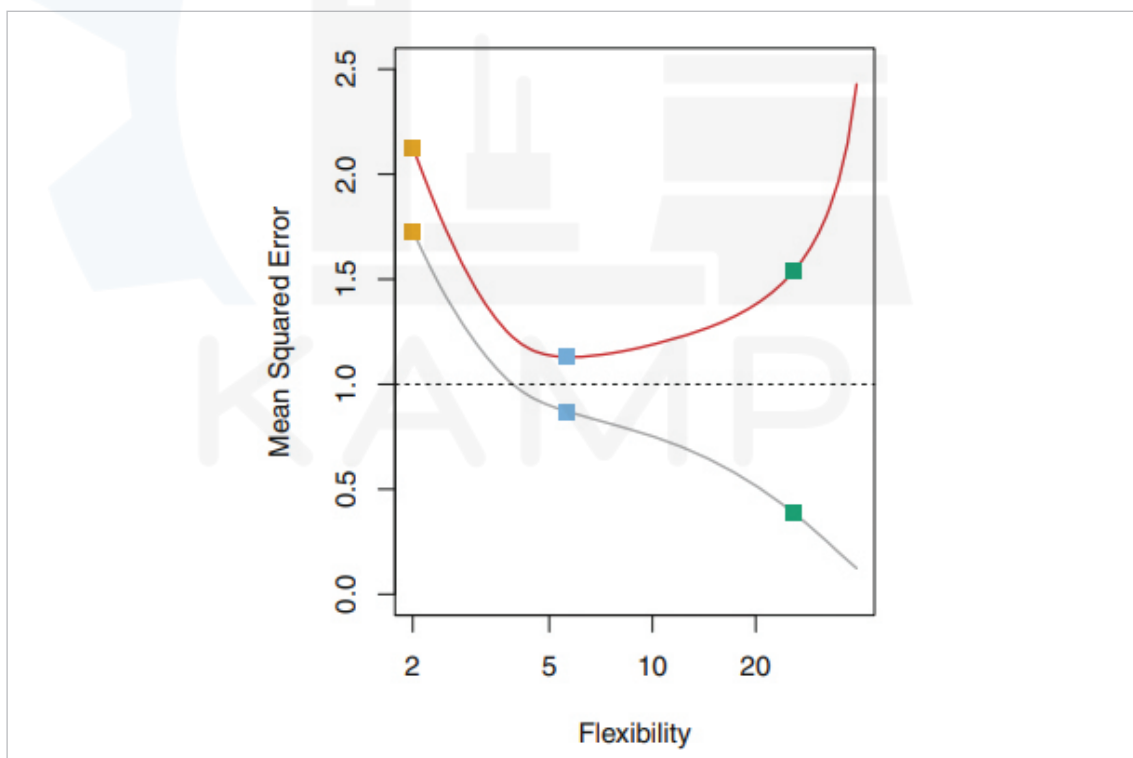
- Loss Function(손실함수)는 기본적으로는 학습에 있어서 최소화해야 하는 대상인 objective function과 동일하다. 최소화의 기본 개념은 Ground-Truth-Value 또는 Labeled Data와 연산 결과 도출된 Predicted Value 사이의 오차(Error)를 최소화하는 것이다.
- 가장 기본적인 형태로는 우선 Regression problem의 Mean Squared Error loss(MSE), Residual Sum of Square loss(RSS), Total Sum of Square loss(TSS), Residual Standard Error(RSE), R^2 loss 등이 있다.

$$\text{MSE: } \frac{1}{n} \sum_{i=1}^n (y_i - \hat{f}(x_i))^2 / \text{RSS: } \sum_{i=1}^n (y_i - \hat{f}(x_i))^2$$

$$\text{RSE: } \sqrt{\sum_{i=1}^n (y_i - \hat{f}(x_i))^2 / (n-2)} = \sqrt{\frac{\text{RSS}}{n-2}}$$

$$\text{TSS: } \sum_{i=1}^n (y_i - \bar{f})^2 \text{ where } \bar{f} = \frac{1}{n} \sum_{i=1}^n \hat{f}(x_i)$$

$$R^2 \text{ loss: } \frac{\text{TSS} - \text{RSS}}{\text{TSS}} = 1 - \frac{\text{RSS}}{\text{TSS}}$$



[그림 43] Flexibility & Loss

또한, Classification problem의 가장 기본적인 loss들로는 Error Rate, Bayes Error Rate, Gini Index, Entoropy가 있다.

$$\text{Error Rate: } \frac{1}{n} \sum_{i=1}^n I(y_i \neq \hat{f}(x_i))$$

$$\text{Bayes Error: } E = 1 - \max_k \Pr[Y = k|X]$$

$$\text{Gini Index } G = \sum_{k=1}^K \Pr[Y = k|X](1 - \Pr[Y = k|X])$$

$$\text{Entropy } D = - \sum_{k=1}^K \Pr[Y = k|X] \log(\Pr[Y = k|X])$$

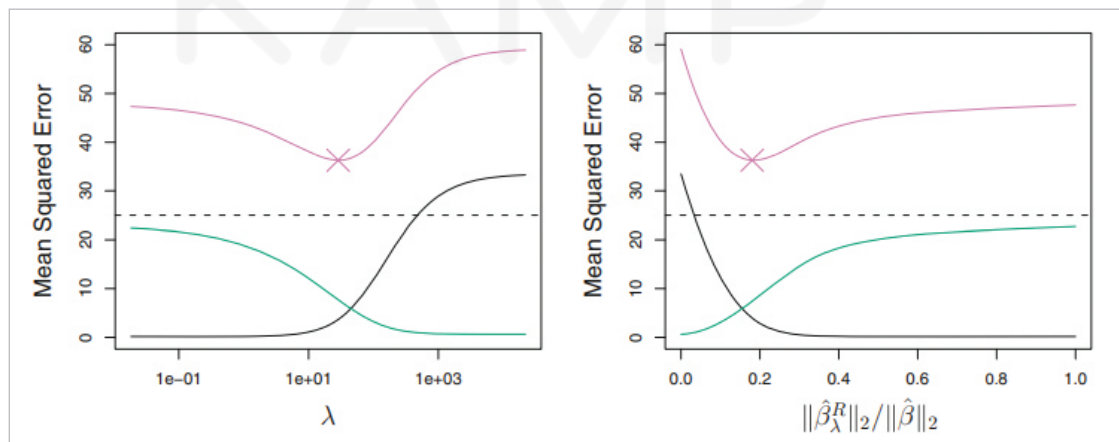
- 그러나, Ridge regression, LASSO나, Tree-based Method의 Bagging, Random Forest, Boosting과 같은 경우에는, 과적합(Overfitting)으로 인한 Test Error를 줄이기 위해서 최소화 대상인 Objective Function에 Penalty Term이 추가로 결합되어 있다.

$$\text{Ridge: } \min \left[\sum_{i=1}^n (y_i - \hat{f}(x_i))^2 + \lambda \sum_{j=1}^n \beta_j^2 \right]$$

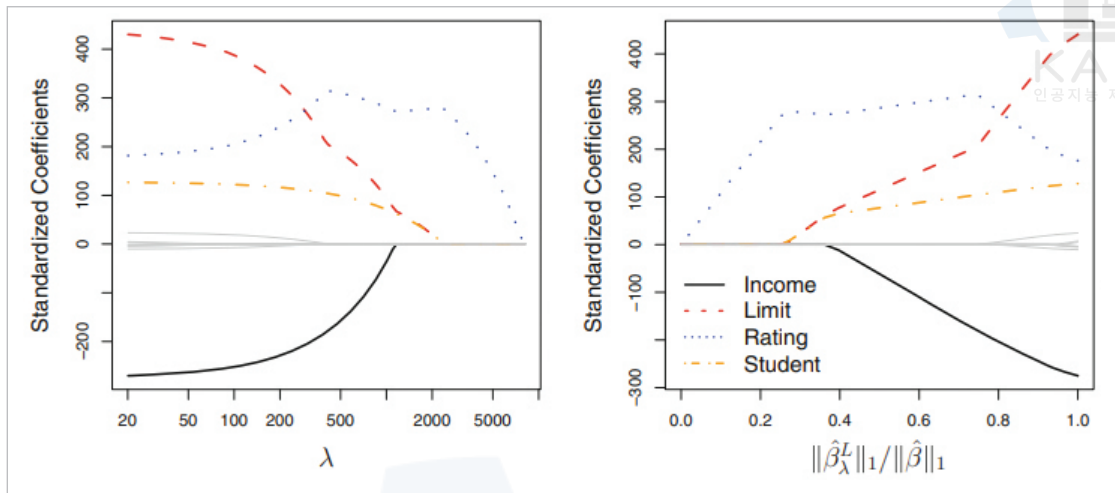
$$\text{LASSO: } \min \left[\sum_{i=1}^n (y_i - \hat{f}(x_i))^2 + \lambda \sum_{j=1}^n |\beta_j| \right]$$

$$\text{Tree-based: } \min \left[\sum_{j=1}^{|T|} \sum_{i \in R_j} (y_i - \hat{y}_{R_j})^2 + \alpha |T| \right]$$

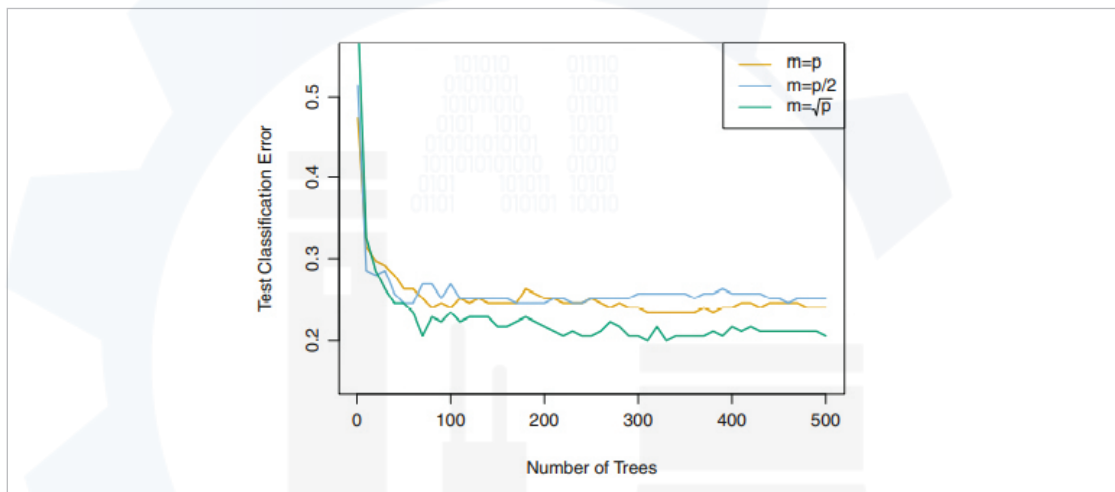
이러한 경우, Objective Function과 Loss Function은 다르다. Objective Function은 Training을 완수하기 위해 최소화해야 하는 전체 대상인 반면, Loss Function은 단순히 학습 결과의 오류 최소화를 목표로만 움직인다.



[그림 44] Ridge Regression Loss



[그림 45] LASSO Regression Loss



[그림 46] Tree-based Loss

- 또한, High-dimensional problem에서 추정 계수 및 Test Error 분석을 위해서 기존 Loss Function에 대한 수정 및 변형이 필요하다. 변형된 Loss Function으로는 C_p , Akaike Information Criterion(AIC), Bayesian Information Criterion(BIC), Adjusted 등이 R^2 있다.

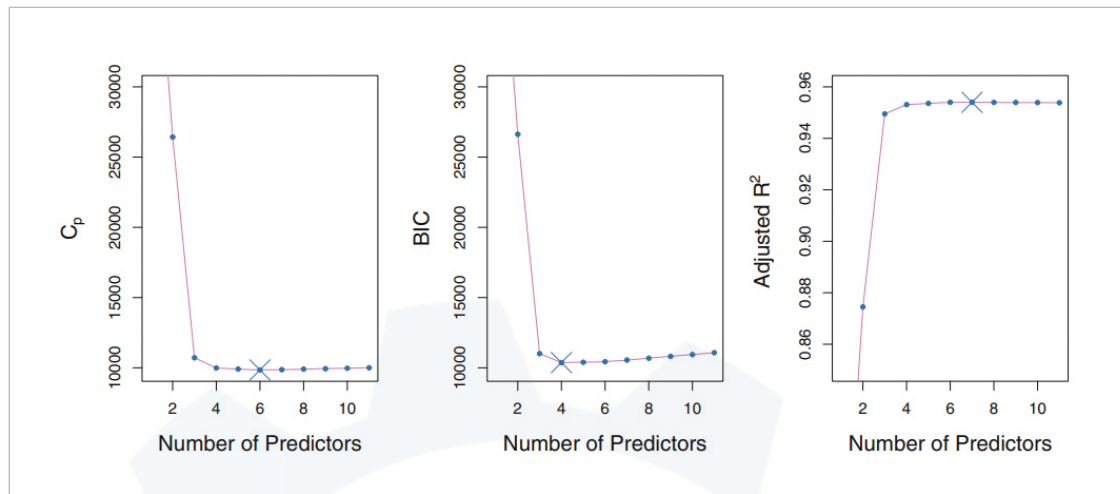
$$C_p = \frac{1}{n}(RSS + 2d\hat{\sigma}^2)$$

$$AIC: \frac{1}{n\hat{\sigma}^2}(RSS + 2d\hat{\sigma}^2)$$

$$BIC: \frac{1}{n\hat{\sigma}^2}(RSS + d\hat{\sigma}^2 \log n)$$

$$\text{Adjusted } R^2: 1 - \frac{RSS/(n-d-1)}{TSS/(n-1)}$$

여기서 $\hat{\sigma}$ 는 표준편차 추정치이고, n 은 사용된 데이터의 수, d 는 사용된 Feature (Predictor/Variable)의 수이다.



[그림 47] C_p , BIC, adjusted R^2

- Deep Learning 분야에서는 추가적으로, Square loss나 Entropy loss를 활용하기도 한다. True-value로는 prelabeled data 를 활용한다.

$$\text{Square loss } E(y, t) = \frac{1}{2} \sum_{k=1}^n |t_k - y_k| \text{ with } \sum_{k=1}^n y_k = 1$$

$$\text{Entoropy loss } E(y, t) = - \sum_{k=1}^n t_k \log \left(\frac{\exp[y_k]}{\sum_{j=1}^n \exp[y_j]} \right)$$

- 특히, Deep Learning 방식에서는 필수적으로 순방향 전파(Feed- Forward Network/ Propagation)과 역전파(Back-proppagation)이 수반되며, 특히 역전파를 위해 각 parameter에 대응하는 편미분 구조를 찾는 것이 loss function 관리에 있어서 중요하다. 예를 들어서 Layer가 전개되는 구조를 가장 보편적인 Affine 구조인 $xW+b$ (W : weight matrix, b : bias vector)라 할 때, 아래와 같다.

Square loss Jacobbian Matrix

$$\frac{\partial}{\partial W} E(y, t) = x^T \cdot DF(xW+b) = x^T \cdot (y-t)$$

$$\frac{\partial}{\partial b} E(y, t) = DF(xW+b) = y-t$$

Entropy loss Jacobbian Matrix

$$\frac{\partial}{\partial W} E(y, t) = x^T \cdot DF(xW + b) = x^T \cdot (\log(e^y / \sum_{i=1}^n e^{y_i}) - t)$$

$$\frac{\partial}{\partial b} E(y, t) = DF(xW + b) = \log(e^y / \sum_{i=1}^n e^{y_i}) - t$$

• AI 분석 방법론(알고리즘) 구축 절차 설명

0. 문제 정의

1. 데이터 준비

가. 수집한 데이터를 조작, 가공하기 편한 형태로 변환

◦ 참고로 데이터의 특성은 변경하지 않는다.

예) 데이터.text 파일 => 데이터.csv 파일로 변경

나. 중요한 정보나 민감한 내용 여부 확인 후 삭제 및 보존 여부 결정

다. 데이터 크기와 타입(시계열, 표본, 지리 정보 등) 확인

라. 작업 환경을 구성 * 부록 '분석환경 구축을 위한 설치 가이드' 참고

① 라이브러리(library) 설치

② 라이브러리(library) 불러오기

③ 데이터 불러오기

2. 데이터 탐색

가. 사이트에 대한 기초 지식 공부

나. 데이터 탐색의 결과를 히스토리화를 위한 주피터 사용

다. 데이터 기본 특성 파악

① 데이터 사본 생성

② 레이블링 확인

③ 데이터 유형확인

(1) 범주형 데이터

(2) 정수 / 부동소수

(3) 최대값, 최소값 여부

(4) 구조적, 비구조적 문자열인지 확인

④ 데이터 비율 확인 및 데이터 확인

(1) 중복, 누락, 이상치 확인

⑤ 데이터 분포 형태

라. 학습 방법 고려

마. 데이터 시각화

바. 특성 간의 상관관계 파악

① 데이터 간의 상관관계 파악

3. 데이터 정제(전처리)

- ① 데이터의 특성을 기반으로 이상치를 수정 혹은 제거
- ② 누락된 값을 채우거나 열 제거
- ③ 무의미한 특성 제거

여기서 Tip.

여기서 혼동 될 수 있는 것이 데이터 정제 파트와 데이터 탐색과 전처리 파트이다. 두 과정이 같다고 판단할 수 있다. 또한, 상황에 따라 전처리 과정에서 데이터를 정제하는 게 빠르고 좋을 수도 있다. 하지만 데이터는 최대한 원본을 유지하고 사본으로 작업을 진행하는 것을 권장한다.

4. 알고리즘 선택

- ① 데이터의 특성에 맞게 알고리즘(및 모델)을 선택
- ② 선택된 알고리즘 중 사용할 알고리즘 채택

5. 학습, 평가 데이터 준비

- ① 데이터 분리
 - (1) 학습용 데이터
 - (2) 테스트용 데이터
 - (3) 검증용 데이터
- ② 데이터 스케일 조정

6. 모델링

- ① 모델 동작 테스트

7. 모델 훈련

8. 모델 튜닝

- ① 하이퍼 파라미터 수정

9. 모델 평가 및 해석

- ① 결과 시각화 및 해석

2.3 분석 체험

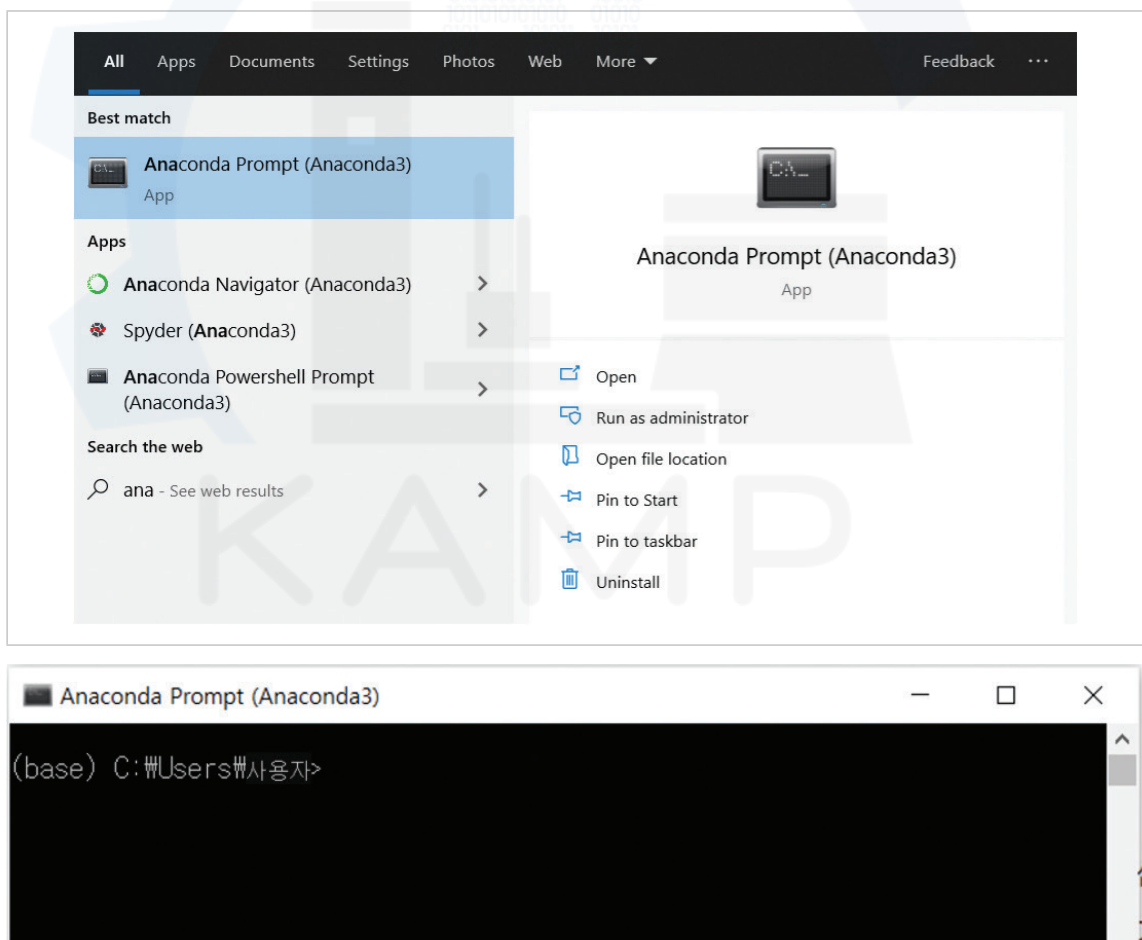
1) 필요 SW, 패키지 설치 방법 및 절차 가이드

▶ SW 설치는 ‘부록 분석환경 구축을 위한 설치 가이드’참고

▶ 가상환경 구축 가이드

- 3. 부록 [분석 환경 구축을 위한 설치 가이드]에서 설치한 아나콘다(Anaconda)에서 파이썬의 독립적인 가상의 작업 환경을 구축한다. 파이썬의 모듈은 다양한 버전이 존재하기 때문에 충돌이 일어나기 쉬우며, 이러한 충돌을 방지하기 위해 실습마다 독립적인 가상 환경 구축을 권장한다. 가상 환경을 사용하기 위해서는 환경을 구축하고 실행(활성화)해야하고, 실습이 끝난 이후에는 환경을 비활성화 시켜야 한다. 주피터 노트북에서 가상 환경을 실행하는 구체적인 순서 및 실행 방법은 아래와 같다.

- 아나콘다 프롬프트 실행



- 가상 환경 생성

```
# conda create -n 가상환경이름 python=버전
```

```
conda create -n KAMP python=3.7.0
```

```

Anaconda Prompt (Anaconda3)

(base) C:\Users\사용자>conda create -n KAMP python=3.8.8
Collecting package metadata (current_repodata.json): done
Solving environment: failed with repodata from current_repodata.json, will retry with next repodata source.
Collecting package metadata (repodata.json): done
Solving environment: done

==> WARNING: A newer version of conda exists. <==
  current version: 4.10.1
  latest version: 4.10.3

Please update conda by running

  $ conda update -n base -c defaults conda
  
```

Proceed ([y]/n)? y # 해당 문구가 뜨면 'y' 를 입력한다.

```

done
#
# To activate this environment, use
#
#     $ conda activate KAMP
#
# To deactivate an active environment, use
#
#     $ conda deactivate
#
(base) C:\Users\사용자>
  
```

- 가상 환경 실행

가상 환경이 활성화되면 아래 그림처럼 명령 프롬프트 앞에 (가상 환경 이름)이 표시된다.

```
# conda activate 가상환경이름
```

```
conda activate KAMP
```

```

(base) C:\Users\사용자>conda activate KAMP
(KAMP) C:\Users\사용자>
  
```

```
pip install ipykernel
```

```

# python -m ipykernel install --user --name 가상환경이름 --display-name "가상환경이름"
python -m ipykernel install --user --name KAMP --display-name "KAMP"
  
```

```

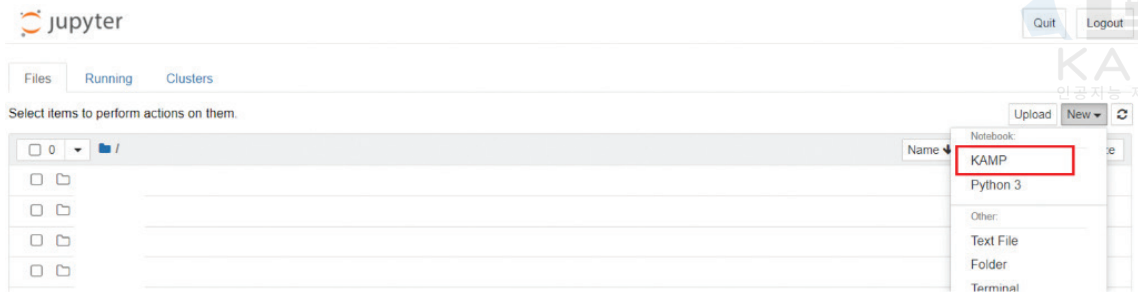
# jupyter notebook 실행
jupyter notebook
  
```

```

(KAMP) PS C:\Users\사용자> pip install ipykernel
Collecting ipykernel
  Using cached ipykernel-6.6.0-py3-none-any.whl (126 kB)
Collecting traitlets<6.0,>=5.1.0
  Using cached traitlets-5.1.1-py3-none-any.whl (102 kB)
Collecting ipython>=7.23.1
  Using cached ipython-7.30.1-py3-none-any.whl (791 kB)
Collecting debugpy<2.0,>=1.0.0
  Using cached debugpy-1.5.1-cp37-cp37m-win_amd64.whl (4.3 MB)
  
```

```

(KAMP) PS C:\Users\사용자> python -m ipykernel install --user --name KAMP --display-name "KAMP"
Installed kernelspec KAMP in C:\Users\사용자\AppData\Roaming\jupyter\kernels\kamp
(KAMP) PS C:\Users\사용자> jupyter notebook
  
```



2) 분석 단계별 프로세스 - LSTM-AE

• 분석 단계별 세부 설명



[그림 48] Flow Chart

[단계 ①] 데이터 준비

①-1. 라이브러리(Import) 설치

```

!pip install tensorflow ==1.15.0
!pip install keras ==2.3.1

!pip install pandas
!pip install numpy
!pip install sklearn
!pip install seaborn
!pip install datetime
!pip install matplotlib
!pip install pydot
!pip install graphviz
  
```

[코드 1] pip 패키지 설치 방법

①-2. 라이브러리(Import) 불러오기

아래 보이는 패키지들이 앞으로 우리가 사용할 패키지다.

```
# 데이터 분석 툴
import pandas as pd
import numpy as np

# 운영체제와 관련된 툴
import os
import glob

# 시각화
import seaborn as sns
import matplotlib.pyplot as plt # matplotlib
%matplotlib inline

# 경고 방지
import warnings # 워닝 방지
warnings.filterwarnings('ignore')

plt.figure(figsize=(15,15))
plt.style.use('ggplot')
```

[코드 2] 패키지 import 코드

1) 데이터 분석 툴

- 가) 판다스(Pandas) 모듈은 대표적인 데이터 분석 패키지보다 쉽게 데이터 분석 기능이 가능하다.
- 나) 넘파이(Numpy)는 C언어로 구현된 파이썬(Python) 라이브러리로 행렬이나 일반적으로 대규모 다차원 배열을 쉽게 핸들링 가능하다.

2) 운영체제와 관련된 패키지

- 가) Os 모듈은 Operating System의 약자로 운영체제에서 제공되는 여러 기능을 Python에서 사용 가능하다.
- 나) Glob 모듈은 특정한 패턴이나 확장자를 가진 파일들의 경로를 가져와 사용 가능하다.

3) 시각화 툴

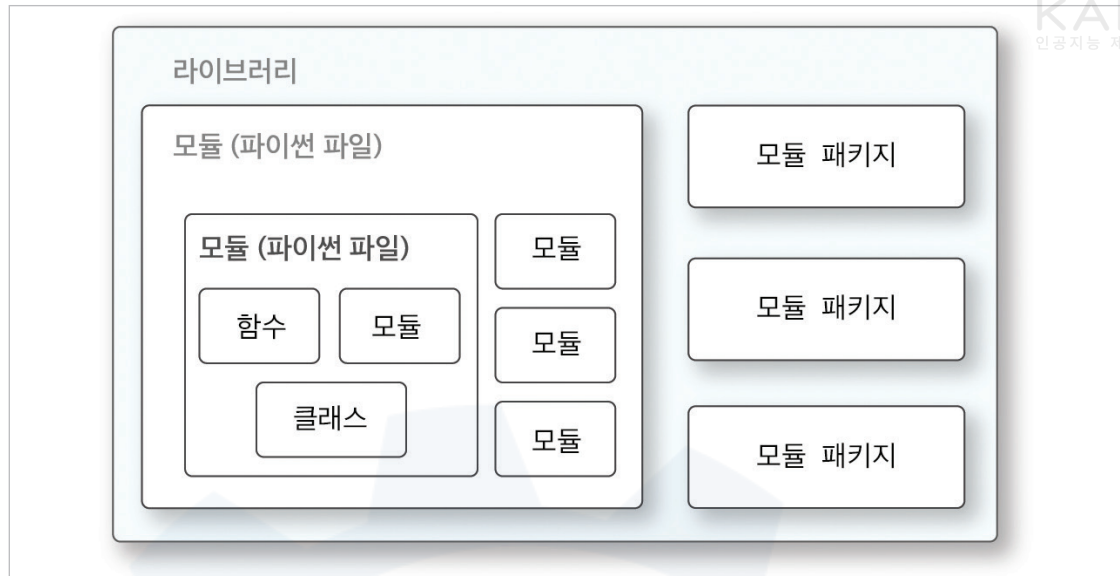
- 가) Matplotlib Python 프로그래밍 언어 및 수학적 확장 NumPy 라이브러리를 활용한 플로팅 라이브러리다.
- 나) Seaborn 은 Matplotlib 의 상위 호환 데이터 시각화를 위한 라이브러리다.

4) 경고 방지

- 가) Warnings 모듈은 Python에서 발생하는 다양한 경고를 무시 할 수 있도록 하는 라이브러리다.

여기서 Tip. 그렇다면 라이브러리, 패키지, 모듈의 차이점은 뭘까?

아래의 그림과 같이 표현 할 수 있다.



[그림 49] 라이브러리, 패키지, 모듈

①-3. 분석용 데이터 불러오기

1) 데이터 분석을 위해서 분석용 데이터를 불러온다.

열고자 하는 파일이 내 경로에 없다면 데이터를 로드 할 때 전체 경로(절대 경로) 혹은 부분 경로(상대 경로)를 불러와야 데이터를 불러올 수 있다.

tip) 절대 경로와 상대경로의 차이

- 절대 경로 : 파일이 가지고 있는 고유한 경로(주소)

예) 'C:\\data\\5공정_180sec\\kemp-abh-sensor-2021.09.06.csv'

Index	Process	Time	Temp	Current	Date
0	1	1 오후 4:24:03.0	75.139142	1.610	2021-09-06
1	2	1 오후 4:24:08.0	76.660421	1.530	2021-09-06
2	3	1 오후 4:24:13.0	77.177680	1.701	2021-09-06
3	4	1 오후 4:24:18.0	76.586434	1.736	2021-09-06
4	5	1 오후 4:24:23.0	77.877104	1.748	2021-09-06

[코드 3] 절대 경로

- 상대 경로 : 현재 폴더를 기준으로 한 경로

예) '.\\data\\5공정_180sec\\kemp-abh-sensor-2021.09.06.csv'

- (1) ./ : 파일이 현재 디렉토리
- (2) ../ : 상위 디렉토리
- (3) ../../ : 상.상위 디렉토리

	Index	Process	Time	Temp	Current	Date
0	1	1	오후 4:24:03.0	75.139142	1.610	2021-09-08
1	2	1	오후 4:24:08.0	76.660421	1.530	2021-09-08
2	3	1	오후 4:24:13.0	77.177660	1.701	2021-09-08
3	4	1	오후 4:24:18.0	76.586434	1.736	2021-09-08
4	5	1	오후 4:24:23.0	77.877104	1.748	2021-09-08

[코드 4] 상대 경로

2) 병합 데이터 불러오기

데이터를 잘 확인하기 위해서는 각각의 csv를 병합하여 하나의 데이터로 병합한 후 데이터의 정보나 특성을 확인해야 한다.

가) 현재 내 파일 경로를 확인 - `os.getcwd()`

```
#현재 위치
os.getcwd()
```

[코드 5] 실행 파일의 경로

나) 데이터 파일 위치 확인

- 파일을 실행하는 경로와 데이터를 불러오는 경로는 서로 다르다. 데이터를 불러오기 위해서는 데이터가 있는 경로에서 불러와야 한다.
- 현재 데이터가 있는 경로는 아래의 경로다.

```
C:\jupyter_gpu\data\5공정_180sec
```

[코드 6] 경로(절대경로)

Error Lot list.csv	2021-11-11 오후 5:48	Microsoft Excel ...	2KB
kemp-abh-sensor-2021.09.06.csv	2021-11-18 오후 4:26	Microsoft Excel ...	97KB
kemp-abh-sensor-2021.09.07.csv	2021-11-18 오후 4:26	Microsoft Excel ...	96KB
kemp-abh-sensor-2021.09.08.csv	2021-11-18 오후 4:26	Microsoft Excel ...	94KB
kemp-abh-sensor-2021.09.09.csv	2021-11-18 오후 4:26	Microsoft Excel ...	98KB
kemp-abh-sensor-2021.09.10.csv	2021-11-18 오후 4:26	Microsoft Excel ...	96KB
kemp-abh-sensor-2021.09.13.csv	2021-11-18 오후 4:26	Microsoft Excel ...	96KB
kemp-abh-sensor-2021.09.14.csv	2021-11-18 오후 4:26	Microsoft Excel ...	94KB
kemp-abh-sensor-2021.09.15.csv	2021-11-18 오후 4:26	Microsoft Excel ...	96KB
kemp-abh-sensor-2021.09.16.csv	2021-11-18 오후 4:26	Microsoft Excel ...	96KB
kemp-abh-sensor-2021.09.17.csv	2021-11-18 오후 4:26	Microsoft Excel ...	98KB
kemp-abh-sensor-2021.09.23.csv	2021-11-18 오후 4:26	Microsoft Excel ...	96KB
kemp-abh-sensor-2021.09.24.csv	2021-11-18 오후 4:26	Microsoft Excel ...	94KB
kemp-abh-sensor-2021.09.27.csv	2021-11-18 오후 4:26	Microsoft Excel ...	96KB
kemp-abh-sensor-2021.09.28.csv	2021-11-18 오후 4:26	Microsoft Excel ...	94KB
kemp-abh-sensor-2021.09.29.csv	2021-11-18 오후 4:26	Microsoft Excel ...	95KB
kemp-abh-sensor-2021.09.30.csv	2021-11-18 오후 4:26	Microsoft Excel ...	99KB
kemp-abh-sensor-2021.10.01.csv	2021-11-18 오후 4:26	Microsoft Excel ...	96KB
kemp-abh-sensor-2021.10.05.csv	2021-11-18 오후 4:26	Microsoft Excel ...	96KB
kemp-abh-sensor-2021.10.06.csv	2021-11-18 오후 4:26	Microsoft Excel ...	96KB
kemp-abh-sensor-2021.10.07.csv	2021-11-18 오후 4:26	Microsoft Excel ...	96KB
kemp-abh-sensor-2021.10.08.csv	2021-11-18 오후 4:26	Microsoft Excel ...	97KB
kemp-abh-sensor-2021.10.12.csv	2021-11-18 오후 4:26	Microsoft Excel ...	96KB
kemp-abh-sensor-2021.10.13.csv	2021-11-18 오후 4:26	Microsoft Excel ...	97KB
kemp-abh-sensor-2021.10.14.csv	2021-11-18 오후 4:26	Microsoft Excel ...	94KB
kemp-abh-sensor-2021.10.15.csv	2021-11-18 오후 4:26	Microsoft Excel ...	94KB
kemp-abh-sensor-2021.10.18.csv	2021-11-18 오후 4:26	Microsoft Excel ...	97KB
kemp-abh-sensor-2021.10.19.csv	2021-11-18 오후 4:26	Microsoft Excel ...	97KB
kemp-abh-sensor-2021.10.20.csv	2021-11-18 오후 4:26	Microsoft Excel ...	95KB
kemp-abh-sensor-2021.10.21.csv	2021-11-18 오후 4:26	Microsoft Excel ...	94KB
kemp-abh-sensor-2021.10.22.csv	2021-11-18 오후 4:26	Microsoft Excel ...	94KB
kemp-abh-sensor-2021.10.25.csv	2021-11-18 오후 4:26	Microsoft Excel ...	94KB
kemp-abh-sensor-2021.10.26.csv	2021-11-18 오후 4:26	Microsoft Excel ...	97KB
kemp-abh-sensor-2021.10.27.csv	2021-11-18 오후 4:26	Microsoft Excel ...	94KB

[코드 7] 경로 내부의 파일 리스트

– os.path.join 함수를 이용해 데이터가 있는 경로를 조합한다.

```
new_path = os.path.join(os.getcwd(), 'data')
new_path

# 열고자 하는 파일과 그 파일의 경로를 입력
pathName = os.path.join(new_path, '5공정_180sec')
pathName

'C:\\jupyter_gpu\\data\\5공정_180sec'
```

[코드 8] 데이터의 경로 조합

– pathName이라는 변수에 저장된 경로를 저장해 둔 후 경로가 저장된 이 변수를 이용하여 데이터를 불러와야 한다.

```
glob.glob(pathName+'/*')
```

```

C:\www\jupyter_gpu\dat\aw55060180sec\Error Lot_list.csv,
C:\www\jupyter_gpu\dat\aw55060180sec\kemp-abh-sensor-2021_09_06.csv,
C:\www\jupyter_gpu\dat\aw55060180sec\kemp-abh-sensor-2021_09_07.csv,
C:\www\jupyter_gpu\dat\aw55060180sec\kemp-abh-sensor-2021_09_08.csv,
C:\www\jupyter_gpu\dat\aw55060180sec\kemp-abh-sensor-2021_09_09.csv,
C:\www\jupyter_gpu\dat\aw55060180sec\kemp-abh-sensor-2021_09_10.csv,
C:\www\jupyter_gpu\dat\aw55060180sec\kemp-abh-sensor-2021_09_13.csv,
C:\www\jupyter_gpu\dat\aw55060180sec\kemp-abh-sensor-2021_09_14.csv,
C:\www\jupyter_gpu\dat\aw55060180sec\kemp-abh-sensor-2021_09_15.csv,
C:\www\jupyter_gpu\dat\aw55060180sec\kemp-abh-sensor-2021_09_16.csv,
C:\www\jupyter_gpu\dat\aw55060180sec\kemp-abh-sensor-2021_09_17.csv,
C:\www\jupyter_gpu\dat\aw55060180sec\kemp-abh-sensor-2021_09_23.csv,
C:\www\jupyter_gpu\dat\aw55060180sec\kemp-abh-sensor-2021_09_24.csv,
C:\www\jupyter_gpu\dat\aw55060180sec\kemp-abh-sensor-2021_09_27.csv,
C:\www\jupyter_gpu\dat\aw55060180sec\kemp-abh-sensor-2021_09_28.csv,
C:\www\jupyter_gpu\dat\aw55060180sec\kemp-abh-sensor-2021_09_29.csv,
C:\www\jupyter_gpu\dat\aw55060180sec\kemp-abh-sensor-2021_09_30.csv,
C:\www\jupyter_gpu\dat\aw55060180sec\kemp-abh-sensor-2021_10_01.csv,
C:\www\jupyter_gpu\dat\aw55060180sec\kemp-abh-sensor-2021_10_05.csv,
C:\www\jupyter_gpu\dat\aw55060180sec\kemp-abh-sensor-2021_10_06.csv,
C:\www\jupyter_gpu\dat\aw55060180sec\kemp-abh-sensor-2021_10_07.csv,
C:\www\jupyter_gpu\dat\aw55060180sec\kemp-abh-sensor-2021_10_08.csv,
C:\www\jupyter_gpu\dat\aw55060180sec\kemp-abh-sensor-2021_10_12.csv,
C:\www\jupyter_gpu\dat\aw55060180sec\kemp-abh-sensor-2021_10_13.csv,
C:\www\jupyter_gpu\dat\aw55060180sec\kemp-abh-sensor-2021_10_14.csv,
C:\www\jupyter_gpu\dat\aw55060180sec\kemp-abh-sensor-2021_10_15.csv,
C:\www\jupyter_gpu\dat\aw55060180sec\kemp-abh-sensor-2021_10_18.csv,
C:\www\jupyter_gpu\dat\aw55060180sec\kemp-abh-sensor-2021_10_19.csv,
C:\www\jupyter_gpu\dat\aw55060180sec\kemp-abh-sensor-2021_10_20.csv,
C:\www\jupyter_gpu\dat\aw55060180sec\kemp-abh-sensor-2021_10_21.csv,
C:\www\jupyter_gpu\dat\aw55060180sec\kemp-abh-sensor-2021_10_22.csv,
C:\www\jupyter_gpu\dat\aw55060180sec\kemp-abh-sensor-2021_10_25.csv,
C:\www\jupyter_gpu\dat\aw55060180sec\kemp-abh-sensor-2021_10_26.csv,
C:\www\jupyter_gpu\dat\aw55060180sec\kemp-abh-sensor-2021_10_27.csv,

```

[코드 9] 데이터의 경로 조합

* 참고로 '*'은 모든 것이라는 의미

- 데이터를 병합하기 전 여러 데이터는 제외하고 병합한다. 여러 데이터를 제외하는 이유는 아래와 같다.

	0	1	2	3	4	5	6	7		A	B	C	D	E	F	G		A	B	C	D	E	F	U
										Index	Process	Time	Temp	Current	Date			Index	Process	Time	Temp	Current	Date	
2021-09-06		32	33	20	21	22	31			1	1 号	4:24	75.13914	1.61	2021-09-06			1	1 号	4:23	76.00785	1.518	2021-09-07	
2021-09-07		32	33	34						2	1 号	4:24	76.66042	1.53	2021-09-06			2	1 号	4:23	74.11938	1.499	2021-09-07	
2021-09-08										3	1 号	4:24	77.17766	1.701	2021-09-06			3	1 号	4:23	74.12181	1.721	2021-09-07	
										4	1 号	4:24	76.58643	1.736	2021-09-06			4	1 号	4:23	74.06198	1.678	2021-09-07	
2021-09-09	15	16	17	21	22	23	29			5	1 号	4:24	77.87771	1.748	2021-09-06			5	1 号	4:23	73.0865	1.678	2021-09-07	
2021-09-10	27	28	29	30	31					6	1 号	4:24	76.01783	1.773	2021-09-06			6	1 号	4:23	73.25678	1.677	2021-09-07	
2021-09-13	32	28	29							7	1 号	4:24	71.00785	1.763	2021-09-06			7	1 号	4:23	72.13914	1.655	2021-09-07	
										8	1 号	4:24	71.11938	1.721	2021-09-06			8	1 号	4:23	71.66042	1.659	2021-09-07	
2021-09-14										9	1 号	4:24	74.12181	1.608	2021-09-06			9	1 号	4:23	75.17766	1.633	2021-09-07	
										10	1 号	4:24	74.06198	1.61	2021-09-06			10	1 号	4:23	73.58643	1.623	2021-09-07	
2021-09-15	40	41	39							11	1 号	4:24	71.0865	1.678	2021-09-06			11	1 号	4:24	73.58643	1.633	2021-09-07	
2021-09-16	2	35	3	34	36					12	1 号	4:25	69.00785	1.609	2021-09-06			12	1 号	4:24	75.01353	1.687	2021-09-07	
										13	1 号	4:25	71.11938	1.658	2021-09-06			13	1 号	4:24	71.07109	1.65	2021-09-07	
2021-09-17	12	13	14	16	17	18	28			14	1 号	4:25	72.12181	1.562	2021-09-06			14	1 号	4:24	72.19334	1.698	2021-09-07	
2021-09-23	8	9	6	7						15	1 号	4:25	71.06198	1.61	2021-09-06			15	1 号	4:24	69.28386	1.607	2021-09-07	
2021-09-24										16	1 号	4:25	70.0865	1.632	2021-09-06			16	1 号	4:24	72.33945	1.635	2021-09-07	
										17	1 号	4:25	71.25678	1.609	2021-09-06			17	1 号	4:24	71.49934	1.568	2021-09-07	
2021-09-27	13	14	15							18	1 号	4:25	69.0849	1.587	2021-09-06			18	1 号	4:24	70.81609	1.638	2021-09-07	
2021-09-28										19	1 号	4:25	74.39229	1.752495	2021-09-06			19	1 号	4:24	69.39114	1.618	2021-09-07	
										20	1 号	4:25	68.94317	1.563	2021-09-06			20	1 号	4:24	68.66042	1.603	2021-09-07	
2021-09-29	10	11								21	1 号	4:25	70.55671	1.559	2021-09-06			21	1 号	4:24	71.17766	1.565	2021-09-07	
										22	1 号	4:25	69.93958	1.566	2021-09-06			22	1 号	4:25	70.58643	1.554	2021-09-07	
2021-09-30	32	1	2	3	22	23	24			23	1 号	4:26	69.13914	1.597	2021-09-06			23	1 号	4:25	72.87771	1.611	2021-09-07	
2021-10-01	17	18	19	20	21					24	1 号	4:26	68.66042	1.583	2021-09-06			24	1 号	4:25	73.01353	1.598	2021-09-07	
2021-10-05	40	38	39							25	1 号	4:26	70.17766	1.565	2021-09-06			25	1 号	4:25	68.20044	1.65	2021-09-07	
2021-10-06	18	19	20							26	1 号	4:26	67.58643	1.54	2021-09-06			26	1 号	4:25	69.87178	1.653	2021-09-07	
										27	1 号	4:26	68.87771	1.589	2021-09-06			27	1 号	4:25	68.49958	1.544	2021-09-07	
2021-10-07	40	38	39							28	1 号	4:26	66.04198	1.559	2021-09-06			28	1 号	4:25	69.86391	1.582	2021-09-07	
2021-10-08	41	42	43	18	19	20				29	1 号	4:26	66.31247	1.562	2021-09-06			29	1 号	4:25	67.90815	1.551	2021-09-07	
										30	1 号	4:26	65.53366	1.494	2021-09-06			30	1 号	4:25	70.6621	1.581	2021-09-07	
2021-10-12	34	35	36							31	1 号	4:26	66.74042	1.501	2021-09-06			31	1 号	4:26	67.13914	1.537	2021-09-07	
2021-10-13	35	36	37	39	40	41				32	1 号	4:26	68.9519	1.483	2021-09-06			32	1 号	4:25	67.66042	1.599	2021-09-07	
										33	1 号	4:26	67.47249	1.51	2021-09-06			33	1 号	4:25	65.17766	1.51	2021-09-07	
2021-10-14	37	2	22	75.20044	1.54	2021-09-06				34	2 号	4:27	65.58643	1.532	2021-09-07			34	1 号	4:26	65.58643	1.532	2021-09-07	
										35	2 号	4:27	76.87178	1.527	2021-09-06			35	1 号	4:26	67.87771	1.475	2021-09-07	
2021-10-15										36	2 号	4:27	76.49958	1.703	2021-09-06			36	1 号	4:26	68.01353	1.494	2021-09-07	

[코드 10] 통합 날짜 에러 데이터 세트

[코드 11] 2021-09-06자 데이터 세트

[코드 12] 2021-09-07자 데이터 세트

- 데이터들을 보면 구조가 다른 것을 알 수 있다. 같은 구조를 가진 데이터끼리 모아 병합을 한 후 여러 데이터 세트는 별도로 병합을 진행할 것이다.

```
dataList = list()
cnt = 0

for pth in glob.glob(pathName+'/*')[1:]: # 에러 데이터를 제외하고 병합
    origin = pd.read_csv(pth, index_col=False) # read_csv를 이용해 파일을 읽어서 변수에 담는다
    origin['Date'] = "-".join(pth.split('-')[-1].split('.')[:-1]) # 날짜 데이터 추가
    cnt = cnt + len(origin) # 데이터 검증용 row 누적코드
    dataList.append(origin)
```

[코드 13] 데이터 병합 코드

- 앞서 봤던 `glob.glob(pathName+'/*')` 함수다. 모든 경로를 호출한 것은 아니다. 이전에 파일 경로 파트에서 정상 파일과 에러 파일과 함께 담긴 것을 확인했다. 그 에러 파일을 제외하고 병합을 진행한다.

- `pd.read_csv(path, index_col=False)` 코드를 이용해 파일 각각의 데이터(DataFrame)를 호출한다.

- 데이터 분석을 할 때 다량의 데이터 파일이 존재할 경우 대부분은 파일 이름에 또 다른 특징을 가진 경우들이 있다. 해당 코드에서는 그 특징은 날짜이며 그 날짜를 추출하는 작업을 추가로 진행한다.

```
origin['Date'] = "-".join(path.split('-')[-1].split('.')[:-1])
```

*cnt는 그냥 데이터 개수를 직관적으로 확인하기 위한 의미 없는 코드

```
CatList= pd.concat(dataList,axis=0, ignore_index=True)
CatList
```

	Index	Process	Time	Temp	Current	Date
0	1	1	오후 4:24:03.0	75.139142	1.610	2021-09-06
1	2	1	오후 4:24:08.0	76.660421	1.530	2021-09-06
2	3	1	오후 4:24:13.0	77.177680	1.701	2021-09-06
3	4	1	오후 4:24:18.0	76.586434	1.736	2021-09-06
4	5	1	오후 4:24:23.0	77.877104	1.748	2021-09-06
...	...	...	...	...	...	...
51079	1544	43	오후 6:45:03.4	67.312474	1.560	2021-10-27
51080	1545	43	오후 6:45:08.4	65.533664	1.472	2021-10-27
51081	1546	43	오후 6:45:13.4	66.740424	1.523	2021-10-27
51082	1547	43	오후 6:45:18.4	68.045185	1.483	2021-10-27
51083	1548	43	오후 6:45:23.4	68.472490	1.511	2021-10-27

51084 rows × 6 columns

[코드 14] 판다스의 concat 함수를 사용한 병합 결과

* axis는 0은 상하로 합칠 수 있고 1은 좌우로 가능하다. ignore_index는 기존 인덱스를 유지하느냐의 여부이다.

에러 데이터를 포함하고 병합했을 경우

	0	1	2	3	4	5	6	7	8	9	10	11		Date	Index	Process	Time	Temp	Current
0	2021-09-06	32.0	33.0	20.0	21.0	22.0	31.0	NaN	NaN	NaN	NaN	NaN	C:\jupyter_gpu\data\5공정_180sec\Error Lot list	NaN	NaN	NaN	NaN	NaN	NaN
1	2021-09-07	32.0	33.0	34.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	C:\jupyter_gpu\data\5공정_180sec\Error Lot list	NaN	NaN	NaN	NaN	NaN	NaN
2	2021-09-08	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	C:\jupyter_gpu\data\5공정_180sec\Error Lot list	NaN	NaN	NaN	NaN	NaN	NaN
3	2021-09-09	15.0	16.0	17.0	21.0	22.0	23.0	29.0	30.0	31.0	NaN	NaN	C:\jupyter_gpu\data\5공정_180sec\Error Lot list	NaN	NaN	NaN	NaN	NaN	NaN
4	2021-09-10	32.0	28.0	29.0	30.0	31.0	NaN	NaN	NaN	NaN	NaN	NaN	C:\jupyter_gpu\data\5공정_180sec\Error Lot list	NaN	NaN	NaN	NaN	NaN	NaN
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
51112	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN		2021-10-27	1544.0	43.0	오후 6:45:03.4	67.312474	1.560
51113	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN		2021-10-27	1545.0	43.0	오후 6:45:08.4	65.533664	1.472
51114	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN		2021-10-27	1546.0	43.0	오후 6:45:13.4	66.740424	1.523
51115	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN		2021-10-27	1547.0	43.0	오후 6:45:18.4	68.045185	1.483
51116	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN		2021-10-27	1548.0	43.0	오후 6:45:23.4	68.472490	1.511

[코드 15] 에러 데이터를 포함하고 병합했을 경우

[단계 ②] 데이터 탐색

②-1. 데이터 사본 생성

병합한 데이터의 사본을 생성해 이 사본으로 작업을 진행한다.

앞서 절차 파트에서 언급했다시피 데이터는 데이터의 훼손을 막기 위해서 원본에서 작업하는 것보다 사본에서 작업하는 것을 추천한다.

복사된 데이터는 원본과 100% 동일한 구조다.

```
df = CatList.copy()
```

	Index	Process	Time	Temp	Current	Date
0	1	1	오후 4:24:03.0	75.139142	1.610	2021-09-06
1	2	1	오후 4:24:08.0	76.680421	1.530	2021-09-06
2	3	1	오후 4:24:13.0	77.177680	1.701	2021-09-06
3	4	1	오후 4:24:18.0	76.586434	1.736	2021-09-06
4	5	1	오후 4:24:23.0	77.877104	1.748	2021-09-06
...	...	...	...	...	...	...
51079	1544	43	오후 6:45:03.4	67.312474	1.560	2021-10-27
51080	1545	43	오후 6:45:08.4	65.533664	1.472	2021-10-27
51081	1546	43	오후 6:45:13.4	66.740424	1.523	2021-10-27
51082	1547	43	오후 6:45:18.4	68.045185	1.483	2021-10-27
51083	1548	43	오후 6:45:23.4	68.472490	1.511	2021-10-27

51084 rows × 6 columns

[코드 16] 데이터 사본 생성 코드

②-2. 데이터 레이블링 확인

- DataFrame.columns를 입력하면 각각의 데이터의 레이블(Column)을 확인 할 수 있다. 상황에 따라 식별이 어려운 데이터의 레이블을 변경 해 주는데 아래의 데이터는 데이터의 특징에 맞게 잘 만들어진 것을 확인할 수 있다.

```
df.columns
Index(['Index', 'Process', 'Time', 'Temp', 'Current', 'Date'], dtype='object')
```

[코드 17] 데이터 컬럼(특성) 확인 코드

②-3. 데이터 유형 확인

- DataFrame.info() 함수를 사용하면 데이터 각각의 속성 정보를 간단하게 확인할 수 있다. Column명, 결측치의 여부 그리고 데이터 유형을 간단하게 확인할 수 있다.

```
df.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 51084 entries, 0 to 51083
Data columns (total 6 columns):
#   Column      Non-Null Count  Dtype
---  ---
0    Index      51084 non-null  int64
1    Process    51084 non-null  int64
2    Time       51084 non-null  object
3    Temp       51084 non-null  float64
4    Current    51084 non-null  float64
5    Date       51084 non-null  object
dtypes: float64(2), int64(2), object(2)
memory usage: 2.3+ MB
```

[코드 18] 데이터 정보 확인

②-4. 데이터 비율 확인

현재는 정상 데이터만 존재하며 DataFrame의 정보로 기본적인 정보는 다 제공되어 있다. 또한, 데이터 정보에서 언급된 것처럼 데이터의 결측값은 존재하지 않는다.

```
df = df[['Date', 'Process', 'Time', 'Temp', 'Current']]

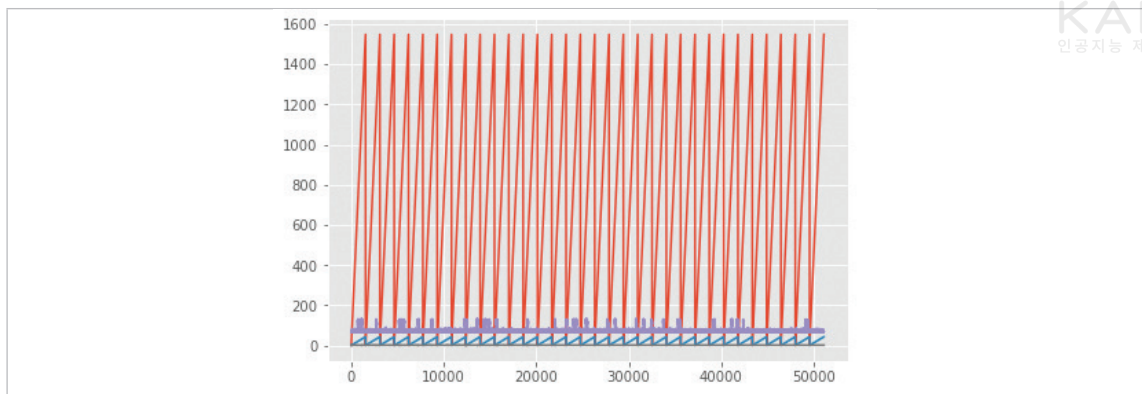
for col in df.columns:
    percent = df[col].isnull().sum() / df[col].shape[0]
    msg='col:{>20}\t Percent of NaN value: {:.2f}%'.format(col,100 * (percent))
    print(msg)
```

col :	Date	Percent of NaN value: 0,00%
col :	Process	Percent of NaN value: 0,00%
col :	Time	Percent of NaN value: 0,00%
col :	Temp	Percent of NaN value: 0,00%
col :	Current	Percent of NaN value: 0,00%

[코드 19] 데이터 결측 정보 확인

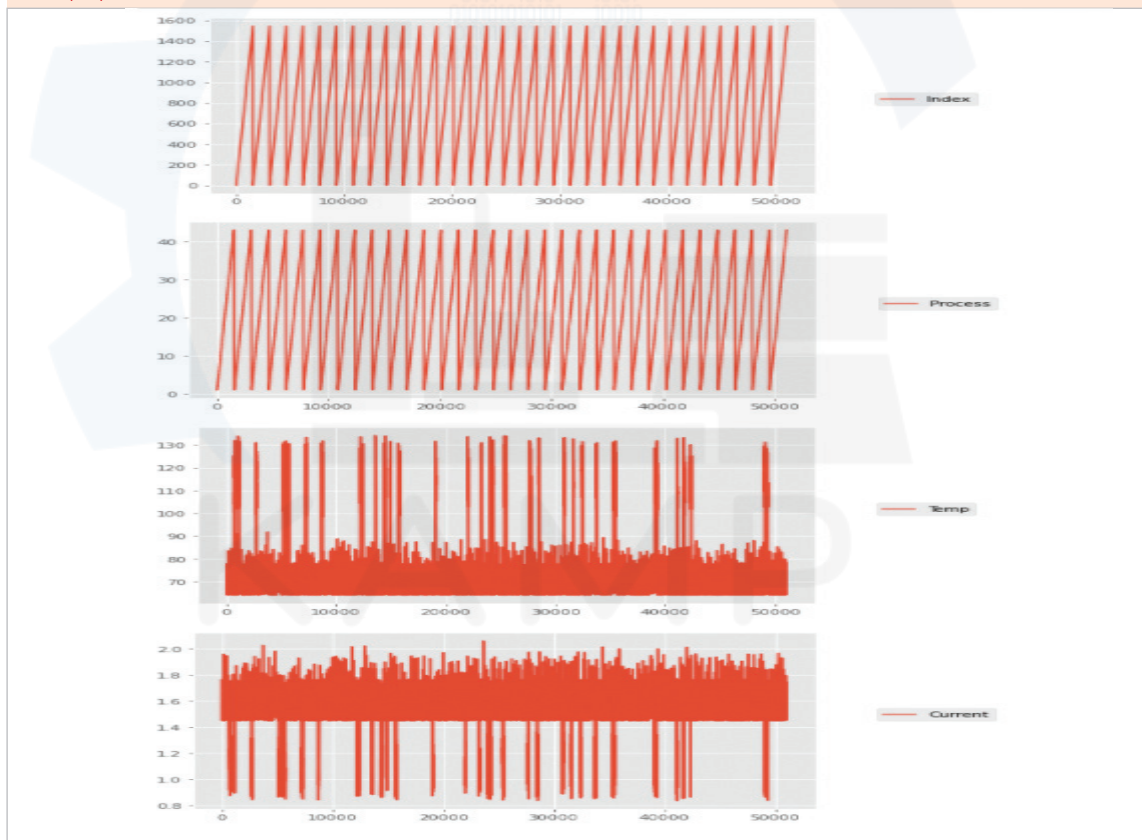
*[3] 데이터 정제 - 데이터 특성을 기반으로 이상치를 수정 혹은 제거 파트에서 확인

②-5. 데이터 분포 확인



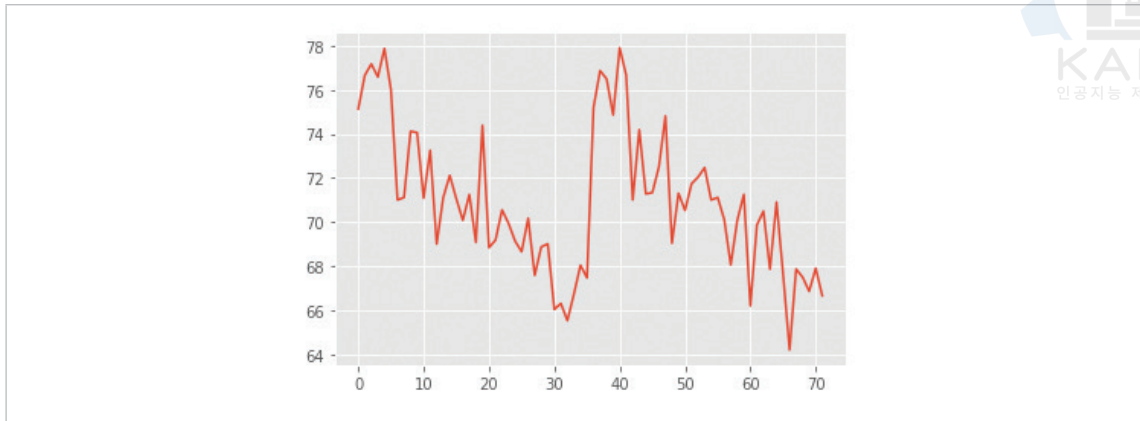
[코드 20] 전체 데이터 분포

```
for i in df.columns:
    try:
        df[i].plot()
        plt.tight_layout()
        plt.show()
    except: pass
```



[코드 21] 각 데이터별 코드와 그래프 데이터

- 전체적인 그래프만 확인해보아도 해당 데이터가 연속성을 가지는 데이터라는 것을 확인할 수 있다.
- Process 변수 같은 경우는 lot 작업을 나타내며 36개로 나누어져 있다.



[코드 22] 72개의 데이터를 라인 그래프로 시각화

위와 같이 온도 데이터의 일정 패턴을 확인 할 수 있다.

②-6. 데이터 간의 상관관계 파악

- 상관 분석(Correlation analysis)이란, 두 변수 간의 관계를 확인하여 그 관계가 선형적 혹은 비선형적 관계가 있는지를 확인하는 분석이다. 주로 상관 계수의 여부를 확인할 경우 많이 사용한다.

- 앞으로 진행할 상관 분석에 사용하게 될 변수는 전압과 온도 데이터이다.

- 우선 상관 분석에 사용하고자 하는 데이터를 추출한다.

```
#데이터 추출
pick_data= df[df.columns[3:]]
pick_data.head(1)
```

	Temp	Current
0	75.139142	1.61

[코드 23] 변수 데이터 추출

* DataFrame.head()를 하면 기본으로 1번 index에서 4번까지 보여준다. 하지만 간단하게 잘 추출되었는지를 확인 하고자 했기 때문에 가장 위에서 1번 index만 추출한다.

- 변수를 추출한 후 상관 계수를 구하는 DataFrame.corr() 함수를 이용하여 상관 계수를 구한다.

```
# 상관 계수
corr = pick_data.corr()
corr
```

	Temp	Current
0	75.139142	1.61

[코드 24] 상관 계수 추출

Temp와 Current 데이터는 음의 상관관계를 가지며 그중 강한 음의 상관관계를 가진다고 볼 수 있다.

```
sns.heatmap(corr, annot=True, cmap='Greens', annot_kws={'size':20})  
plt.show()
```



[코드 25] 상관 계수 시각화

- 위 두 상관관계는 아래와 같이 방법으로 해석 할 수 있다. 두 변수 간의 선형적 정도를 -1 에서 1 사이의 값으로 나타낸다.

* r(상관 계수)

〈〈r이 양, 음의 값을 가질 때〉〉

- 두 변수가 동시에 증가하는 경우 혹은 비례하는 경우 r이 1에 가까울수록 두 변수 간의 관계가 강하다고 할 수 있다.
- 음수이냐 양수이냐에 따라서 음의 상관관계를 가진다, 혹은 양의 상관관계를 가진다고 할 수 있다.

r 값이 ± 0.1 이면, 없다고 할 수 있는 상관관계
 r 값이 $\pm 0.1 \sim \pm 0.3$ 이면, 약한 양적 상관관계
 r 값이 $\pm 0.3 \sim \pm 0.7$ 이면, 양적 상관관계
 r 값이 ± 0.7 이상이면, 강한 양적 상관관계

[단계 ③] 데이터 정제(전처리)

- 이상치 처리 혹은 결측치를 처리하기 위해서 에러 데이터 세트를 불러와 진행한다.

```
df_er = pd.read_csv(glob.glob(pathName+'/*')[0], index_col=False)
df_er.head()
```

	0	1	2	3	4	5	6	7	8	9	10	11
0	2021-09-06	32.0	33.0	20.0	21.0	22.0	31.0	NaN	NaN	NaN	NaN	NaN
1	2021-09-07	32.0	33.0	34.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
2	2021-09-08	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
3	2021-09-09	15.0	16.0	17.0	21.0	22.0	23.0	29.0	30.0	31.0	NaN	NaN
4	2021-09-10	32.0	28.0	29.0	30.0	31.0	NaN	NaN	NaN	NaN	NaN	NaN

[코드 26] 에러 데이터 확인

- 결측치(Missing Value)란, Missing feature 혹은 Missing Value라고하며 말 그대로 누락된 값을 의미한다. 보통은 데이터 분석을 진행하면서 데이터 전처리 과정을 거치는데 이때 데이터 탐색을 하면서 결측치의 여부를 확인하는 작업을 거치고 정제 부분에서 제거하게 된다.

- DataFrame의 전체 널값 확인 : `df.isnull().sum()`
- 결측치인 아이만 출력 : `df[df.'Column이름'.isnull()==True]`

- 불러온 데이터세트를 확인해보면 결측값이 상당히 많은 것을 확인 할 수 있다. 해당 DataFrame을 회전시키면 좀 더 쉽게 데이터를 사용 수 있을 것 같으므로 `DataFrame.transpose()`을 이용해 DataFrame을 회전하는 작업을 진행한다.

- 참고로 `DataFrame.T()`를 사용해도 동일하게 회전이 가능하다.

```
df_err = df_er.transpose() # 전치
df_err
```

	0	1	2	3	4	5	6	7	8	9	...
0	2021-09-06	2021-09-07	2021-09-08	2021-09-09	2021-09-10	2021-09-13	2021-09-14	2021-09-15	2021-09-16	2021-09-17	...
1	32.0	32.0	NaN	15.0	32.0	27.0	NaN	40.0	2.0	12.0	...
2	33.0	33.0	NaN	16.0	28.0	28.0	NaN	41.0	35.0	13.0	...
3	20.0	34.0	NaN	17.0	29.0	29.0	NaN	39.0	3.0	14.0	...
4	21.0	NaN	NaN	21.0	30.0	NaN	NaN	NaN	34.0	16.0	...
5	22.0	NaN	NaN	22.0	31.0	NaN	NaN	NaN	36.0	17.0	...
6	31.0	NaN	NaN	23.0	NaN	NaN	NaN	NaN	NaN	18.0	...
7	NaN	NaN	NaN	29.0	NaN	NaN	NaN	NaN	NaN	28.0	...
8	NaN	NaN	NaN	30.0	NaN	NaN	NaN	NaN	NaN	29.0	...
9	NaN	NaN	NaN	31.0	NaN	NaN	NaN	NaN	NaN	NaN	...
10	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...
11	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...

12 rows x 33 columns

[코드 27] 상관 계수 시각화

- 해당 데이터 세트를 좀 더 사용하기 편리하게 추가로 처리작업을 진행한다. 우선, 0번 Index를 Column 이름에 대치시킨 후 Index 0번을 제외한 값을 다시 저장할 예정이다.

*아쉽게도 필자는 제거하는 것보다 사본으로 생성하여 작업하고 사본을 저장하는 것으로 선호하기 때문에 제거에 관한 설명은 하겠지만 특이사항이 없을 경우 바로 사본화 시켜 진행할 것이다.

```
df_err.columns = df_err.iloc[0] # 데이터 컬럼 이름 변경
df_err
```

0	2021-09-06	2021-09-07	2021-09-08	2021-09-09	2021-09-10	2021-09-13	2021-09-14	2021-09-15	2021-09-16	2021-09-17	...
0	2021-09-06	2021-09-07	2021-09-08	2021-09-09	2021-09-10	2021-09-13	2021-09-14	2021-09-15	2021-09-16	2021-09-17	...
1	32.0	32.0	NaN	15.0	32.0	27.0	NaN	40.0	2.0	12.0	...
2	33.0	33.0	NaN	16.0	28.0	28.0	NaN	41.0	35.0	13.0	...
3	20.0	34.0	NaN	17.0	29.0	29.0	NaN	39.0	3.0	14.0	...
4	21.0	NaN	NaN	21.0	30.0	NaN	NaN	NaN	34.0	16.0	...
5	22.0	NaN	NaN	22.0	31.0	NaN	NaN	NaN	36.0	17.0	...
6	31.0	NaN	NaN	23.0	NaN	NaN	NaN	NaN	NaN	18.0	...
7	NaN	NaN	NaN	29.0	NaN	NaN	NaN	NaN	NaN	28.0	...
8	NaN	NaN	NaN	30.0	NaN	NaN	NaN	NaN	NaN	29.0	...
9	NaN	NaN	NaN	31.0	NaN	NaN	NaN	NaN	NaN	NaN	...
10	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...
11	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...

12 rows x 33 columns

[코드 28] 에러 데이터 컬럼명 변경

여기서 Tip. loc와 iloc.

loc(location)는 DataFrame에서 행이나 Column의 레이블로 접근하는 것이고 iloc(integer location)는 DataFrame에서 Row나 Column의 Index 값(순서)으로 접근하는 것이다. 두 가지 모두 데이터를 사본화 하는 주된 함수들로 활용하면 좋지만 캐시나 메모리에서 과 부하가 일어날 수가 있으므로 남발은 주의하는 게 좋다.

```
df_err = df_err.iloc[1 :,:] # 필요 없는 row 제거
df_err
```

0	2021-09-06	2021-09-07	2021-09-08	2021-09-09	2021-09-10	2021-09-13	2021-09-14	2021-09-15	2021-09-16	2021-09-17	...
1	32.0	32.0	NaN	15.0	32.0	27.0	NaN	40.0	2.0	12.0	...
2	33.0	33.0	NaN	16.0	28.0	28.0	NaN	41.0	35.0	13.0	...
3	20.0	34.0	NaN	17.0	29.0	29.0	NaN	39.0	3.0	14.0	...
4	21.0	NaN	NaN	21.0	30.0	NaN	NaN	NaN	34.0	16.0	...
5	22.0	NaN	NaN	22.0	31.0	NaN	NaN	NaN	36.0	17.0	...
6	31.0	NaN	NaN	23.0	NaN	NaN	NaN	NaN	NaN	18.0	...
7	NaN	NaN	NaN	29.0	NaN	NaN	NaN	NaN	NaN	28.0	...
8	NaN	NaN	NaN	30.0	NaN	NaN	NaN	NaN	NaN	29.0	...
9	NaN	NaN	NaN	31.0	NaN	NaN	NaN	NaN	NaN	NaN	...
10	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...
11	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...

11 rows x 33 columns

[코드 29] 에러데이터 추출

③-1. 데이터 특성을 기반으로 이상치를 수정 혹은 제거

```
for col in df_err.columns:
    percent = df_err[col].isnull().sum() / df_err[col].shape[0]
    msg = 'column: {>20}\t Percent of NaN value: {:.2f}%'.format(col, 100 * (percent))
    print(msg)
```

column:	2021-09-06	Percent of NaN value:	45.45%
column:	2021-09-07	Percent of NaN value:	72.73%
column:	2021-09-08	Percent of NaN value:	100.00%
column:	2021-09-09	Percent of NaN value:	18.18%
column:	2021-09-10	Percent of NaN value:	54.55%
column:	2021-09-13	Percent of NaN value:	72.73%
column:	2021-09-14	Percent of NaN value:	100.00%
column:	2021-09-15	Percent of NaN value:	72.73%
column:	2021-09-16	Percent of NaN value:	54.55%
column:	2021-09-17	Percent of NaN value:	27.27%
column:	2021-09-23	Percent of NaN value:	63.64%
column:	2021-09-24	Percent of NaN value:	100.00%
column:	2021-09-27	Percent of NaN value:	72.73%
column:	2021-09-28	Percent of NaN value:	100.00%
column:	2021-09-29	Percent of NaN value:	81.82%
column:	2021-09-30	Percent of NaN value:	0.00%
column:	2021-10-01	Percent of NaN value:	54.55%
column:	2021-10-05	Percent of NaN value:	72.73%
column:	2021-10-06	Percent of NaN value:	72.73%
column:	2021-10-07	Percent of NaN value:	72.73%
column:	2021-10-08	Percent of NaN value:	45.45%
column:	2021-10-12	Percent of NaN value:	72.73%
column:	2021-10-13	Percent of NaN value:	45.45%
column:	2021-10-14	Percent of NaN value:	100.00%
column:	2021-10-15	Percent of NaN value:	100.00%
column:	2021-10-18	Percent of NaN value:	45.45%
column:	2021-10-19	Percent of NaN value:	45.45%
column:	2021-10-20	Percent of NaN value:	81.82%
column:	2021-10-21	Percent of NaN value:	100.00%
column:	2021-10-22	Percent of NaN value:	100.00%
column:	2021-10-25	Percent of NaN value:	100.00%
column:	2021-10-26	Percent of NaN value:	45.45%
column:	2021-10-27	Percent of NaN value:	100.00%

[코드 30] 에러데이터 결측치 확인

- 보통은 데이터에서 데이터를 함부로 제거하는 작업은 좋지 않다. 제거하는 것 자체로 정보의 손실을 얻기 때문이다. 그렇기 때문에 결측치도 제거의 조건이 존재한다.

결측치 비율로 보는 방법.

- 10% 미만인 경우 - 제거, 치환등 모든 방법이 사용 가능
- 10%에서 20%인 경우 - 과거 정보로 추정 혹은 예측, 대치
- 20%에서 50%미만인 경우 - 예측이나 대치 방법을 사용
- 50% 이상인 경우 - 컬럼(특성)을 제거 혹은 데이터 재수집

- 첫 번째로 결측치의 경우 제거와 삭제하는 방법이다.

제거에서도 두 가지로 분류가 되는데 row를 전부 제거하거나 열만 특정해서 제거해야 하는 경우가 있다. 특성 열만 제거하는 경우 다른 특징(Column)을 유지하고 작업을 진행 할 수 있다.

데이터 결측의 비율이 너무 높은 상태에서 그 결측치를 모두 제거하게 되면 DataFrame의 정보가 다 사라지기 때문에 주의하는 것이 좋다.

- 두 번째 방법으로는 대치 혹은 채우기 방법이다.

평균값 중앙값을 이용하여 결측치 부분을 메우는 것이다. 가장 보편적인 방법이다. 결측치를 빠르게 채울 수 있다는 장점이 있지만, 그 결측치가 유사 혹은 같은 값을 가지기 때문에 데이터 세트에 좋지 않은 영향을 줄 수가 있다.

특히 통계 분석의 경우 큰 타격을 입을 수도 있다.

- 마지막으로 예측기법을 통해서 예측하여 결측된 값을 예측하여 예측치 값으로 결측치를 채우는 것이다.

각각 다 장단점이 존재한다. 가장 좋은 것을 올바른 데이터를 추출하여 이상치도 결측치도 없는 것이 가장 좋겠지만 그것은 거의 불가능한 일이라고 본다.

앞으로 마주하는 수많은 결측치를 어떻게 핸들링 하느냐로 사실 데이터의 새로운 인사이트를 찾을 수도 있다. 아주 사소하다고 느낄 수 있는 이 일은 굉장히 중요하고 가장 필요한 작업임으로 가볍게 넘어가지 말고 꼭꼭 이해하고 넘어가야 한다.

③-2. 누락된 값을 채우거나 열 제거

- 현재 데이터는 이상치 제거 조건에 부합된다. 하지만 조건에 맞게 제거하게 되면 데이터의 절반 이상이 제거된다. 현재 상황은 NaN값을 모두 사용해야 하는 상황이다. 이럴 때는 단순히 0으로 대치하면 된다. 결측치를 사용하지 않고 사용은 가능하지만, 나중에 혹시라도 생길 문제를 방지하기 위해 전부 0으로 변경한다.

```
df_err = df_err.fillna(0)
df_err.head()
```

0	2021-09-06	2021-09-07	2021-09-08	2021-09-09	2021-09-10	2021-09-13	2021-09-14	2021-09-15	2021-09-16	2021-09-17	...
1	32.0	32.0	0	15.0	32.0	27.0	0	40.0	2.0	12.0	...
2	33.0	33.0	0	16.0	28.0	28.0	0	41.0	35.0	13.0	...
3	20.0	34.0	0	17.0	29.0	29.0	0	39.0	3.0	14.0	...
4	21.0	0.0	0	21.0	30.0	0.0	0	0.0	34.0	16.0	...
5	22.0	0.0	0	22.0	31.0	0.0	0	0.0	36.0	17.0	...

5 rows x 33 columns

[코드 31] 결측치 0으로 대치

- DataFrame.fillna(변경 할 값) 함수를 이용하면 [코드 44]와 같이 결측값을 0으로 변경할 수 있다.

③-3. 무의미한 특성 제거

- 무의미한 특성 사실 이 데이터 세트에는 딱히 없다. 다른 시각으로 보면 아래와 같이 정의할 수 있다.
- 현재 에러 데이터는 재가공을 해야 한다. 에러의 특성을 뽑아내서 정상 데이터와 병합하는 작업을 추가로 진행한다.

```
pick_data = df[df.columns[3:]]
pick_data.head(1)
```

	Temp	Current
0	75.139142	1.61

[코드 32] 컬럼 내용 확인

- 에러 데이터와 정상 데이터 세트는 형태가 다르다. 두 데이터의 형태를 같게 바꾸거나 아니면 특정 요소를 기준으로 특징만 추출하여 작업하는 것이다. 데이터를 유사하게 변경하는 작업은 앞에서 진행했기 때문에 앞으로 해야 하는 일은 특징만 추출해서 재가공하는 작업을 진행한다.
- 에러 데이터 세트에서 추출한 특징을 정상 데이터 세트에 NG 컬럼을 만든 후 추출한 특징을 에러 여부에 따라 0과 1로 변경하여 특성을 추가한다.
- 에러 Column에서 Column명을 뽑아내서 해당 날짜의 데이터만 추출한다.

```
df[df['Process'] == df_err['2021-09-06']][0]
```

	Index	Process	Time	Temp	Current	Date
50688	1153	33	오후 6:08:47.0	75.010939	1.495	2021-10-27
50689	1154	33	오후 6:08:52.0	75.193336	1.477	2021-10-27
50690	1155	33	오후 6:08:57.0	74.283864	1.699	2021-10-27
50691	1156	33	오후 6:09:02.0	74.339455	1.702	2021-10-27
50692	1157	33	오후 6:09:07.0	75.499337	1.682	2021-10-27
50693	1158	33	오후 6:09:12.0	75.816086	1.684	2021-10-27
50694	1159	33	오후 6:09:17.0	72.041983	1.710	2021-10-27
50695	1160	33	오후 6:09:22.0	71.312474	1.717	2021-10-27
50696	1161	33	오후 6:09:27.0	71.533864	1.631	2021-10-27
50697	1162	33	오후 6:09:32.0	72.740424	1.614	2021-10-27
50698	1163	33	오후 6:09:37.0	75.045185	1.619	2021-10-27

[코드 33] 일정 날짜 추출 테스트

- [코드 34] 같은 방식으로 Process 번호가 X인 36개 요소를 뽑아낸 후 single_feature 에 Process 번호가 저장 되게 된다. 그리고 이 번호가 0이 아니면 아래의 코드로 진입하게 된다.

- 날짜와 Process가 일치하면 해당하는 날짜의 Process의 모든 인덱스를 출력하고 pick_data의 NG Column 해당 Index 위치에 0 대신 1의 값을 추가한다.

```
pick_data['NG'] = 0
for dt in df_err.columns:
    pick_DATE = df[df.Date == dt]
    for i in range(len(df_err)): # len(df_err) : 11개 -> 한 날짜에 최대 에러 11개
        single_feature = df_err[dt][i] # 에러 루트 위치 검출
        if single_feature != 0 :
            c_inx = pick_DATE['Temp'][pick_DATE['Process'] == single_feature].index
            pick_data['NG'].loc[c_inx] = 1
        else : pass

pick_data['NG'].value_counts()
```

[코드 34] 일정 날짜 추출 테스트

[단계 ④] 알고리즘 선택

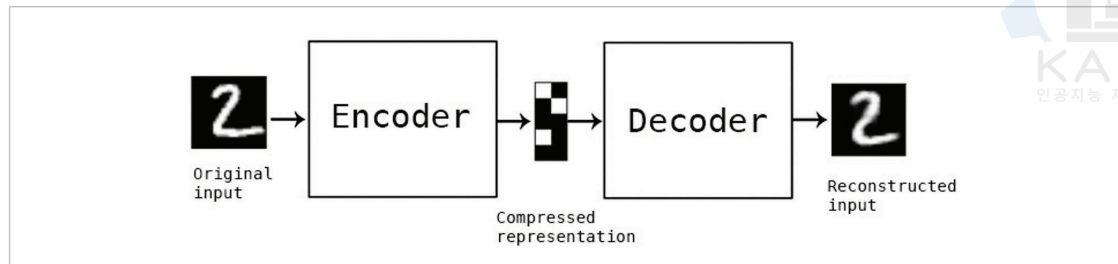
④-1. 데이터의 특성에 맞게 알고리즘(및 모델)을 선택

- 이전 과정들을 통해서 데이터를 탐색하고 데이터 특성을 간단하게 확인해보는 작업을 진행했다.
- 각각의 데이터들은 시계열성을 띄고 있으며 일정 패턴들이 존재한다. 보통 이런 경우에는 RNN이나, LSTM, Clustering, Classification 등의 모델을 사용 가능할 것으로 보인다.

④-2. 선택된 알고리즘 중 사용할 알고리즘 선택

- 앞으로 진행할 과제는 이상 탐지이기 때문에 LSTM 중에서 LSTM-AE 라는 모델을 이용하여 이상 탐지를 진행하고자 한다. 해당 모델은 딥러닝을 사용한 Anomaly Detection(설비 이상 탐지) 방법에 대표적으로 사용되는 모델이다.
 - 앞에서 모델을 소개할 때 RNN과 LSTM에 관해서 설명을 장대하게 하였다. LSTM-AE 모델에서 가장 기본이 되는 부분이기도 하고 무조건 알고 넘어와야 했기 때문에 알고리즘 설명을 분리하여 진행하는 것이다.
- 앞으로 진행할 설명은 기본적으로 LSTM에 대해 기본적인 지식이 있다고 판단하고 추가로 앞으로 우리가 사용할 모델이 무엇인지 LSTM-AE가 무엇인지 가볍게 언급하고 넘어가고자 한다.

- 이상 탐지란, 설비의 데이터를 통해 이상 패턴(이상 징후)를 찾아내는 것을 의미한다.
- 오토인코더란, 비지도 학습 중 하나로 목표값은 존재한다. 하지만 입력값이 같은 것을 말한다. 그 말은 모델의 인풋과 아웃풋이 같은 출력이라는 의미이다. 출력값과 입력값 x 를 유사하게 만들려고 하는 것이 바로 오토인코더다.



[그림 50] 오토 인코더 동작 방식

- 오토인코더를 사용하는 이유는 아래와 같다.

데이터 내부의 이상 데이터는 생각보다 충분하지 않다. 이러한 케이스는 데이터 불균형이라고 하는데 정상만 굉장히 많고 이상 데이터가 거의 없는 경우에 주로 이 방법을 사용한다.

이상 감지에는 Reconstruction Error라는 것을 사용하는데 모델 입력값과 모델 출력의 차이를 사용해서 스레드(임계값)을 정하고 이 임계값을 넘어서면 이상 데이터라고 정의한다. 정상인 경우에는 출력값은 입력과 매우 유사하게 출력되며 이상 데이터일 경우 확연한 차이를 확인할 수 있다.

- 이 모델이 최대의 효율을 발휘하는 경우가 정상 데이터만 굉장히 많은 경우 효과적인 성능을 발휘한다.
- 시계열 데이터의 경우는 시간적인 특성을 고려하기 때문에 lstm과 오토인코더를 결합한 lstm-AE를 시계열 특성을 가진 이상 탐지 모델에 주로 사용한다.

[단계 ⑤] 학습, 평가 데이터 준비

⑤-1. 데이터 분리

- 33일치의 데이터 중 10일을 제외하고 추출한다. 일일 데이터 43개, 1의 작업당 36개의 Lot, 10일치 테스트 데이터를 제외한 데이터 세트를 학습 데이터로 사용할 예정이다.

- $43(\text{일일데이터}) * 36(\text{lot 개수}) * 10(10\text{일치}) = 15480$
- 전체 데이터 - 15480 = 35604

```
train_data = pick_data[:35604]
train_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15480 entries, 35604 to 51083
Data columns (total 3 columns):
#   Column  Non-Null Count  Dtype
---  ---
0    Temp    15480 non-null   float64
1   Current  15480 non-null   float64
2     NG     15480 non-null   int64
dtypes: float64(2), int64(1)
memory usage: 362.9 KB
```

```
test_data = pick_data[35604:]
test_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 35604 entries, 0 to 35603
Data columns (total 3 columns):
#   Column  Non-Null Count  Dtype
---  ---
0    Temp    35604 non-null   float64
1   Current  35604 non-null   float64
2     NG     35604 non-null   int64
dtypes: float64(2), int64(1)
memory usage: 834.6 KB
```

[코드 35] 데이터 테스트 데이터로 분리

- LSTM_AE에는 정상 데이터만 학습을 진행할 것이기 때문에 정상인 데이터셋 세트만 추출한다.

```
ng_idx_train = train_data[train_data[NG]== 1].index
ok_idx_train = train_data[train_data[NG]== 0].index
tNc_ok_train = train_data.loc[ok_idx_train]
tNc_ng_train = train_data.loc[ng_idx_train]
```

[코드 36] 학습 데이터 세트에서 정상인 데이터만 추출

- 그런 다음 스케일링을 진행한 후 정상인 값과 대치를 시킨다. 자세한 내용은 ⑤-2 데이터 스케일 조정에서 언급하도록 하겠다.

```
from sklearn.preprocessing import StandardScaler
```

```
train = tNc_ok_train
scaler = StandardScaler()
scaler = scaler.fit(train[['Temp']])
train[['Temp']] = scaler.transform(train[['Temp']])
```

[코드 37] 학습 데이터 세트에서 정상인 데이터만 추출

- 이제 모델에 사용할 수 있도록 데이터 분리 작업을 진행한다. LSTM은 (samples, timesteps, feature) 형태를 가진다. 이 말은 3차원의 shape의 모양을 가진다는 의미이다. 그러므로 데이터를 시퀀스 형태로 변환한다. timesteps은 Lot의 단위인 36으로 설정했다.

```
TIME_STEP=36
def create_sequences(X, y, time_steps =TIME_STEP):
    Xs, ys = [], []
    for i in range(len(X)-time_steps):
        Xs.append(X.iloc[i:(i+time_steps)].values)
        ys.append(y.iloc[i:(i+time_steps)].values)
        # ys.append(y.iloc[i+time_steps])

    return np.array(Xs), np.array(ys)

X_train, Y_train = create_sequences(train[["Temp"]], train[["Temp"]])
```

[코드 38] 시계열성 데이터 분리

- 학습 데이터 세트를 앞으로 생성하게 될 모델에 적용할 수 있게 3단계로 분리한다.

```
# 데이터 분리 작업 진행 - 7:3 비율로 나눌 예정
from sklearn.model_selection import train_test_split
x_train, x_valid, y_train, y_valid = train_test_split(X_train, Y_train, test_size=0.2)
print(len(x_train))
print(len(y_train))
```

[코드 39] 정상 학습 데이터 분리 코드

```
ss_data=StandardScaler().fit_transform(test_data[["Temp"]])
print(ss_data)
```

```
[[ 0.29003725]
 [ 0.02479778]
 [ 0.24064141]
 ...
 [-0.6138397 ]
 [-0.48704981]
 [-0.4455265 ]]
```

```
X_data = pd.DataFrame()
X_data["Temp"] = list(ss_data.reshape(-1))
```

```
X_test, Y_test = create_sequences(X_data[["Temp"]], test_data[["NG"]])
```

```
def flatten(X):
    # sample x features array.
    flattened_X = np.empty((X.shape[0], X.shape[2]))
    for i in range(X.shape[0]):
        flattened_X[i] = X[i, (X.shape[1]-1), :]
    return (flattened_X)
```

```
label = flatten(Y_test).reshape(-1)
label.shape
```

[코드 40] 불량이포함된 테스트 데이터 분리 코드

⑤-2. 데이터 스케일 조정

- 데이터 스케일링이란, 데이터 분석 시 혹은 모델 학습 시 변수들의 단위 차이가 크게 차이나는 경우 혹은 데이터 세트의 데이터들이 들쭉날쭉할 경우 사용한다.
- 현 데이터 세트에서는 Standard Scaler를 사용하여 스케일링을 진행한다.
- 스케일링에는 다양한 종류가 있는데 대표적으로 Standard Scaler, MinMaxScaler,

RobustScaler, Normalizer 가 있다.

간단하게 각각의 스케일링에 관해서 설명하자면

- StandardScaler는 평균은 0으로 분산은 1로 조정하여 모든 특성들이 같은 스케일을 가지게 하는 역할을 한다. 즉, 각 특징들을 정규분포로 만든다고 보면 될 것 같다.
- MinMaxScaler는 모든 특성의 값을 0과 1사이의 값으로 만든다.
- RobustScaler는 StandardScaler와 유사하게 모든 특성들이 같은 크기를 갖는다는 것이다. 하지만 RobustScaler는 평균과 분산을 사용하는 StandardScaler과는 다르게 중앙값과 사분위 값을 이용하여 스케일링을 진행한다.
- Normalizer는 다른 스케일링처럼 특성(Column)의 통계치를 사용하는 것이 아닌 Row(행)단위로 정규화하여 진행한다. 또한 Normalizer는 유클리드 거리를 이용하여 유클리드 거리의 값이 1이 되도록 데이터를 조정하면서 스케일링을 진행한다.

[단계 ⑥] 모델링

⑥-1. 모델 동작 테스트

```
timesteps = X_train.shape[1]
features = X_train.shape[2]

import keras
import tensorflow
from keras.models import Sequential, Model
from keras.layers import Dense, Dropout, Flatten
from keras.layers import Conv2D, MaxPooling2D
from keras.layers import LSTM
from keras import backend as K
from keras.layers import *
from keras.applications import imagenet_utils
from keras.callbacks import ReduceLROnPlateau, ModelCheckpoint, EarlyStopping
from keras.optimizers import Adam
```

[코드 41] 모델 패키지 임포트

아래와 같이 모델을 구성하고 모델링을 진행하겠다.

*LSTM 모델 같은 경우 연산이 많이 발생하므로 학습의 속도가 느리다.

- 모델의 구성은 LSTM-AE 모델 구조는 2개의 Encoder 계층과 Decoder 계층으로 구성된다. Encoding layer는 32개의 1번, Hidden layer와 2번, 16개의 Hidden layer를 가지고 그 이후에는 Decoder 계층으로 변경되어 Encoder와 반대로 1번, Decoder Hidden layer는 16개의 Hidden layer를 가지고 32개의 Hidden Decoding layer를 가진다. Activation Function(활성화 함수)는 렐루로 설정해 작업을 진행하였다.


```
Lstm_AE_3 = Sequential()

# Encoder
Lstm_AE_3.add(LSTM(32,activation='relu', input_shape=(timesteps, features), return_sequences=True))
Lstm_AE_3.add(LSTM(16,activation='relu', return_sequences=False))
Lstm_AE_3.add(RepeatVector(timesteps))

# Decoder
Lstm_AE_3.add(LSTM(32, activation='relu', return_sequences=True))
Lstm_AE_3.add(LSTM(16, activation='relu', return_sequences=True))
Lstm_AE_3.add(TimeDistributed(Dense(features)))
Lstm_AE_3.summary()
```

```
Model: "sequential_1"
-----
Layer (type)                Output Shape          Param #
-----
lstm_1 (LSTM)                (None, 36, 32)        4352
lstm_2 (LSTM)                (None, 16)            3136
repeat_vector_1 (RepeatVecto (None, 36, 16)         0
lstm_3 (LSTM)                (None, 36, 16)        2112
lstm_4 (LSTM)                (None, 36, 32)        6272
time_distributed_1 (TimeDist (None, 36, 1)         33
-----
Total params: 15,905
Trainable params: 15,905
Non-trainable params: 0
-----
```

[코드 42] 모델 요약

*Activation Function에 대한 설명은 적용하고자 하는 AI 분석 방법론(알고리즘)의 구체적 소개를 참조하기를 바란다.

[단계 ⑦] 모델 훈련

- 모델을 훈련 전 주의해야 할 것은 반드시 정상 데이터만 학습해야 한다는 점이다.

```
# GPU 사용 경우
from keras import backend as K

epochs = 200
batch = 128
lr = 0.001

if K.image_data_format() == 'channels_last':
    optimizer = keras.optimizers.Adam(lr)
    Lstm_AE_3.compile(loss='mse', optimizer=optimizer)
    history = Lstm_AE_3.fit(x_train, y_train,
                            epochs=epochs, batch_size=batch,
                            validation_data=(x_valid, y_valid))
else : pass

# CPU 사용 경우
epochs = 200
batch = 128
lr = 0.001

optimizer = keras.optimizers.Adam(lr)
Lstm_AE_3.compile(loss='mse', optimizer=optimizer)
history = Lstm_AE_3.fit(x_train, y_train, epochs=epochs, batch_size=batch,
                        validation_data=(x_valid, y_valid))
```

```

Train on 20666 samples, validate on 6459 samples
Epoch 1/200
20666/20666 [=====] - 19s 900us/step - loss: 0.2407 - val_loss: 0.2290
Epoch 2/200
20666/20666 [=====] - 17s 835us/step - loss: 0.2282 - val_loss: 0.2309
Epoch 3/200
20666/20666 [=====] - 17s 820us/step - loss: 0.2301 - val_loss: 0.2264
Epoch 4/200
20666/20666 [=====] - 17s 828us/step - loss: 0.2273 - val_loss: 0.2231
Epoch 5/200
20666/20666 [=====] - 17s 818us/step - loss: 0.2232 - val_loss: 0.2307
Epoch 6/200
20666/20666 [=====] - 17s 822us/step - loss: 0.2225 - val_loss: 0.2205
Epoch 7/200
20666/20666 [=====] - 17s 801us/step - loss: 0.2201 - val_loss: 0.2243
Epoch 8/200
20666/20666 [=====] - 17s 808us/step - loss: 0.2200 - val_loss: 0.2201
Epoch 9/200
20666/20666 [=====] - 17s 838us/step - loss: 0.2165 - val_loss: 0.2201
Epoch 10/200
20666/20666 [=====] - 17s 839us/step - loss: 0.2157 - val_loss: 0.2248
Epoch 11/200
20666/20666 [=====] - 17s 812us/step - loss: 0.2146 - val_loss: 0.2245
Epoch 12/200
20666/20666 [=====] - 17s 822us/step - loss: 0.2125 - val_loss: 0.2102
Epoch 13/200
20666/20666 [=====] - 17s 832us/step - loss: 0.2121 - val_loss: 0.2098
Epoch 14/200
20666/20666 [=====] - 17s 817us/step - loss: 0.2082 - val_loss: 0.2076
Epoch 15/200
20666/20666 [=====] - 17s 818us/step - loss: 0.2074 - val_loss: 0.2066
Epoch 16/200
20666/20666 [=====] - 17s 804us/step - loss: 0.2062 - val_loss: 0.2065
Epoch 17/200
20666/20666 [=====] - 17s 828us/step - loss: 0.2064 - val_loss: 0.2061
Epoch 18/200
20666/20666 [=====] - 17s 844us/step - loss: 0.2029 - val_loss: 0.2048
Epoch 19/200
20666/20666 [=====] - 16s 871us/step - loss: 0.2005 - val_loss: 0.2000

```

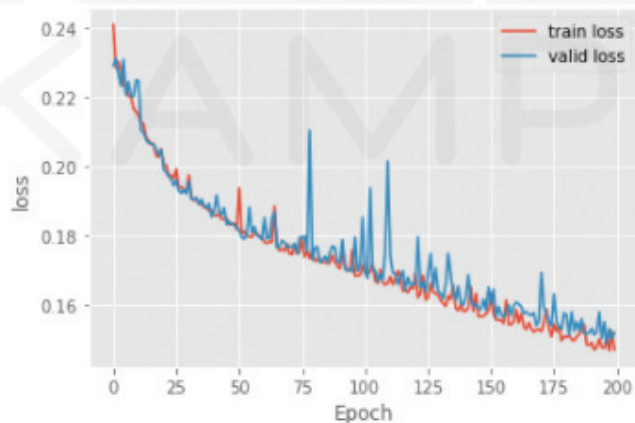
[코드 43] LSTM-AE 학습

위와 같이 학습을 진행하였고 아래와 같은 결과를 보였다.

```

plt.plot(history.history['loss'], label='train loss')
plt.plot(history.history['val_loss'], label='valid loss')
plt.legend()
plt.xlabel('Epoch'); plt.ylabel('loss')
plt.show()

```



[코드 44] LSTM-AE 손실값 시각화

- 손실 값이 점점 감소하는 것을 확인 할 수 있다. 딥러닝에서 손실함수는 빼놓을 수 없는 매우 중요한 요소이다. 그 손실 값을 기준으로 가중치의 값을 찾아 나가는 작업을 하는 것이 손실함수이다.

* 손실함수는 AI 분석 방법론(알고리즘)의 구체적 소개를 참조하기를 바란다.

[단계 ⑧]모델 튜닝

⑧-1. 하이퍼 파라미터 추가 및 수정

- layer의 수를 32에서 140으로 늘리고 16에서 70으로 대폭 증가시켰다.

```
epochs =200
batch =128
lr =0.001

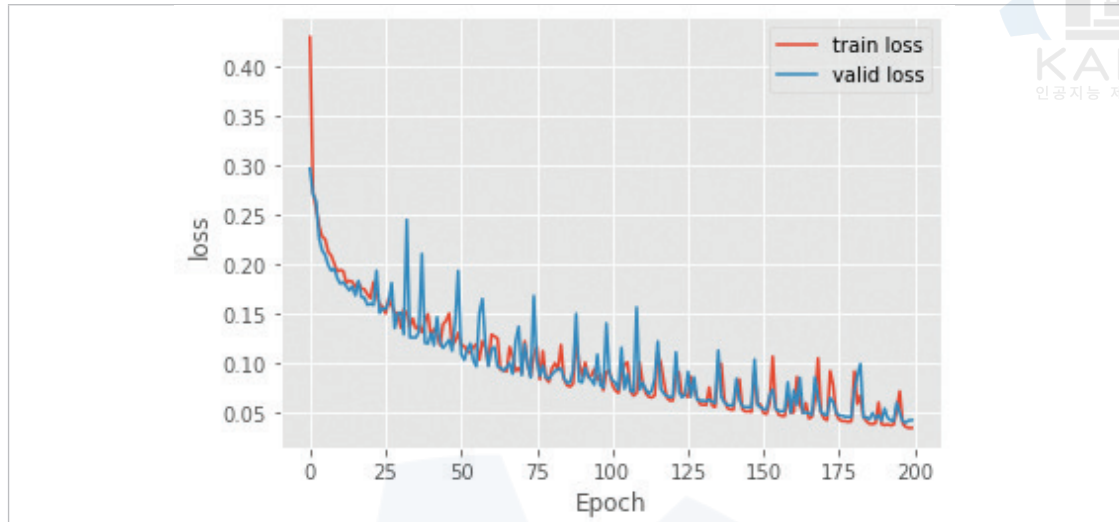
Lstm_AE_3 = Sequential()

# Encoder
Lstm_AE_3.add(LSTM(140,activation ='relu', input_shape =(timesteps, features), return_sequences =True))
Lstm_AE_3.add(LSTM(70,activation ='relu', return_sequences =False))
Lstm_AE_3.add(RepeatVector(timesteps))

# Decoder
Lstm_AE_3.add(LSTM(70 , activation ='relu', return_sequences =True))
Lstm_AE_3.add(LSTM(140 , activation ='relu',return_sequences =True))
Lstm_AE_3.add(TimeDistributed(Dense(features)))
Lstm_AE_3.summary()
```

Model: "sequential_2"		
Layer (type)	Output Shape	Param #
lstm_5 (LSTM)	(None, 36, 140)	79520
lstm_6 (LSTM)	(None, 70)	59080
repeat_vector_2 (RepeatVecto	(None, 36, 70)	0
lstm_7 (LSTM)	(None, 36, 70)	39480
lstm_8 (LSTM)	(None, 36, 140)	118160
time_distributed_2 (TimeDist	(None, 36, 1)	141
Total params: 296,381		
Trainable params: 296,381		
Non-trainable params: 0		

[코드 45] 모델 튜닝



[코드 46] 튜닝 모델 손실값

- 또한, epoch와 batch 크기도 200, 128로 진행하였지만, 결과를 확인해보니 절반 이상의 정상 데이터를 비정상이라고 판단하는 결과가 발생하여 epoch와 batch 치수를 조절하여 아래와 같이 재진행 하였다.

```
epochs = 50
batch = 32
lr = 0.001

Lstm_AE_3 = Sequential()

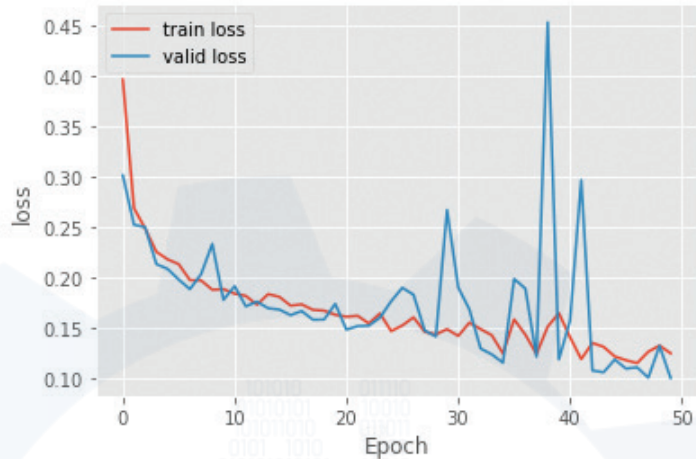
# Encoder
Lstm_AE_3.add(LSTM(140,activation='relu', input_shape=(timesteps, features), return_sequences=True))
Lstm_AE_3.add(LSTM(70,activation='relu', return_sequences=False))
Lstm_AE_3.add(RepeatVector(timesteps))

# Decoder
Lstm_AE_3.add(LSTM(70 , activation='relu', return_sequences=True))
Lstm_AE_3.add(LSTM(140 , activation='relu',return_sequences=True))
Lstm_AE_3.add(TimeDistributed(Dense(features)))
Lstm_AE_3.summary()
```

```
Model: "sequential_2"
Layer (type)                 Output Shape              Param #
-----
lstm_5 (LSTM)                 (None, 36, 140)          79520
lstm_6 (LSTM)                 (None, 70)               59080
repeat_vector_2 (RepeatVecto (None, 36, 70)           0
lstm_7 (LSTM)                 (None, 36, 70)          39480
lstm_8 (LSTM)                 (None, 36, 140)         118160
time_distributed_2 (TimeDist (None, 36, 1)            141
-----
Total params: 296,381
Trainable params: 296,381
Non-trainable params: 0
-----
```

[코드 47] batch, epoch 조절 후 모델

```
# GPU 사용 경우
from keras import backend as K
if K.image_data_format() == 'channels_last':
    optimizer = keras.optimizers.Adam(lr)
    Lstm_AE_3.compile(loss='mse', optimizer=optimizer)
    history = Lstm_AE_3.fit(x_train, y_train,
                           epochs=epochs, batch_size=batch,
                           validation_data=(x_valid, y_valid))
else:pass
```



[코드 48] batch, epoch 조절 후 손실값

[단계 ⑨]모델 평가 및 해석

⑨-1. 결과 시각화 및 해석

- 산출한 데이터의 결과를 해석하면 아래와 같다. 모델에 관해 설명할 때 언급한 것처럼 정상이면 출력값은 입력과 매우 유사하게 출력되며 이상 데이터일 경우 확연한 차이를 확인할 수 있다고 하였다.

```
prediction = Lstm_AE_3.predict(X_test)
mse = np.mean(np.power(X_test - prediction, 2), axis=1)
error_df = pd.DataFrame({'reconstruction_error': mse.reshape(-1),
                        'true_class':label})
error_df.describe()
```

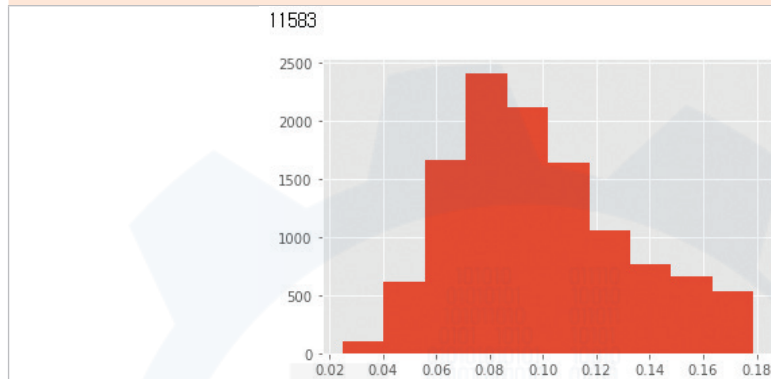
	reconstruction_error	true_class
count	15444.000000	15444.00000
mean	0.759213	0.04662
std	2.759609	0.21083
min	0.024836	0.00000
25%	0.080309	0.00000
50%	0.108481	0.00000
75%	0.178893	0.00000
max	18.966995	1.00000

[코드 49] prediction후 Reconstruction Error 값

- Reconstruction Error의 Min의 값은 0.024고 Max값을 확인 해 보았을 때 18.96임을 확인 할 수 있다.

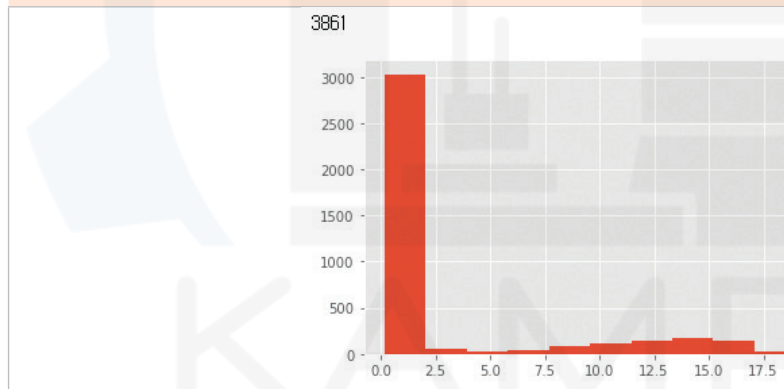
```
thr = np.percentile(mse.reshape(-1),75) # 75%
# thresholds를 75% 제한 했을 경우 - thresholds 보다 작을때(정상)

fig = plt.figure()
ax = fig.add_subplot(111)
fraud_error_df = error_df[error_df['reconstruction_error'] < thr]
print(len(fraud_error_df))
_ = ax.hist(fraud_error_df.reconstruction_error.values, bins=10)
```



```
# thresholds를 75% 제한 했을 경우- thresholds 보다 클때(비정상)

fig = plt.figure()
ax = fig.add_subplot(111)
fraud_error_df = error_df[error_df['reconstruction_error'] > thr]
print(len(fraud_error_df))
_ = ax.hist(fraud_error_df.reconstruction_error.values, bins=10)
```



[코드 50] prediction후 mse 시각화

- 아래의 그래프를 보면 0과 2.5 아래로 굉장히 밀집되어있는 것을 확인 할 수 있다. 이 말은 0과 2.5 아래의 데이터는 정상에 가까운 데이터라는 말이다. 정상으로 판단되는 데이터는 11583개 그리고 비정상으로 판단되는 데이터는 3861개로 보인다.
- 위의 모델은 예측보다는 분류에 가까우므로 정확도라는 개념보다는 결과 데이터가 신뢰할 만한 데이터인지 아닌지 확인하는 작업이 필요하다. 그러므로 오차 행렬을 통해 모델로 얻은 데이터에 대한 검증이 필요하다.
- 오차 행렬(Confusion Matrix)을 이용하여 데이터를 얼마나 신뢰 할 수 있는지를

확인하고자 한다. 오차 행렬은 트레이닝을 통한 결과성능을 측정하기 위해 예측 값과 실제 값을 비교하기 위한 표이다.

사실(사실이 맞음)

◦ 진짜사실(True-Positive:TP) ↔ 거짓거짓(False-Negative:FN)

거짓(사실이 아님)

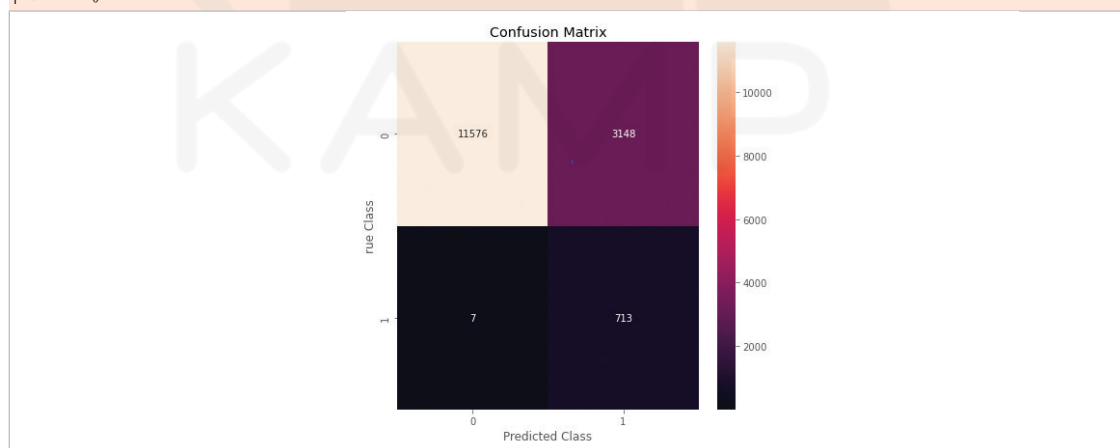
◦ 거짓사실(False-Positive:FP) ↔ 진짜거짓(True-Negative:TN)

	0	1
0	TN	FP
1	FN	TP

[그림 51] 성능 성과 지표 - 오차행렬

```
from sklearn.metrics import *
```

```
pred_y = [1 if e > thr else 0 for e in error_df['reconstruction_error'].values]
conf_matrix = confusion_matrix(error_df['true_class'], pred_y)
plt.figure(figsize=(7, 7))
sns.heatmap(conf_matrix, annot=True, fmt='d')
plt.title('Confusion Matrix')
plt.xlabel('Predicted Class'); plt.ylabel('True Class')
plt.show()
```



[코드 52] 성능 성과 지표 - 오차행렬

TP : 11576, FN : 7, FP:3148, TN:713으로 확인되었다.

오차 행렬로 구한 값으로 아래의 식에 대입하여 얼마나 데이터가 신뢰할만한지 확인해본다.

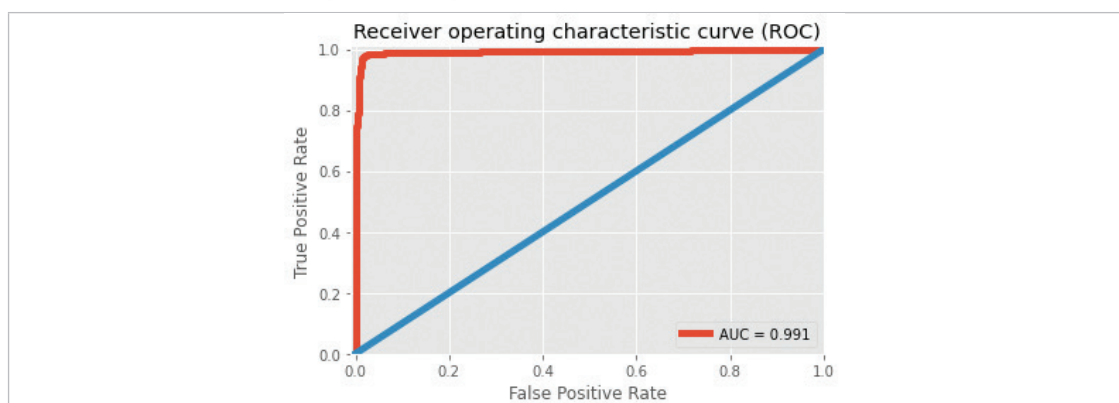
- Recall (재현율) : 출력 결과가 실제 정답 중에서 얼마나 맞았는가를 나타내는 지표
– $\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$
- Precision (정밀도) : 출력 결과에서 정답을 얼마나 맞혔는지 나타내는 지표
– $\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$
- Accuracy (정확도)
– $\text{Accuracy} = (\text{TP} + \text{TN}) / (\text{TP} + \text{FP} + \text{FN} + \text{TN})$
- F1-Score (F-점수) : 균형 잡힌 성능 평가 지표로 정밀도와 재현율의 상충 관계로 평가에 반영하여, 실제 분류기를 비교할 때 사용하는 지표
– $\text{F1-Score} = 2 * (\text{Recall} * \text{Precision}) / (\text{Recall} + \text{Precision})$

```
TP = conf_matrix[0][0]
FN = conf_matrix[0][1]
FP = conf_matrix[1][0]
TN = conf_matrix[0][1]

Recall = TP / (TP + FN)
Precision = TP / (TP + FP)
Accuracy = (TP + TN) / (TP + FP + FN + TN)
F1_Score = 2 * (Recall * Precision) / (Recall + Precision)
```

Recall	0.9993956660623328
Precision	0.7861994023363217
Accuracy	0.7957135457135457
F1_Score	0.8800699433610826

[코드 53] 모델 성능 성과 지표 - 계산



[코드 54] 모델 성능 성과 지표

- 이렇게 불량데이터가 포함된 테스트 데이터를 이용해 정상 데이터에서 생성한 범주를 병합하여 이상 탐지를 진행했다.
- Roc 커브를 확인해 본 결과 모델은 상당히 좋은 성능을 얻은 것으로 확인 할 수 있다. 첫점과 끝점은 모이게 되고 선과 선 사이의 면적으로 평가하는 것이기 때문에 선과 선 사이의 거리가 멀수록(흰 정도가 좋을수록) 모델이 좋은 성능을 가지고 있다고 말할 수 있다.
- 또한, 혼동 행렬로 얻은 값을 확인해 본 결과 아쉽게도 잡음이 많았는지 FP(False-Positive) 비율이 높게 나왔다. 이는 정답을 이상이라고 판단한 것이다.
- 다행스럽게도 정밀도와 재현율이 높게 나왔고 Precision과 Recall 두 값 모두 고려하기 위한 지표인 F1-Score도 1에 가까운 값을 보여주고 있으므로 위 데이터는 신뢰할 수 있다고 할 수 있다. 데이터의 정확도는 79.5%로 상당히 높은 것을 확인 할 수 있다.
- 만일 보다 모델의 성능이나 결과의 품질을 높이하고자 한다면 새로운 기법을 찾아 사용하여 이상 탐지를 진행하거나 현재 모델을 튜닝을 통해 모델의 성능을 최적화하는 방법 또는 변수들을 활용해 새로운 특징을 찾아 모델 학습하는 방법이 있다. 이러한 방법들 활용한다면 더 좋은 결과를 낼 수 있을 것으로 판단 된다.
- 모델의 성능이나 정확도도 중요하지만, 모델에서 나온 결과를 해석하고 평가하는 작업도 매우 중요하다.
- 위 분석결과를 통해 알고리즘을 현장에 적용해 이상징후를 효과적으로 수행할 수 있음을 알 수 있다. 하지만 위와 같은 작업을 수행하기 위해서는 실시간 스트리밍 분석이 필요하며 그에 맞는 환경을 먼저 구축해야 할 필요가 있다. 개발된 분석 모델이 실제 공정에 적용되기 위해서는 사운드 데이터를 실시간으로 추출한다. 또한, 기계에서 수집한 실시간 온도, 전류값과 매칭시켜 팬의 결함 여부를 판단하고 비교하여 빠르게 이상에 대처할 수 있고 만일 이상징후 발견 시 알람이 발생하도록 하여 실제 설비문제가 발생하기 전에 대응할 수 있도록 위 모델을 기반으로 한 모듈을 구현할 수 있다. 만일 생산라인에 적용한다면 실제 설비문제가 발생하기 전 대응 가능할 수 있을 것으로 예상된다. 본 가이드북을 통해 개발된 알고리즘을 활용하기 위해서는 실시간 자료수집이 가능한 수준의 현장 환경이 수반되어야 한다.

3) 분석 단계별 프로세스 - Decision Tree

[단계 ①] 데이터 준비

①-1. 라이브러리(Import) 설치

```
!pip install pandas
!pip install numpy
!pip install sklearn
!pip install seaborn
!pip install datetime
!pip install matplotlib
!pip install pydot
!pip install librosa
```

[코드 1] pip 패키지 설치 방법

①-2. 라이브러리(Import) 불러오기

```
# 데이터 분석 툴
import pandas as pd
import numpy as np

# 운영체제와 관련된 툴
import os
import glob

# 시각화
import seaborn as sns
import matplotlib.pyplot as plt # matplotlib
%matplotlib inline

# 경고 방지
import warnings # 워닝 방지
warnings.filterwarnings('ignore')

plt.figure(figsize=(15,15))
plt.style.use('ggplot')
```

[코드 2] 패키지 import 코드

1) 데이터 분석 툴

- 가) 판다스(Pandas) 모듈은 대표적인 데이터 분석 패키지로 보다 쉽게 데이터 분석 기능이 가능하다.
- 나) 넘파이(Numpy)는 C언어로 구현된 파이썬(Python) 라이브러리로 행렬이나 일반적으로 대규모 다차원 배열을 쉽게 핸들링 가능하다.

2) 운영체제와 관련된 패키지

- 가) Os 모듈은 Operating System의 약자로 운영체제에서 제공되는 여러 기능을 Python에서 사용 가능하다.
- 나) Glob 모듈은 특정한 패턴이나 확장자를 가진 파일들의 경로를 가져와 사용 가능하다.

3) 시각화 툴

가) Matplotlib Python 프로그래밍 언어 및 수학적 확장 NumPy 라이브러리를 활용한 플로팅 라이브러리다.

나) Seaborn 은 Matplotlib 의 상위 호환 데이터 시각화를 위한 라이브러리다.

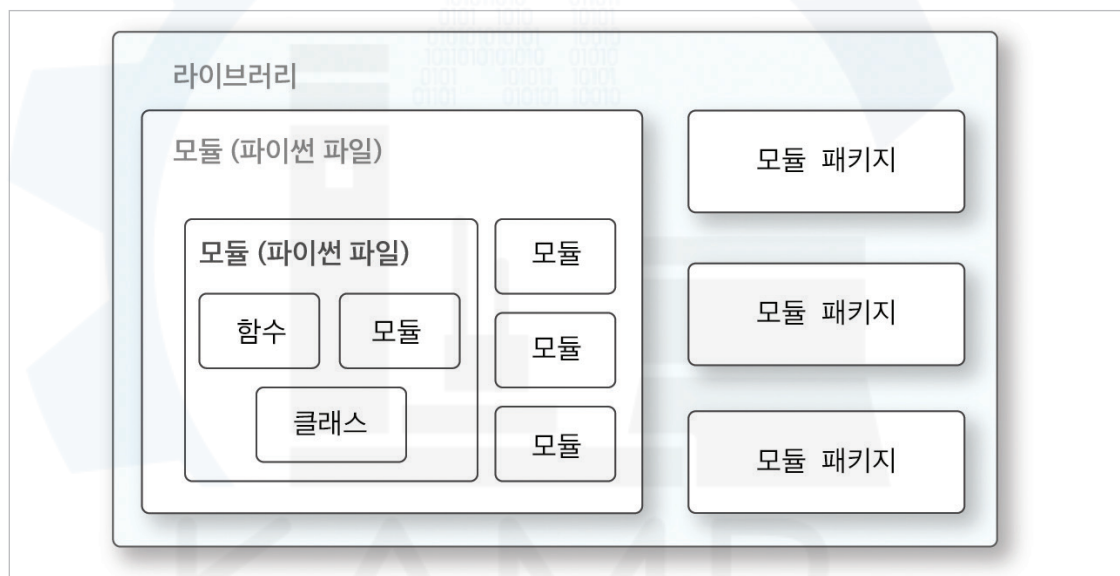
4) 경고 방지

가) Warnings 모듈은 Python에서 발생하는 다양한 경고를 무시 할 수 있도록 하는 라이브러리다.

5) 음성 파일 분석프로그램

가) librosa 패키지는 파이썬(python)에서 많이 쓰이는 음성 파일 분석프로그램

여기서 Tip. 그렇다면 라이브러리, 패키지, 모듈의 차이점은 뭘까?
아래의 그림과 같이 표현 할 수 있다.



[그림 52] 라이브러리, 패키지, 모듈

①-3. 분석용 데이터 불러오기

1) 데이터 분석을 위해서 분석용 데이터를 불러온다.

- 열고자 하는 파일이 내 경로에 없다면 데이터를 로드 할 때 전체 경로(절대 경로) 혹은 부분 경로(상대 경로)를 불러와야 데이터를 불러올 수 있다.

```
new_path = os.path.join(os.getcwd(), 'data')
new_path

dir_path = os.listdir('data')
dir_path

['5공 정_180sec', '5공 정_180sec_1', '5공 정_20sec', 'FAN_sound_error', 'FAN_sound_OK']

err_Name = os.path.join(new_path, 'FAN_sound_error')
err_Name

'C:\\jupyter_gpu\\data\\FAN_sound_error'

ok_Name = os.path.join(new_path, 'FAN_sound_OK')
ok_Name

'C:\\jupyter_gpu\\data\\FAN_sound_OK'
```

[코드 3] Sound Data 경로

- 경로를 확인하고 경로를 추출한다.

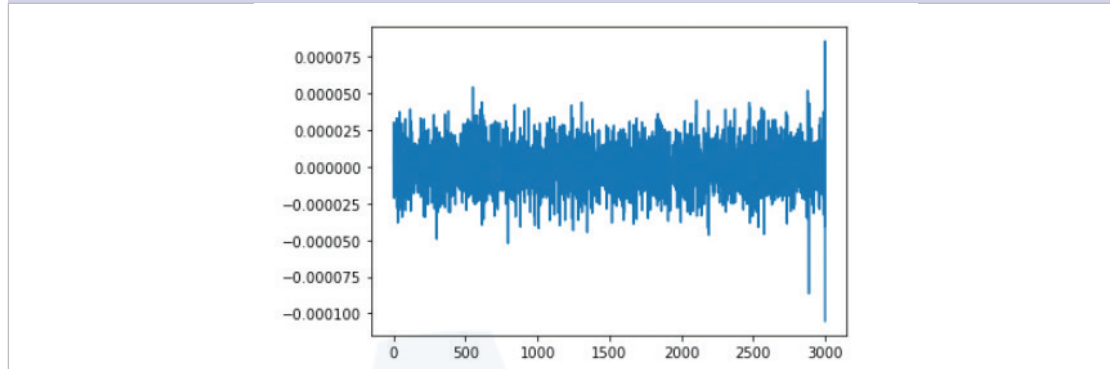
```
glob.glob(err_Name + '/*')

['C:\\jupyter_gpu\\data\\5공 정_180sec\\Error Lot list.csv',
'C:\\jupyter_gpu\\data\\5공 정_180sec\\kemp-abh-sensor-2021,09,06.csv',
'C:\\jupyter_gpu\\data\\5공 정_180sec\\kemp-abh-sensor-2021,09,07.csv',
'C:\\jupyter_gpu\\data\\5공 정_180sec\\kemp-abh-sensor-2021,09,08.csv',
'C:\\jupyter_gpu\\data\\5공 정_180sec\\kemp-abh-sensor-2021,09,09.csv',
'C:\\jupyter_gpu\\data\\5공 정_180sec\\kemp-abh-sensor-2021,09,10.csv',
'C:\\jupyter_gpu\\data\\5공 정_180sec\\kemp-abh-sensor-2021,09,13.csv',
'C:\\jupyter_gpu\\data\\5공 정_180sec\\kemp-abh-sensor-2021,09,14.csv',
'C:\\jupyter_gpu\\data\\5공 정_180sec\\kemp-abh-sensor-2021,09,15.csv',
'C:\\jupyter_gpu\\data\\5공 정_180sec\\kemp-abh-sensor-2021,09,16.csv',
'C:\\jupyter_gpu\\data\\5공 정_180sec\\kemp-abh-sensor-2021,09,17.csv',
'C:\\jupyter_gpu\\data\\5공 정_180sec\\kemp-abh-sensor-2021,09,23.csv',
'C:\\jupyter_gpu\\data\\5공 정_180sec\\kemp-abh-sensor-2021,09,24.csv',
'C:\\jupyter_gpu\\data\\5공 정_180sec\\kemp-abh-sensor-2021,09,27.csv',
'C:\\jupyter_gpu\\data\\5공 정_180sec\\kemp-abh-sensor-2021,09,28.csv',
'C:\\jupyter_gpu\\data\\5공 정_180sec\\kemp-abh-sensor-2021,09,29.csv',
'C:\\jupyter_gpu\\data\\5공 정_180sec\\kemp-abh-sensor-2021,09,30.csv',
'C:\\jupyter_gpu\\data\\5공 정_180sec\\kemp-abh-sensor-2021,10,01.csv',
'C:\\jupyter_gpu\\data\\5공 정_180sec\\kemp-abh-sensor-2021,10,05.csv',
'C:\\jupyter_gpu\\data\\5공 정_180sec\\kemp-abh-sensor-2021,10,06.csv',
'C:\\jupyter_gpu\\data\\5공 정_180sec\\kemp-abh-sensor-2021,10,07.csv',
'C:\\jupyter_gpu\\data\\5공 정_180sec\\kemp-abh-sensor-2021,10,08.csv',
'C:\\jupyter_gpu\\data\\5공 정_180sec\\kemp-abh-sensor-2021,10,12.csv',
'C:\\jupyter_gpu\\data\\5공 정_180sec\\kemp-abh-sensor-2021,10,13.csv',
'C:\\jupyter_gpu\\data\\5공 정_180sec\\kemp-abh-sensor-2021,10,14.csv',
'C:\\jupyter_gpu\\data\\5공 정_180sec\\kemp-abh-sensor-2021,10,15.csv',
'C:\\jupyter_gpu\\data\\5공 정_180sec\\kemp-abh-sensor-2021,10,18.csv',
'C:\\jupyter_gpu\\data\\5공 정_180sec\\kemp-abh-sensor-2021,10,19.csv',
'C:\\jupyter_gpu\\data\\5공 정_180sec\\kemp-abh-sensor-2021,10,20.csv',
'C:\\jupyter_gpu\\data\\5공 정_180sec\\kemp-abh-sensor-2021,10,21.csv',
'C:\\jupyter_gpu\\data\\5공 정_180sec\\kemp-abh-sensor-2021,10,22.csv',
'C:\\jupyter_gpu\\data\\5공 정_180sec\\kemp-abh-sensor-2021,10,25.csv',
'C:\\jupyter_gpu\\data\\5공 정_180sec\\kemp-abh-sensor-2021,10,26.csv',
'C:\\jupyter_gpu\\data\\5공 정_180sec\\kemp-abh-sensor-2021,10,27.csv']
```

[코드 4] 파일 경로 추출

- 추출한 경로를 가지고 데이터를 확인하면 아래와 같은 그래프를 확인 할 수 있다.


```
audio_path = glob.glob(err_Name + '/*')[0]
y, sr = librosa.load(audio_path, 100)
plt.plot(y)
plt.show()
```



[코드 22] sound Data sample 확인

[단계 ②] 데이터 탐색

②-1. 데이터 사본 생성

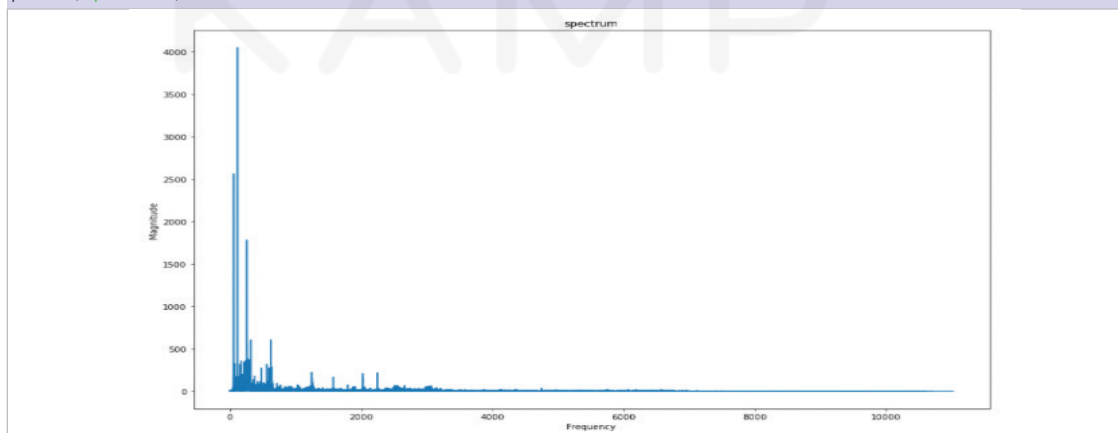
- spectrum은 푸리에 변환을 통과한 후 대칭인 구조로 나와서 frequency 부분 절반을 날리고 절반만 사용한다.

```
audio_path = glob.glob(ok_Name + '/*')[0]
y, sr = librosa.load(audio_path)
fft = np.fft.fft(y)

magnitude = np.abs(fft)
fre = np.linspace(0, sr, len(magnitude))

haf_spectrum = magnitude[:int(len(magnitude)/2)]
haf_fre = fre[:int(len(magnitude)/2)]

plt.figure(figsize=(15, 10))
plt.plot(haf_fre, haf_spectrum)
plt.xlabel("Frequency")
plt.ylabel("Magnitude")
plt.title("spectrum")
```



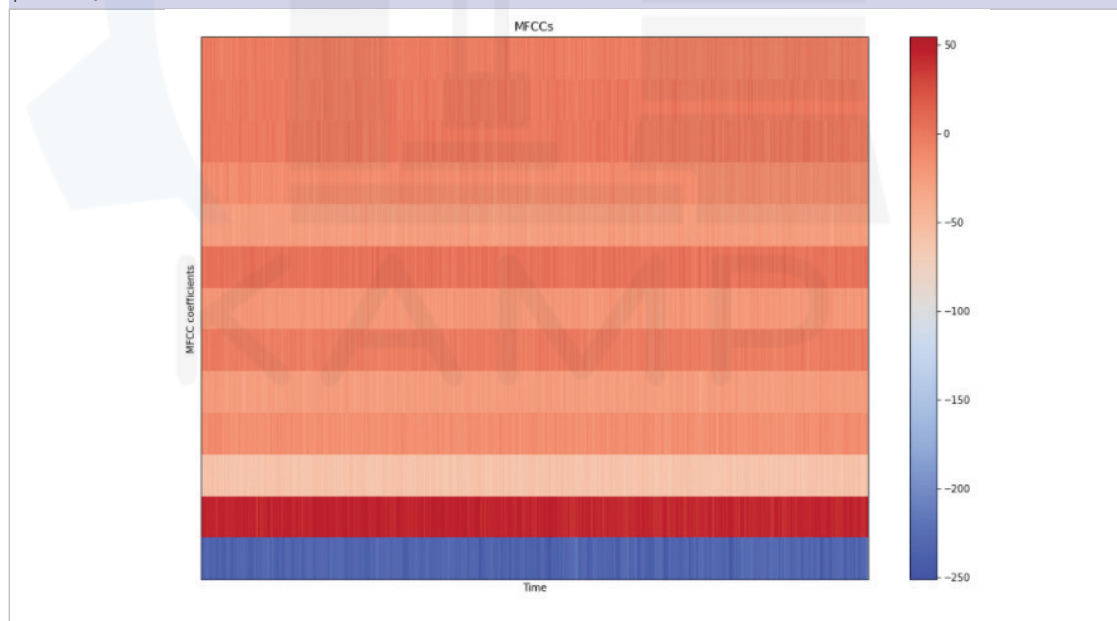
[코드 6] half spectrum 확인

- spectrum은 어떠한 신호를 소리의 높이에 따라 분해하면 높이의 성분의 강도를 확인할 수 있다. 이 신호의 높이를 주파수라고 하며 이를 통해 소리의 특징을 확인할 수 있다.
- 머신러닝의 경우 음원을 그대로 사용하는 것보다 음원에서 발생한 정보(분석정보)를 추출하여 학습하는 것이 음원 자체를 그대로 사용하는 것보다 더 효율 높다.
- MFCC (Mel Frequency Cepstral Coefficient)는 소리 데이터 특징값으로 사람이 예민하게 반응하는 정보를 강조하여 소리가 가지는 고유한 특징으로 추출한 값을 표기하며, 해당 전처리에서 mfcc의 개수를 13개로 특정한다.

```
audio_path = glob.glob(err_Name+'/*')[0]
y, sr = librosa.load(audio_path)
hop_length = 512 # 전체 frame 수
n_fft = 2048 # frame 하나당 sample 수
MFCCs = librosa.feature.mfcc(y, sr, n_fft=n_fft, hop_length=hop_length, n_mfcc=13)

# display MFCCs
plt.figure(figsize=(15, 10))
librosa.display.specshow(MFCCs, sr=sr, hop_length=hop_length)
plt.xlabel("Time")
plt.ylabel("MFCC coefficients")
plt.colorbar()
plt.title("MFCCs")

# show plots
plt.show()
```



[코드 7] MFCCs 확인

- mfcc의 min,max데이터와 half spectrum의 min,max 데이터를 추출 하여 작업을 진행한다.
- 특성을 추출하기 위해서는 데이터프레임의 틀을 먼저 세운다.

```
df = pd.DataFrame(columns=['mfcc_min','mfcc_max','spectrum_min','spectrum_max'])
df['NG'] = 0
df
```

mfcc_min	mfcc_max	spectrum_min	spectrum_max	NG
----------	----------	--------------	--------------	----

[코드 8] 특성 추출 step1

- 그 후 데이터프레임 안에 뽑아낸 특성을 각각의 컬럼에 채운다.

```
def mk_Frequency(y,sr):
    fft = np.fft.fft(y)
    magnitude = np.abs(fft)
    fre = np.linspace(0 ,sr,len(magnitude))
    haf_spectrum = magnitude[:int(len(magnitude)/2)]
    haf_fre = fre[:int(len(magnitude)/2)]
    return haf_spectrum, haf_fre
```

```
spectrum_min = list()
spectrum_max = list()
mfcc_min = list()
mfcc_max = list()
ng = list()
a = 0
b = 0
for n in range(183):
    if (len(glob.glob(err_Name+'/*'))-1) >= n :
        path = glob.glob(err_Name+'/*')[a]
        err, ser = librosa.load(path,sr=100)
        left_spectrum, left_f_ng = mk_Frequency(err, ser)
        mfcc = librosa.feature.mfcc(y=err, sr=ser)
        ng.append(1)
        a += 1
    else :
        path = glob.glob(ok_Name+'/*')[b]
        ok, skr = librosa.load(path,sr=100)
        left_spectrum, left_f_ok = mk_Frequency(ok,skr)
        mfcc = librosa.feature.mfcc(y=ok, sr=skr)
        ng.append(0)
        b += 1

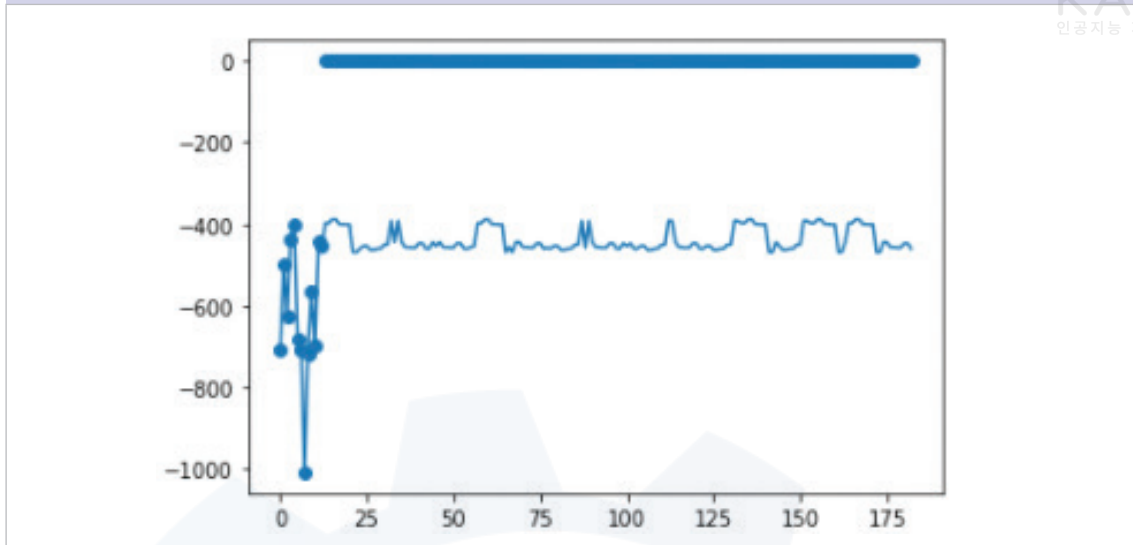
    spectrum_min.append(min(left_spectrum))
    spectrum_max.append(max(left_spectrum))
    mfcc_min.append(mfcc.min())
    mfcc_max.append(mfcc.max())
```

```
df['spectrum_min'] = spectrum_min
df['spectrum_max'] = spectrum_max
df['mfcc_min'] = mfcc_min
df['mfcc_max'] = mfcc_max
df['NG'] = ng
```

[코드 9] 특성 추출

- 이렇게 채운 데이터를 간단하게 시각화해 보면 아래와 같다.

```
df['mfcc_min'].plot()
plt.scatter(df.index, df.NG*df['mfcc_min'])
```

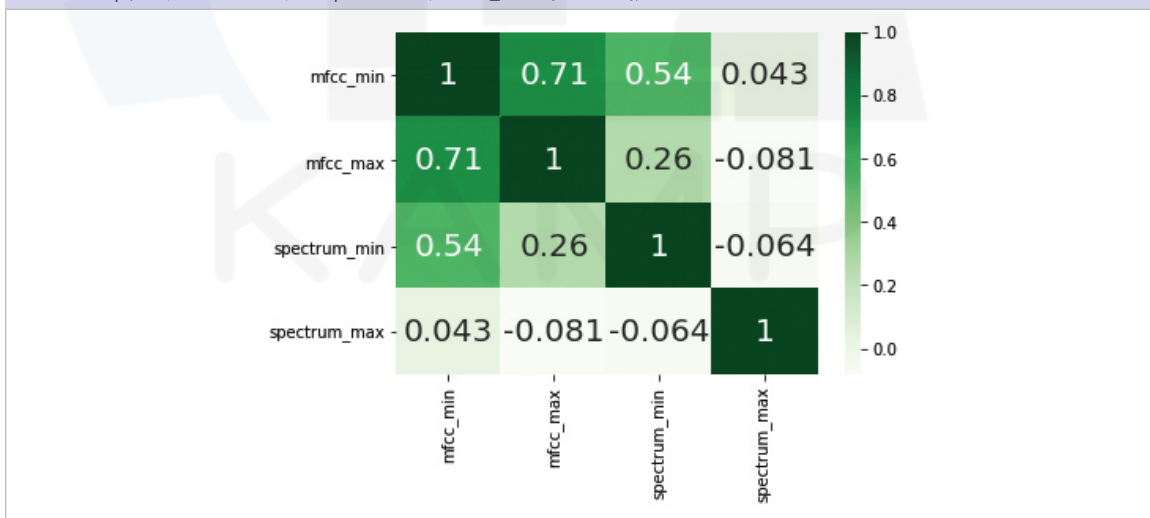


[코드 10] 특성 추출 데이터 확인

②-6. 데이터 간의 상관관계 파악

- 상관 분석(Correlation analysis)이란, 두 변수 간의 관계를 확인하여 그 관계가 선형적 혹은 비선형적 관계가 있는지를 확인하는 분석이다. 주로 상관 계수의 여부를 확인할 경우 많이 사용한다.

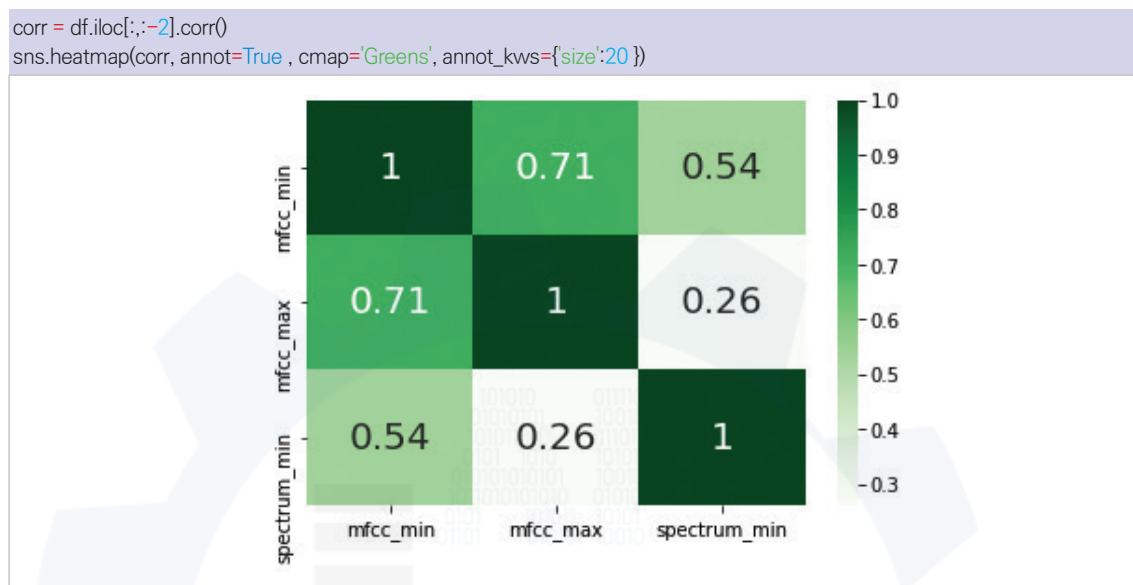
```
corr = df.iloc[:, :-1].corr()
sns.heatmap(corr, annot=True, cmap='Greens', annot_kws={'size':20})
```



[코드 11] 상관 계수 시각화 - 1

- mfcc의 min,max데이터와 half spectrum의 min,max 데이터는 half spectrum의 max를 제외한 컬럼을 추출하여 상관 계수 확인을 진행하였다.

- 해당 특성을 제외한 이유는 현저히 낮은 상관관계와 학습 데이터 세트로 만들어 학습할 때 정확도에 영향을 줄 것이라고 판단했기 때문이다. 해당 기법은 변수 선택법의 후진 소거법이라는 기법으로 전체 변수에서 성능이 낮은 변수 현재 상태에서는 상관관계가 없는 변수를 제거하면서 성능을 높이는 기법이다.



[코드 12] 상관 계수 시각화 -2

- 위 두 상관관계는 아래와 같이 방법으로 해석 할 수 있다. 두 변수 간의 선형적 정도를 -1 에서 1 사이의 값으로 나타낸다.

* r(상관 계수)

<< r이 양, 음의 값을 가질 때 >>

두 변수가 동시에 증가하는 경우 혹은 비례하는 경우 r이 1에 가까울수록 두 변수 간의 관계가 강하다고 할 수 있다.

음수이냐 양수이냐에 따라서 음의 상관관계를 가진다, 혹은 양의 상관관계를 가진다고 할 수 있다.

- r 값이 ± 0.1 이면, 없다고 할 수 있는 상관관계
- r 값이 $\pm 0.1 \sim \pm 0.3$ 이면, 약한 양적 상관관계
- r 값이 $\pm 0.3 \sim \pm 0.7$ 이면, 양적 상관관계
- r 값이 ± 0.7 이상이면, 강한 양적 상관관계

[단계 ③] 데이터 정제(전처리)

소리에서 데이터를 추출하여 병합 한것이므로 별도의 정제처리는 들어가지 않는다.

[단계 ④] 알고리즘 선택

- 소리에서 추출한 수치형 데이터를 활용하여 데이터간 존재하는 패턴을 의사결정나무를 이용하여 탐지하고자 한다.

[단계 ⑤] 학습, 평가 데이터 준비

⑤-1. 데이터 분리

```
# 데이터 분리 작업 진행 - 7:3 비율로 나눌 예정
from sklearn.model_selection import train_test_split

# 학습 데이터와 테스트 데이터 7:3 비율로 나눔
data = df.iloc[:, :-2]
target = df.iloc[:, -1:]
# train_test_split
X_train, X_test, y_train, y_test = train_test_split(data, target, test_size = 0.3, shuffle=True, stratify=target, random_state = 34)

# 잘 되었는지 확인
len(X_train), len(X_test), len(y_train), len(y_test)
```

[코드 55] 학습 데이터 분리 코드 - 사운드 데이터

- 음성데이터 세트의 학습 데이터 분리는 사이킷런에서 제공하는 데이터를 분리 모듈로 간단하게 할 수 있다.

```
from sklearn.model_selection import train_test_split
```

[단계 ⑥] 모델링

- criterion는 entropy를 이용하였고 max_depth는 1로 설정하여 모델이 올바르게 동작하는지 우선적으로 확인한다.

```
# 의사결정 나무
from sklearn.tree import DecisionTreeClassifier
Dtc = DecisionTreeClassifier(criterion='entropy', max_depth=1, random_state=0) # max_depth= 1
```

[코드 14] 모델링

[단계 ⑦] 모델 훈련

- 선택한 모델을 기반으로 학습을 진행하고 결과를 아래와 같이 확인 가능하다.


```
Dtc = Dtc.fit(train_x,train_y)

# 예측값 저장
y_pred = Dtc.predict(test_x)
```

```
# 학습결과 평가
from sklearn import metrics

print('학습 정확도: ', Dtc.score(train_x, train_y),'\n')
accuracy = metrics.accuracy_score(test_y, y_pred)
recall = metrics.recall_score(test_y, y_pred)
precision = metrics.precision_score(test_y, y_pred)
f1_score = metrics.f1_score(test_y, y_pred)

print('정확도: ', format(accuracy, '.2f'), '\n')
print('재현율: ', format(recall, '.2f'), '\n')
print('정밀도: ', format(precision, '.2f'), '\n')
print('F1_score: ', format(f1_score, '.2f'), '\n')
cm = metrics.confusion_matrix(test_y, y_pred)
cm_df = pd.DataFrame(cm).rename(index = {0 : '실제값(N)', 1 : '실제값(P)}, columns = {0 : '예측값(N)', 1 : '예측값(P)})
print('오차행렬:', '\n', cm_df)
```

학습 정확도: 0.984375

정확도: 0.96

재현율: 0.50

정밀도: 1.00

F1_score: 0.67

오차행렬:

	예측값(N)	예측값(P)
실제값(N)	51	0
실제값(P)	2	2

[코드 15] 모델 로드 및 학습 결과

[단계 ⑧] 모델 튜닝

⑧-1. 하이퍼 파라미터 추가 및 수정

- 모델이 정상 적으로 학습였고 결과 또한 위와 같이 나온 것을 확인하였다. 추가적으로 depth를 과감하게 증가시켜 보았으나 정확도가 낮아지는 것을 확인하였다. 그래서 depth를 1에서 3으로 증가시키고 학습하는 변수(특성)를 더 제한하였다.

```
# 학습 데이터와 테스트 데이터 7:3 비율로 나눔
data = df_sound.iloc[:, :-2]
target = df_sound.iloc[:, -1:]

# train_test_split
train_x, test_x, train_y, test_y = train_test_split(data, target, test_size =0.3, shuffle =True, stratify =target, random_state =34)

# 의사결정 나무
from sklearn.tree import DecisionTreeClassifier

Dtc = DecisionTreeClassifier(criterion = 'entropy', max_depth =3, random_state =0) # max_depth= 3
Dtc = Dtc.fit(train_x, train_y)

# 예측값 저장
y_pred = Dtc.predict(test_x)
```

[코드 16] 모델 로드 및 학습

```
# 학습결과와 평가
from sklearn import metrics

print('학습 정확도: ', Dtc.score(X_train, y_train), '\n')
accuracy = metrics.accuracy_score(y_test, y_pred)
recall = metrics.recall_score(y_test, y_pred)
precision = metrics.precision_score(y_test, y_pred)
f1_score = metrics.f1_score(y_test, y_pred)

print('정확도: ', format(accuracy, '.2f'), '\n')
print('재현율: ', format(recall, '.2f'), '\n')
print('정밀도: ', format(precision, '.2f'), '\n')
print('F1_score: ', format(f1_score, '.2f'), '\n')

cm = metrics.confusion_matrix(y_test, y_pred)
cm_df = pd.DataFrame(cm).rename(index={0: '실제값(N)', 1: '실제값(P)}, columns={0: '예측값(N)', 1: '예측값(P)})
print('오차행렬: ', '\n', cm_df)
```

학습 정확도: 0.9921875

정확도: 0.98

재현율: 0.75

정밀도: 1.00

F1_score: 0.86

오차행렬:

	예측값(N)	예측값(P)
실제값(N)	51	0
실제값(P)	1	3

튜닝 후

학습 정확도: 0.9921875

정확도: 0.98

재현율: 0.75

정밀도: 1.00

F1_score: 0.86

오차행렬:

	예측값(N)	예측값(P)
실제값(N)	51	0
실제값(P)	1	3

튜닝 전

학습 정확도: 0.984375

정확도: 0.96

재현율: 0.50

정밀도: 1.00

F1_score: 0.67

오차행렬:

	예측값(N)	예측값(P)
실제값(N)	51	0
실제값(P)	2	2

[코드 17] 튜닝 결과

튜닝 후와 전이 달라진 것을 확인 할 수 있다.

[단계 ⑨]모델 평가 및 해석

⑨-1. 결과 시각화 및 해석

- 오차 행렬(Confusion Matrix)을 이용하여 데이터를 얼마나 신뢰 할 수 있는지를 확인하고자 한다. 오차 행렬은 트레이닝을 통한 결과성능을 측정하기 위해 예측 값과 실제 값을 비교하기 위한 표이다.
- 사실(사실이 맞음)
 - 진짜사실(True-Positive:TP) ↔ 거짓거짓(False-Negative:FN)
- 거짓(사실이 아님)
 - 거짓사실(False-Positive:FP) ↔ 진짜거짓(True-Negative:TN)

	0	1
0	TN	FP
1	FN	TP

[그림 53] 성능 성과 지표 - 오차행렬

- Recall (재현율) : 출력 결과가 실제 정답 중에서 얼마나 맞았는가를 나타내는 지표
 - $\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$
- Precision (정밀도) : 출력 결과에서 정답을 얼마나 맞혔는지 나타내는 지표
 - $\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$
- Accuracy (정확도)
 - $\text{Accuracy} = (\text{TP} + \text{TN}) / (\text{TP} + \text{FP} + \text{FN} + \text{TN})$
- F1-Score (F-점수): 균형 잡힌 성능 평가 지표로 정밀도와 재현율의 상충 관계로 평가에 반영하여, 실제 분류기를 비교할 때 사용하는 지표
 - $\text{F1-Score} = 2 * (\text{Recall} * \text{Precision}) / (\text{Recall} + \text{Precision})$

학습 정확도: 0.9921875
 정확도: 0.98
 재현율: 0.75
 정밀도: 1.00
 F1_score: 0.86
 오차행렬:

	예측값(N)	예측값(P)
실제값(N)	51	0
실제값(P)	1	3

[코드 18] 성능 성과 지표 - 오차 행렬 및 결과

- 사운드 데이터의 특징을 추출하여 머신러닝 작업을 진행했다. 결과는 위와 같이 나왔다. 정밀도는 1인 것을 확인 할 수 있다. 모델의 예측값 중, 실제값과 같은 데이터 비율로 완벽에 가까운 것을 확인 할 수 있다.
- 또한, 재현율 같은 경우 모델의 실제값 중, 모델의 예측값과 같은 데이터 비율인데 상대적으로 낮은 것을 확인 할 수 있다. 재현율과 정밀도는 트레이드 오프 관계*가 있어 균형을 이루는 편이 좋다. 위 모델의 성능 재현율이 조금 낮은 것을 제외하고는 대부분 1에 가까워지므로 신뢰할만하다고 할 수 있을 것 같다.
- 위 모델의 정확도를 높이기 위해서는 sound 데이터에서 새로운 특징을 찾아내고 상관관계를 확인하여 서로 간의 상관관계가 높은 새로운 특징들을 결합하거나 소리 데이터를 불러올 때 진폭의 사이즈를 늘리고 진폭의 개수마다 특징을 좀 더 세세한 부분에서 추출하여 특징을 찾고 추출하여 학습한다면 성능이나 신뢰도 면에서 좋을 것으로 예상된다.

* 트레이드 오프(Trade-off)
한쪽을 얻으려면 다른 한쪽을 희생해야 하는 관계

- 위 분석결과를 통해 알고리즘을 현장에 적용해 이상징후를 효과적으로 수행할 수 있음을 알 수 있다. 하지만 위와 같은 작업을 수행하기 위해서는 실시간 스트리밍 분석이 필요하며 그에 맞는 환경을 먼저 구축해야 할 필요가 있다. 개발된 분석 모델이 실제 공정에 적용되기 위해서는 사운드 데이터를 실시간으로 추출한다. 또한, 기계에서 수집한 실시간 온도, 전류값과 매칭시켜 팬의 결합 여부를 판단하고 비교하여 빠르게 이상에 대처할 수 있고 만일 이상징후 발견 시 알람이 발생하도록 하여 실제 설비문제가 발생하기 전에 대응할 수 있도록 위 모델을 기반으로 한 모듈을 구현할 수 있다. 만일 생산라인에 적용한다면 실제 설비문제가 발생하기 전 대응 가능할 수 있을 것으로 예상된다. 본 가이드북을 통해 개발된 알고리즘을 활용하기 위해서는 실시간 자료수집이 가능한 수준의 현장 환경이 수반되어야 한다.

3. 유사 타현장의 「장비이상 조기탐지 AI 데이터셋」 분석 적용한 현장

3.1 본 분석이 적용 가능한 제조 현장 소개

- 도금에는 많은 방식이 존재한다. 크게 나누어 전기도금, 화학도금, 용융도금 등이 있으며, 화학도금은 전기가 통하지 않는 제품을 화학도금액에 넣어 도금하는 것을 의미하고, 용융도금은 비철 금속을 녹여서 도금할 제품을 넣어 용융금속의 피막을 입히는 것을 의미하며, 전기도금에는 동도금, 니켈도금, 크롬도금, 아연도금, 주석도금 등이 있다. 이 중에서 장비 이상 조기탐지 AI 데이터셋의 경우 전기도금 공정을 의미하며 유사한 Process를 갖는 전기도금 공정에서 사용될 수 있다.

3.2 본 「장비이상 조기탐지 AI 데이터셋」 분석을 원용하여 타 제조 현장 적용 시, 주요 고려사항

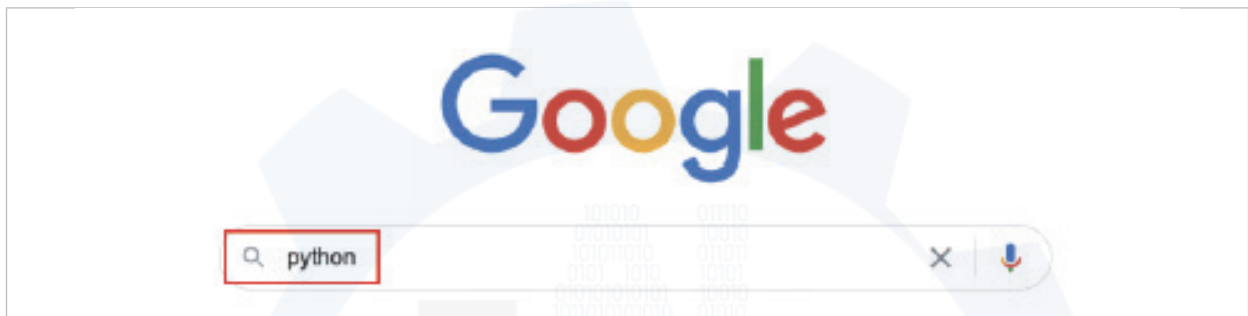
- 공정별 불량데이터 추적이 가능한 제품이 필요하다. 도금공정 특성상 도금이 완료된 제품이 한 번에 나오는데 같은 공정을 진행한 제품에 대한 불량데이터 관리가 필요하다.
- 수집되는 공정 데이터와 불량데이터 관리가 필요하다. 제품이 완료된 후 나오는 불량데이터와 해당 제품에 대한 공정 데이터의 매칭 시 Lot NO와 같은 특정 기준을 통해 통합 관리가 필요하다. 이러한 데이터 매칭을 위해 현장 실무자와 AI 분석 전문가가 이러한 상황을 미리 논의하여 정리한 후 데이터를 수집하는 시스템을 환경을 구축해야 한다.

3 부록 _ 분석환경 구축을 위한 설치 가이드

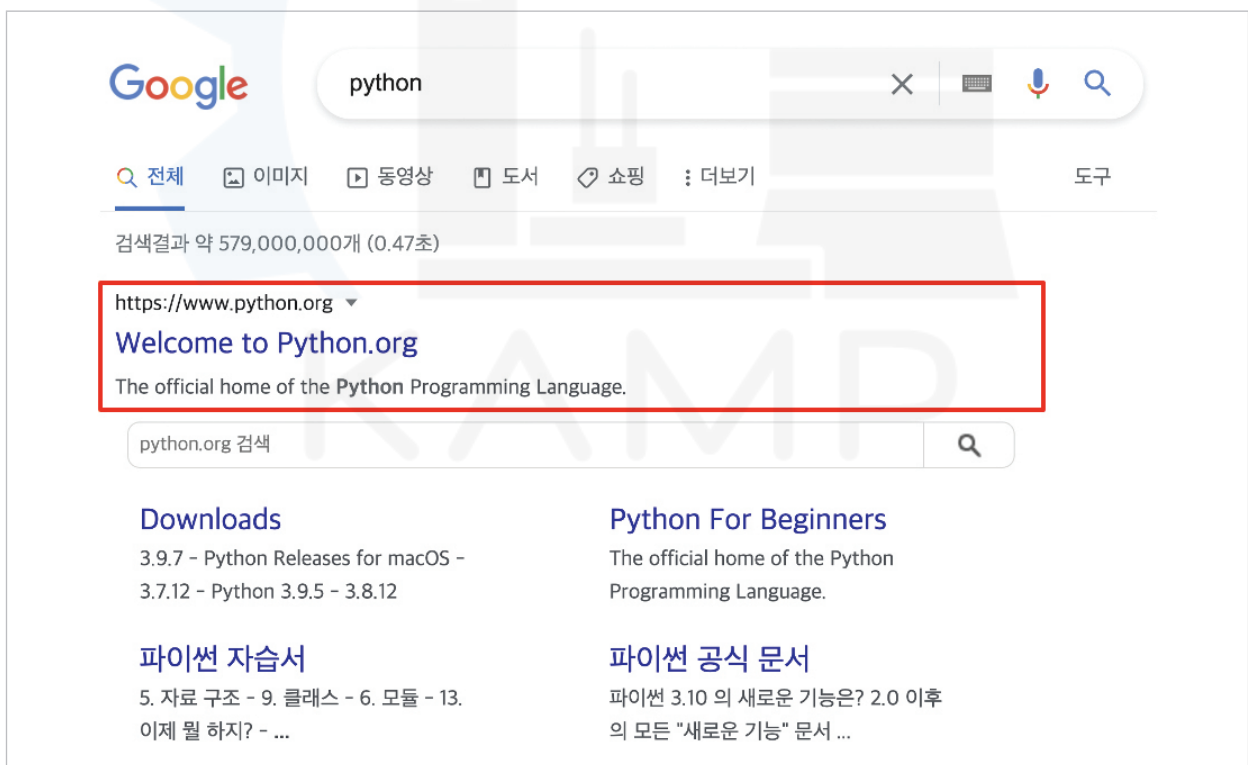
1. 파이썬(Python) 설치

파이썬은 컴퓨터 언어로서 최근 데이터 분석 및 AI 분석모델 개발 등에 널리 사용되는 도구이다. 다운로드 및 설치가 간편하고 활용도가 높은 파이썬을 로컬 컴퓨터에 설치하고 적용하는 방법을 안내한다.

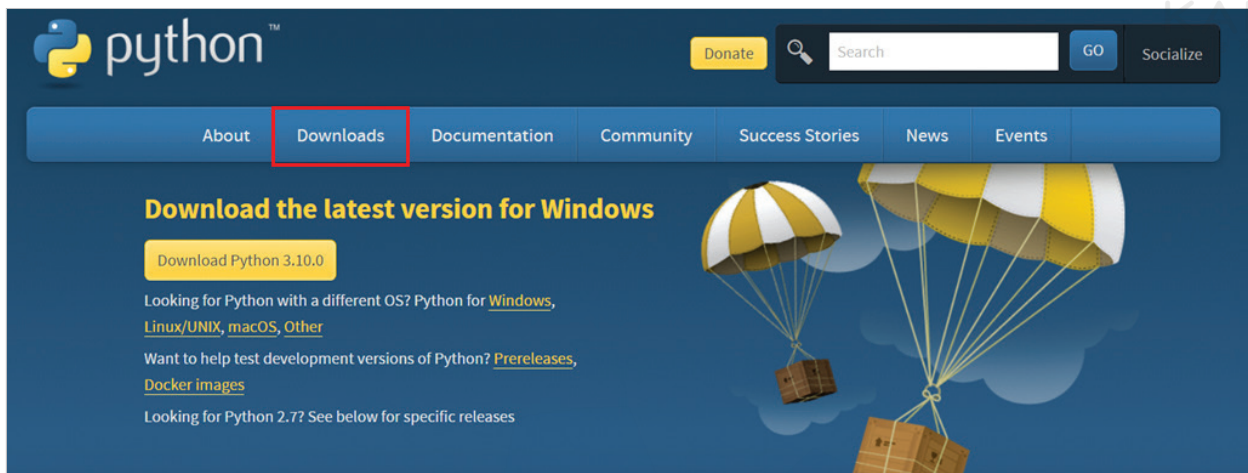
① google.com 등의 검색 엔진에 'python'을 검색한다.



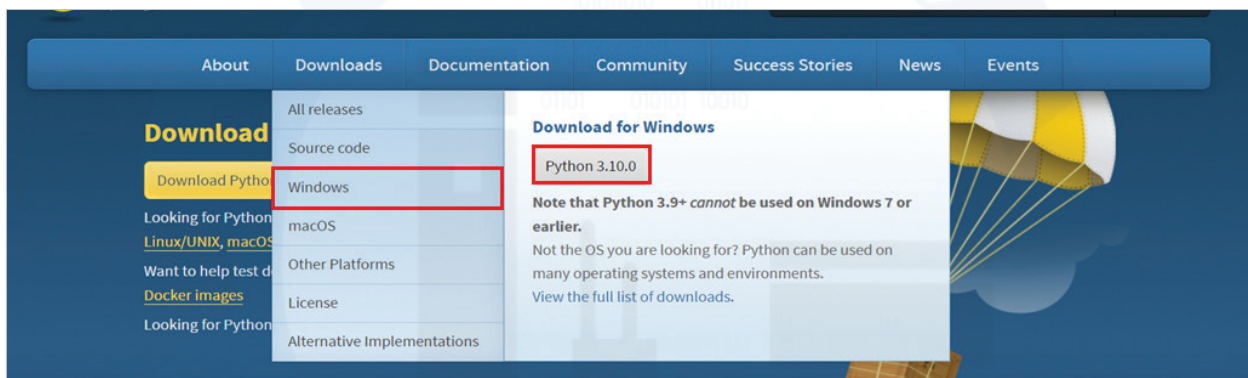
② 제일 처음에 보이는 'Welcome to Python.org'를 클릭한다.



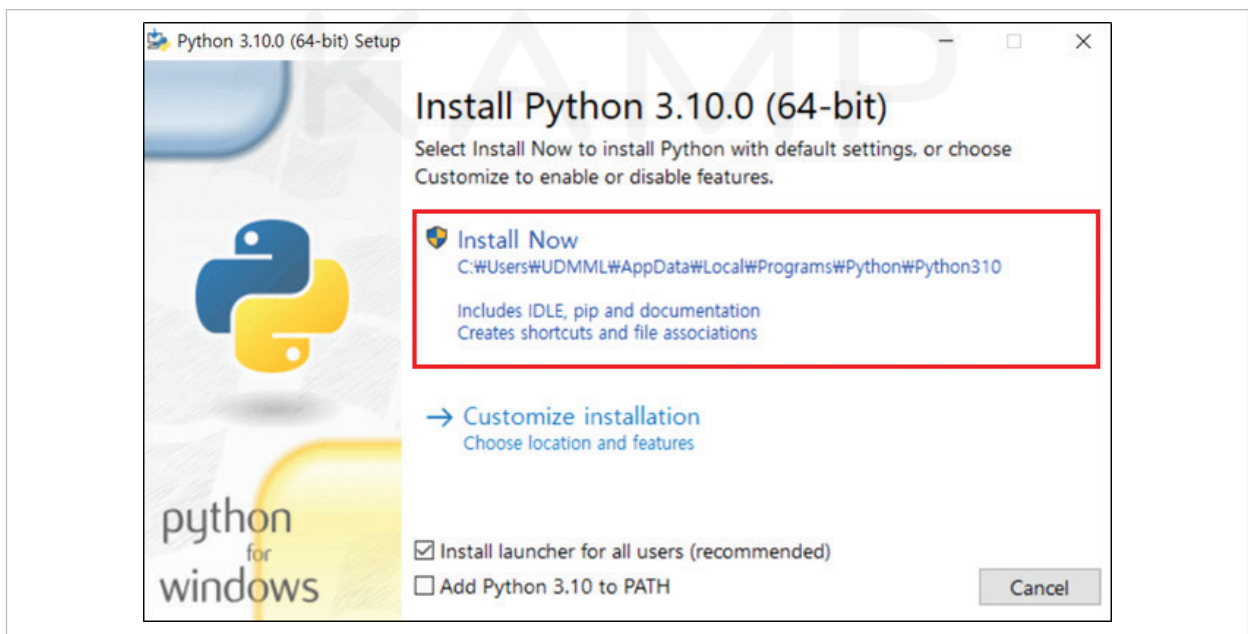
③ 클릭후 보이는 페이지의 정면에서 상단 좌측 2번째 'Downloads'를 클릭한다.



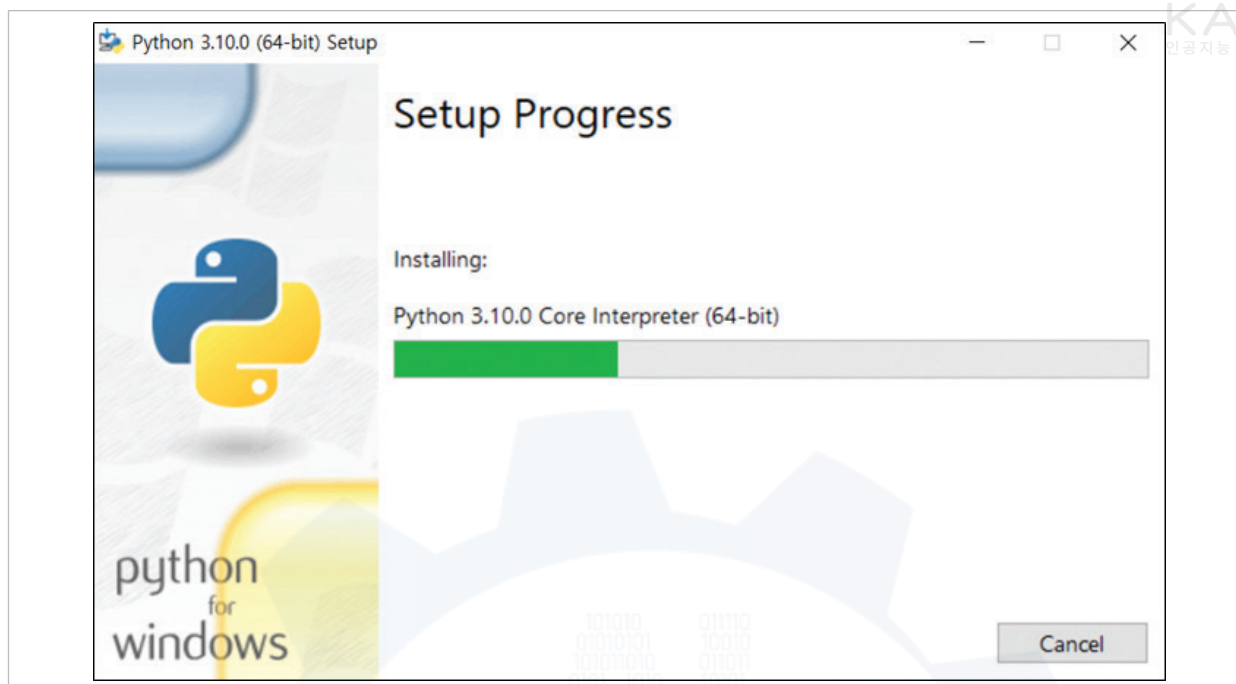
④ 컴퓨터의 운영체제에 따라 상단에서 세번째(macOS의 경우 네번째) 탭을 선택한 후, Python 3.10.0을 다운로드한다. (Python 3.10.0 버전은 업데이트 될 수 있다)



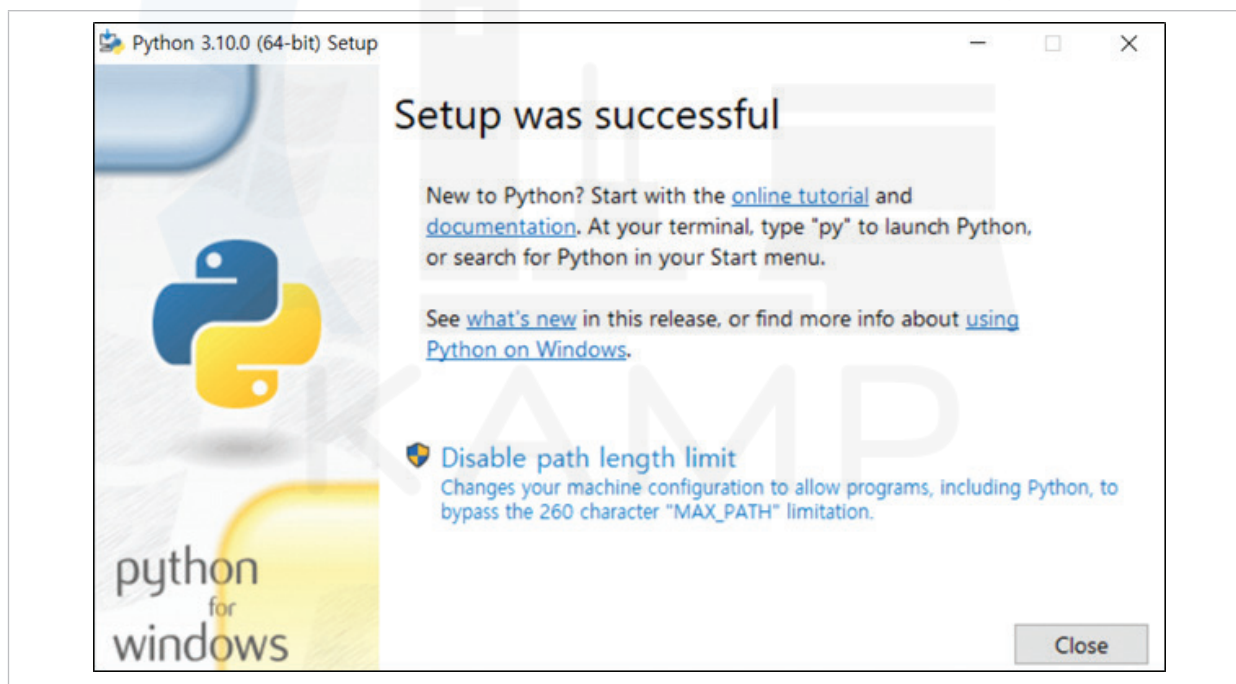
⑤ 아래와 같은 설치창이 뜨면, 'Install Now'를 클릭한다.



⑥ 아래와 같은 설치 진행창이 완료가 될 때까지 유지한다.



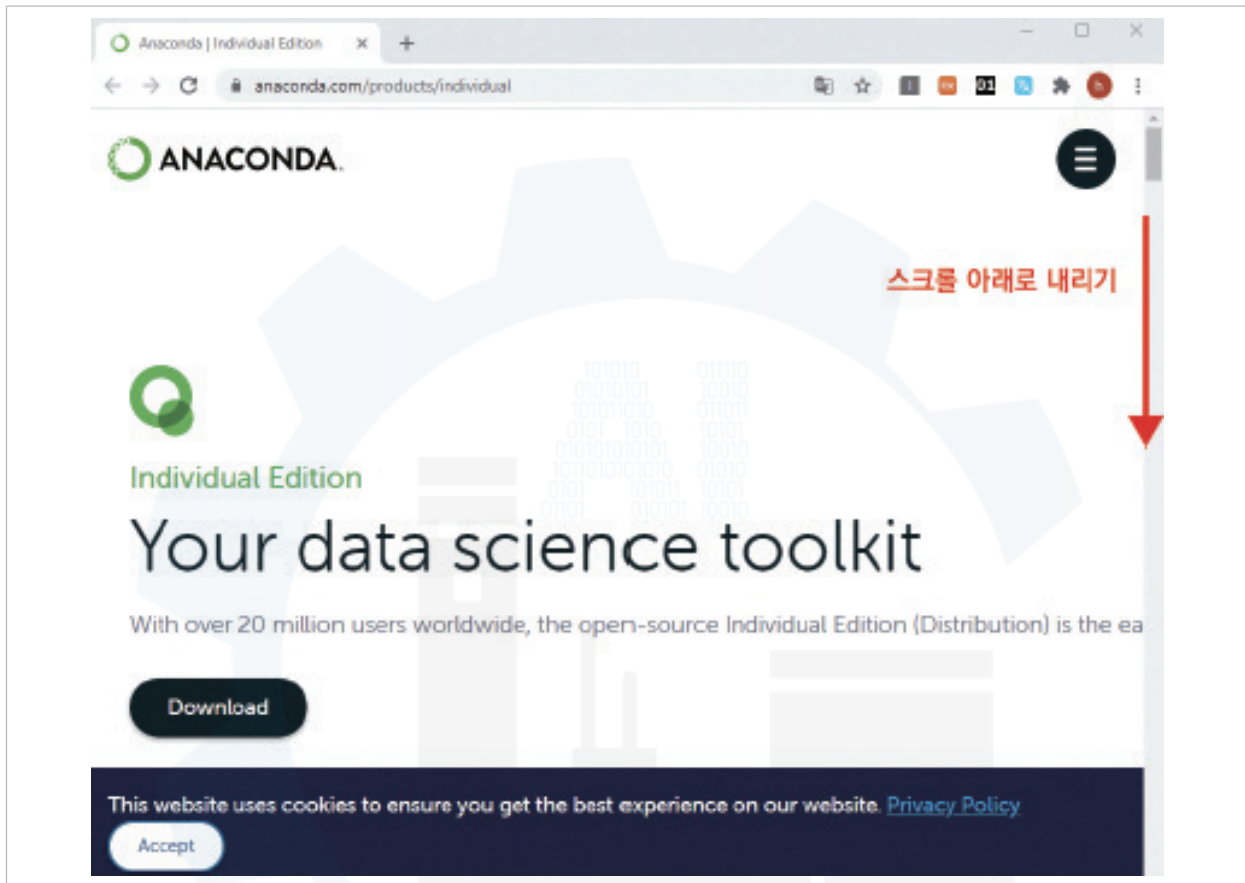
⑦ 완료가 되면 아래와 같은 창이 뜨는 것을 확인 후 종료한다. [설치완료]



2. 아나콘다(Anaconda) 설치

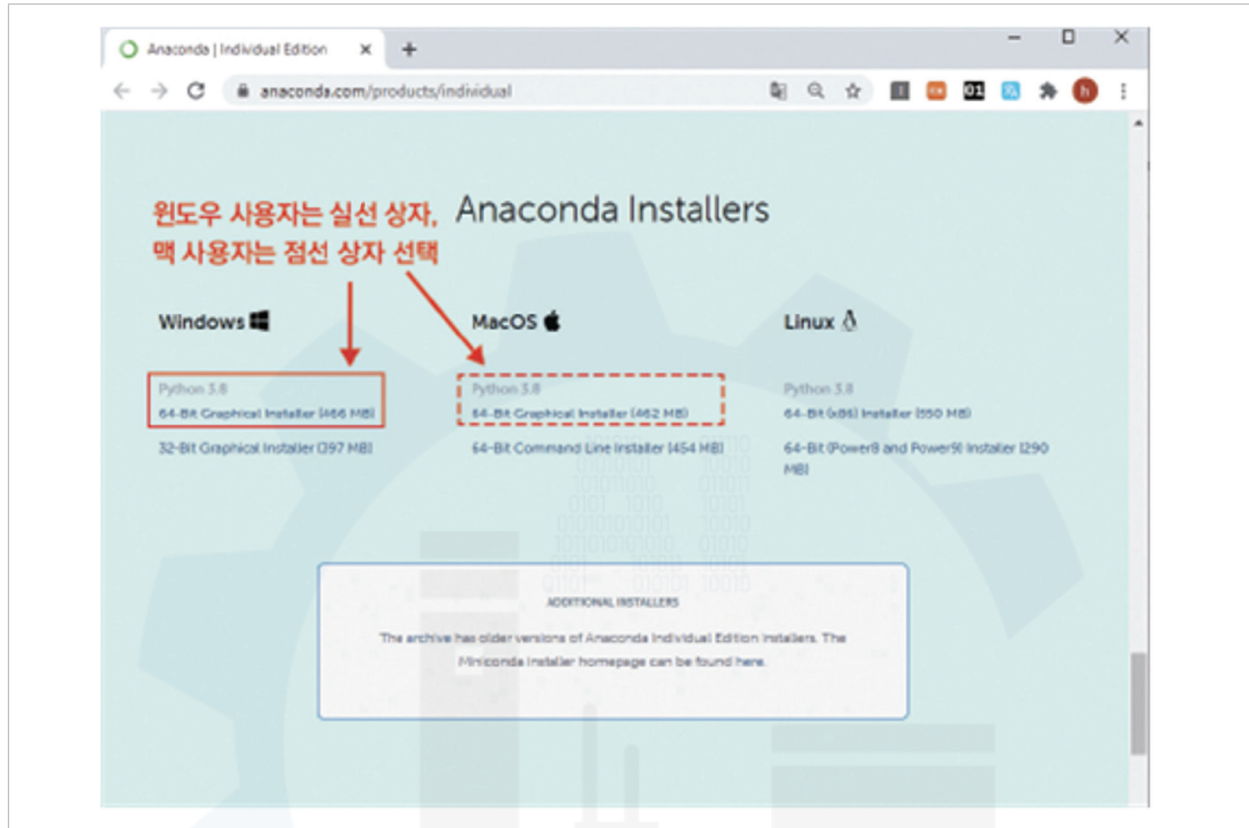
아나콘다란 파이썬과 같은 분석 도구를 사용할 때 필요한 고급 기능 및 분석을 보조하는 도구이다. 아나콘다를 설치함으로써 다양한 기능들을 바로 사용할 수 있고, 결과물을 쉽게 확인할 수 있다. 아나콘다를 설치하고 분석을 실시할 수 있는 환경을 구축하는 방법에 대해 안내한다.

① <https://www.anaconda.com/distribution/> 로 접속한다.



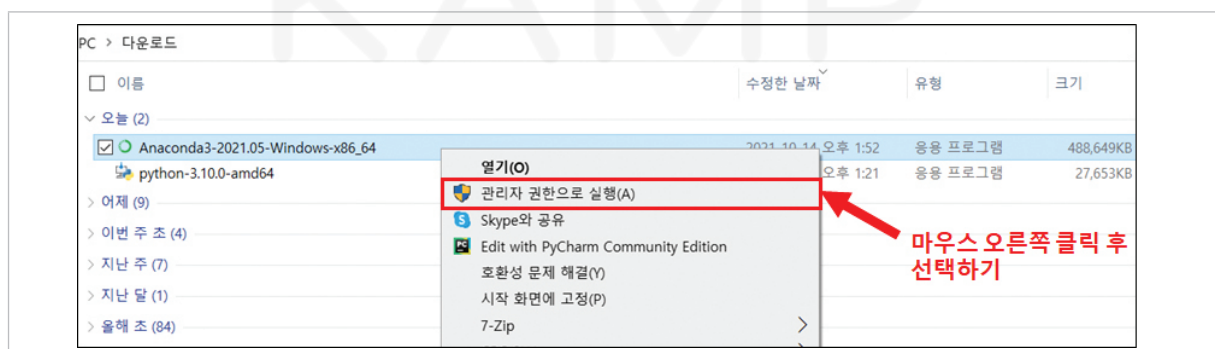
② 다음과 같은 화면이 나올 때 까지 스크롤하여 아래로 내린 후, 로컬 컴퓨터 사용 환경에 맞는 파일을 다운받는다.

- ▶ Windows : 64-Bit Graphical Installer
- ▶ MacOS : 64-Bit Graphical Installer

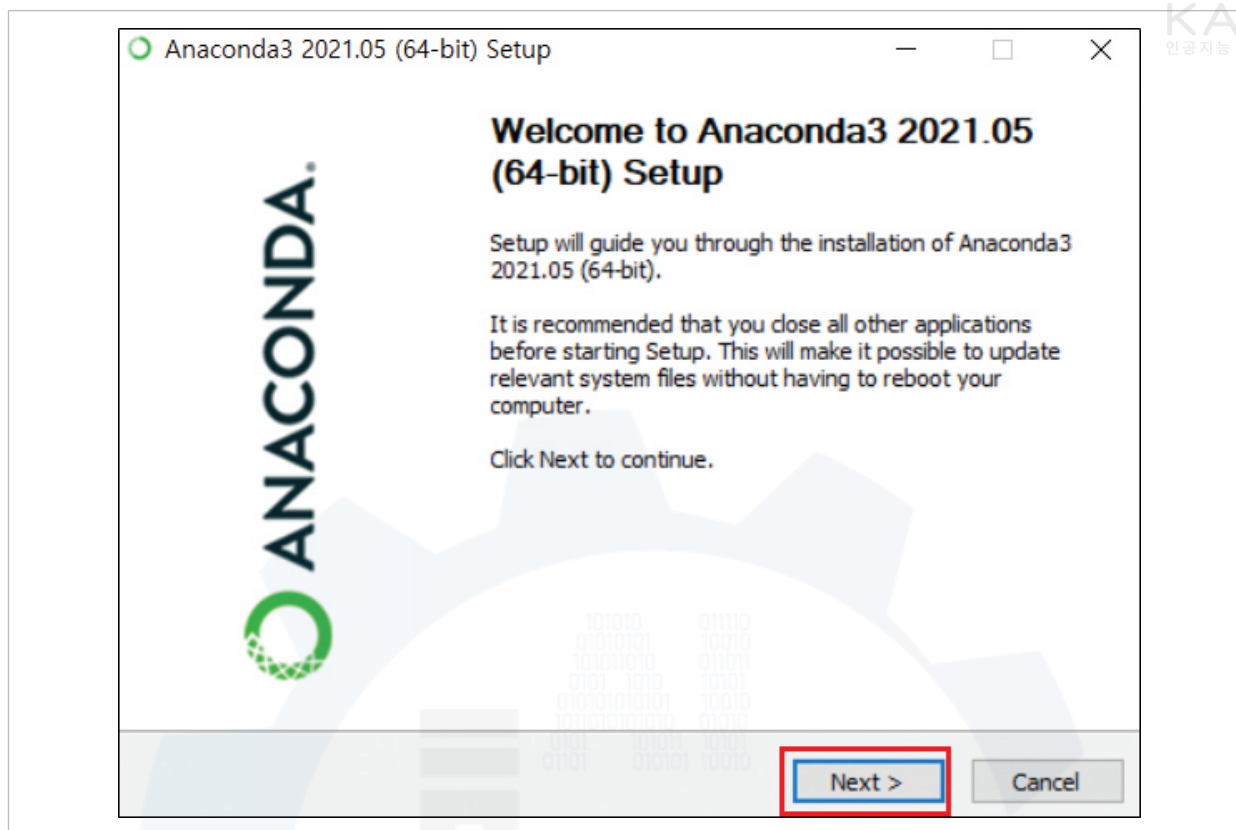


③ 다운로드 받은 파일로 이동하여, 설치된 아나콘다 파일 위에서 마우스 오른쪽을 클릭한 후, 방패모양의 '관리자 권한으로 실행'을 클릭한다.

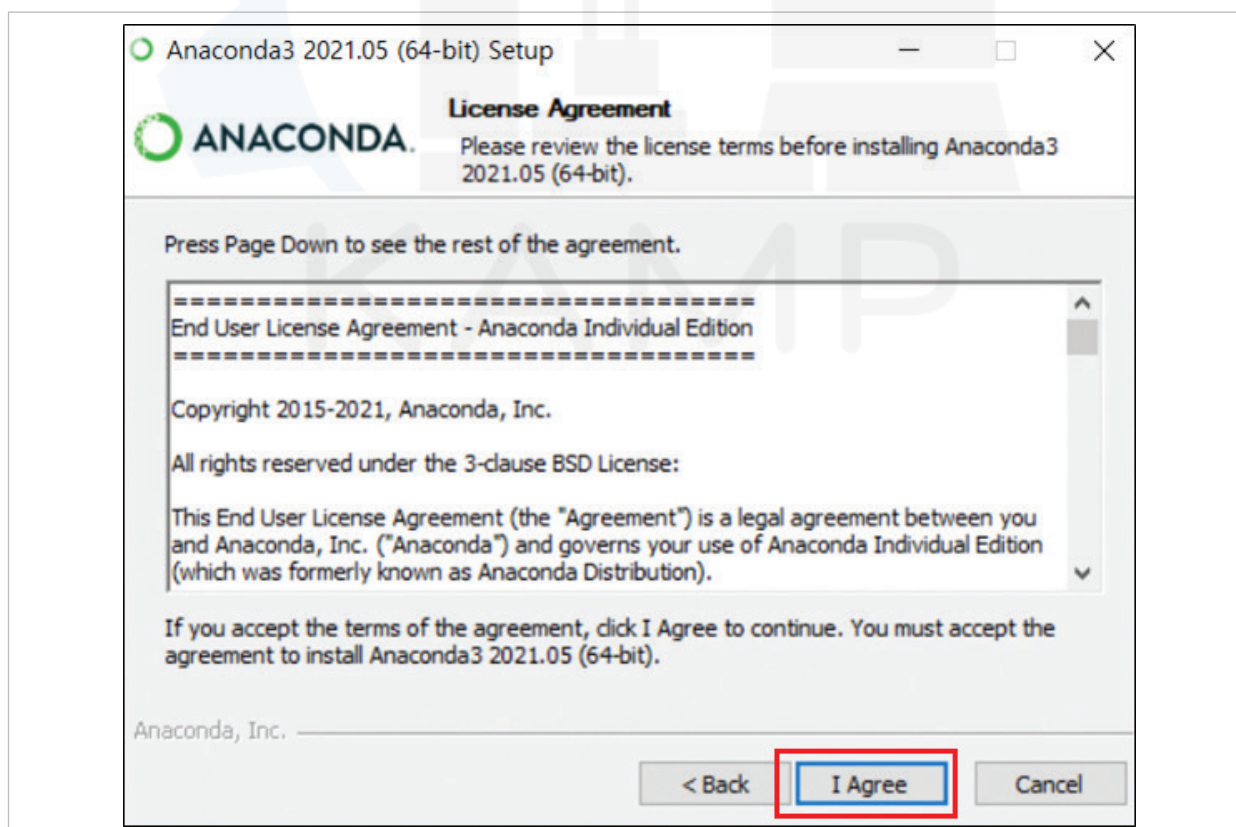
- ▶ (예) '다운로드' 폴더로 아나콘다를 다운 받은 경우



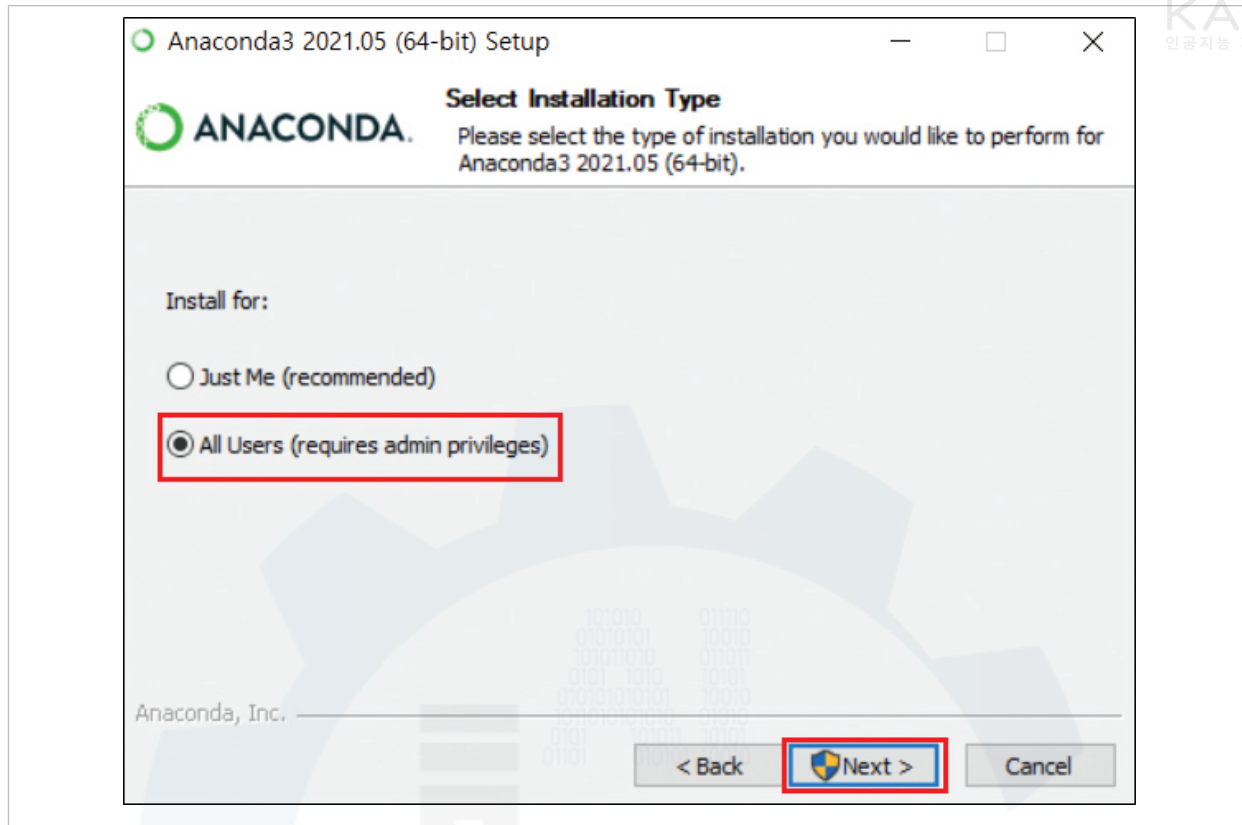
④ 파일을 실행 한 후, 'Next' 버튼을 클릭한다.



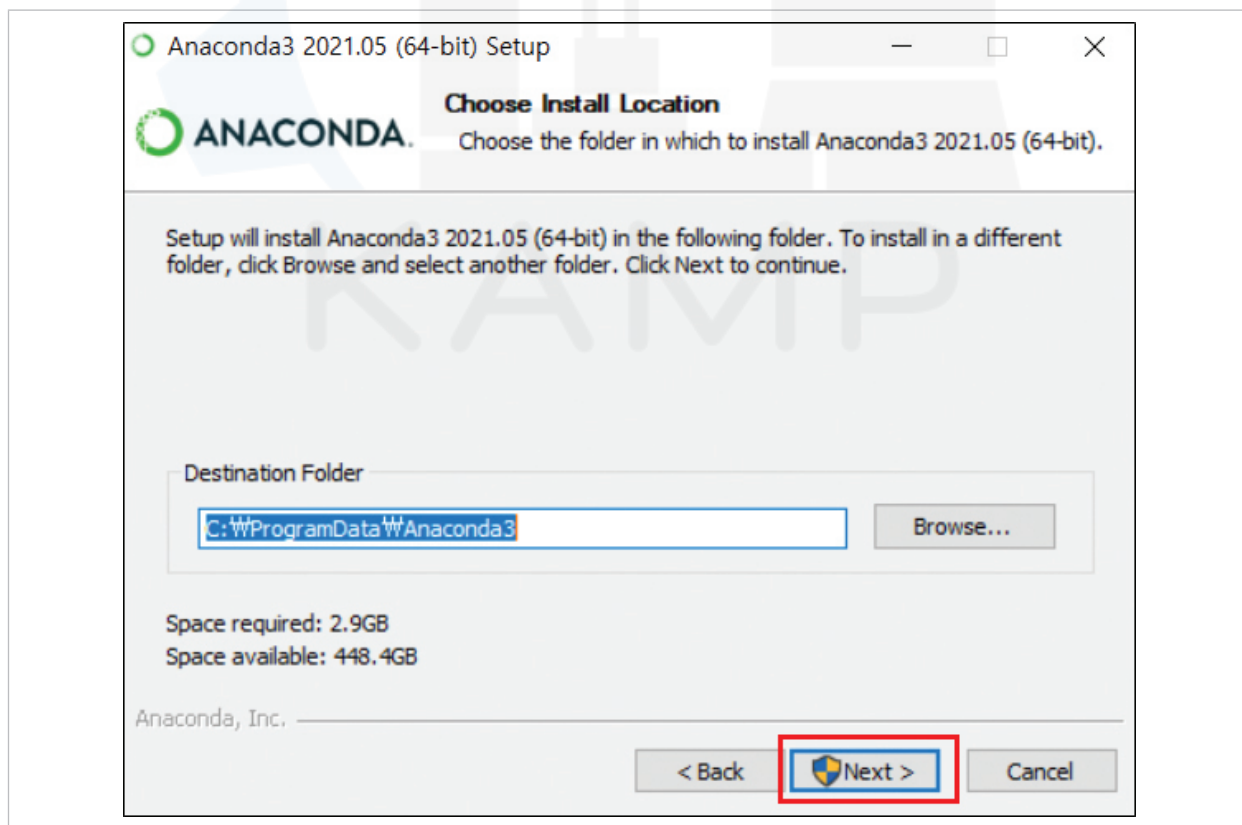
⑤ 다음 창이 나타나면 'I Agree'를 선택한다.



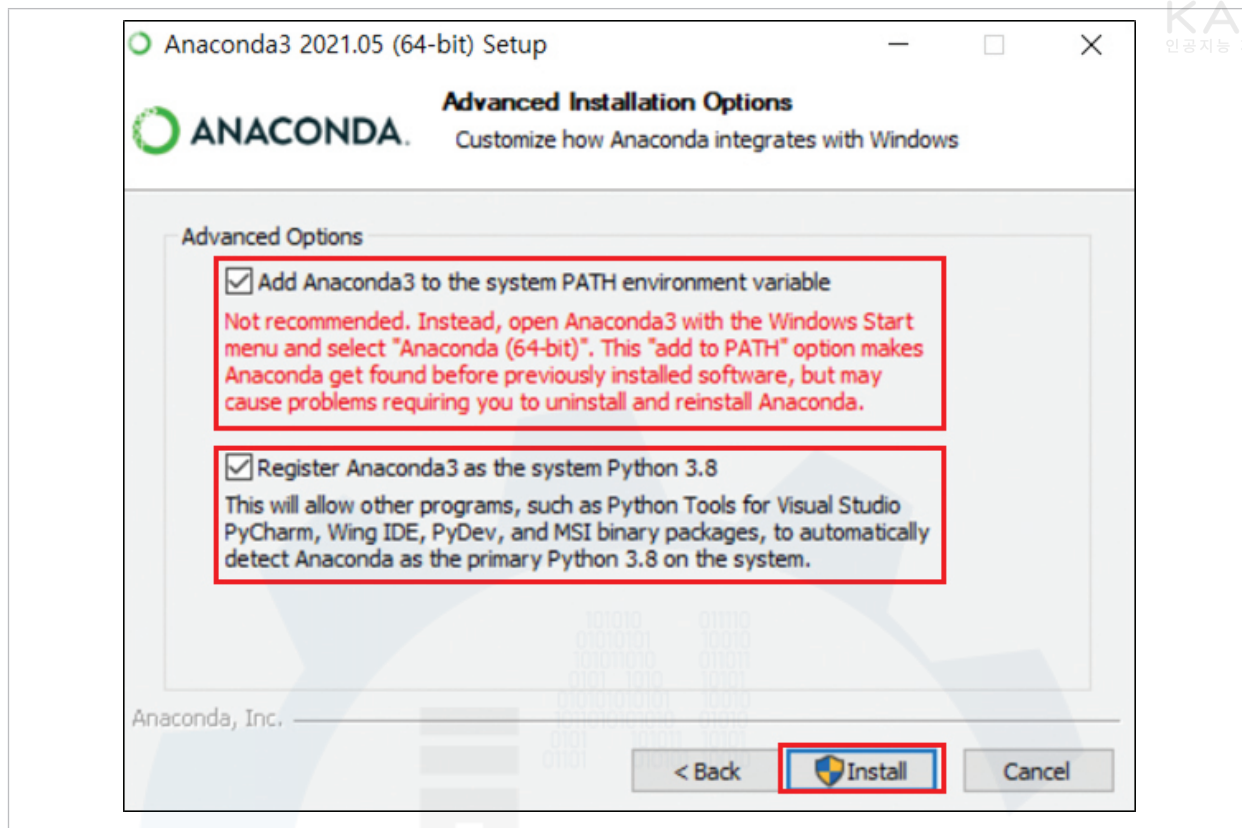
⑥ 셋팅 창이 뜨면 화면의 'All Users'를 선택 후 아래의 'Next'를 클릭한다.



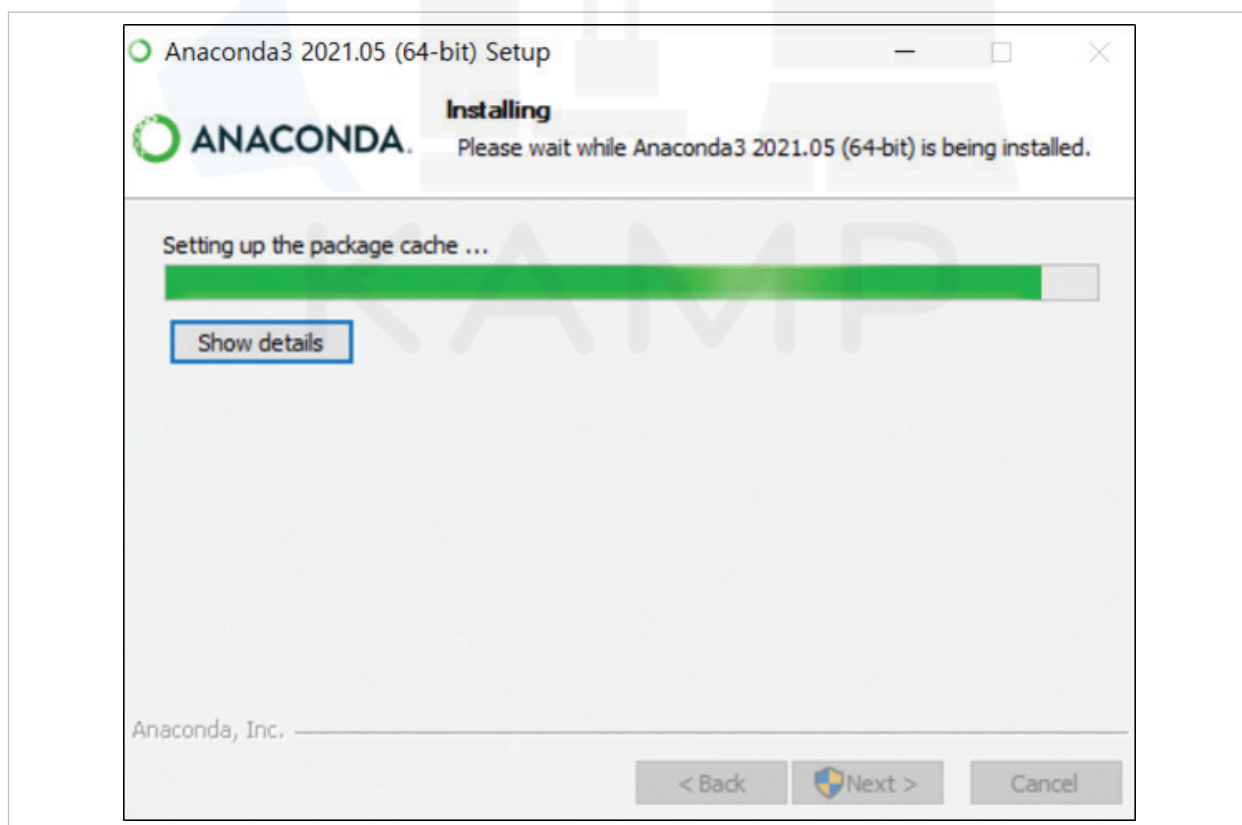
⑦ 다운로드 받을 경로를 물어보는 창이 뜨면, 아래의 'Next'를 클릭한다.



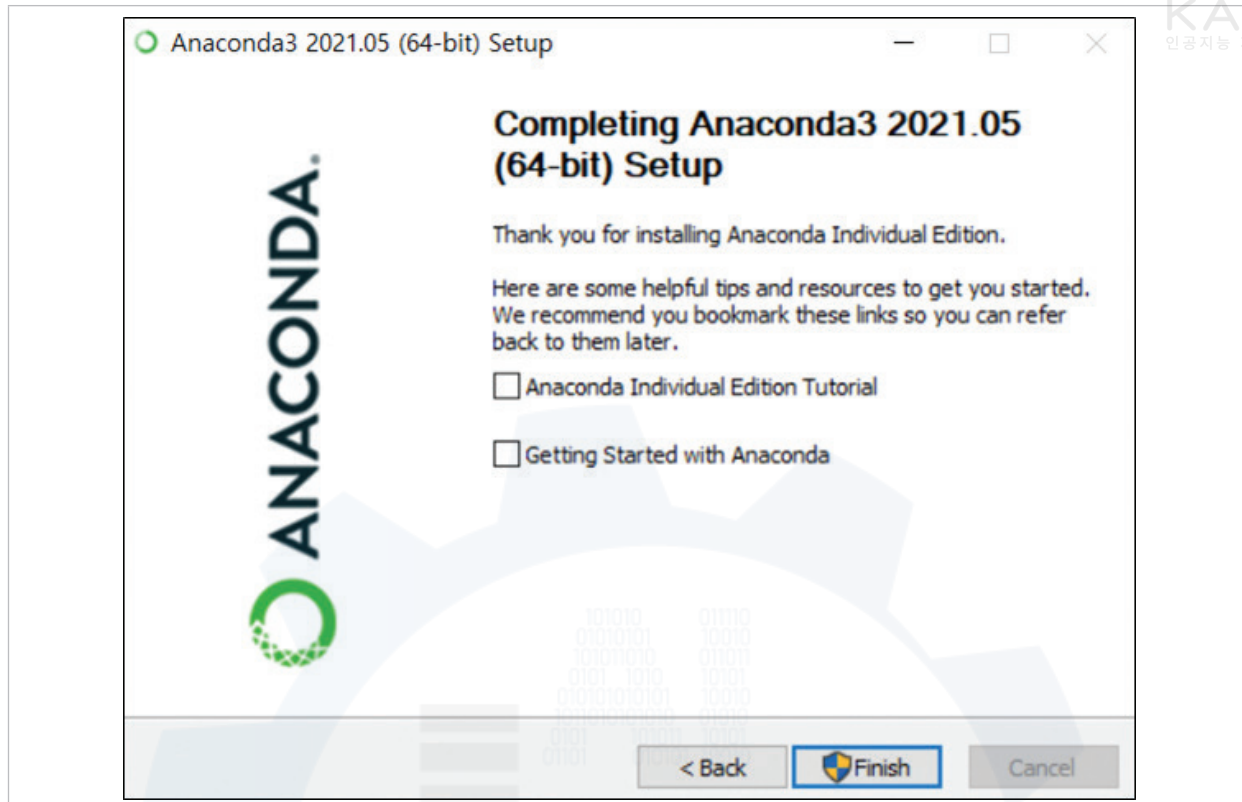
⑧ 고급 옵션 선택창이 뜨면, 아래와 같이 모두 선택 후, 'Install'을 클릭한다.



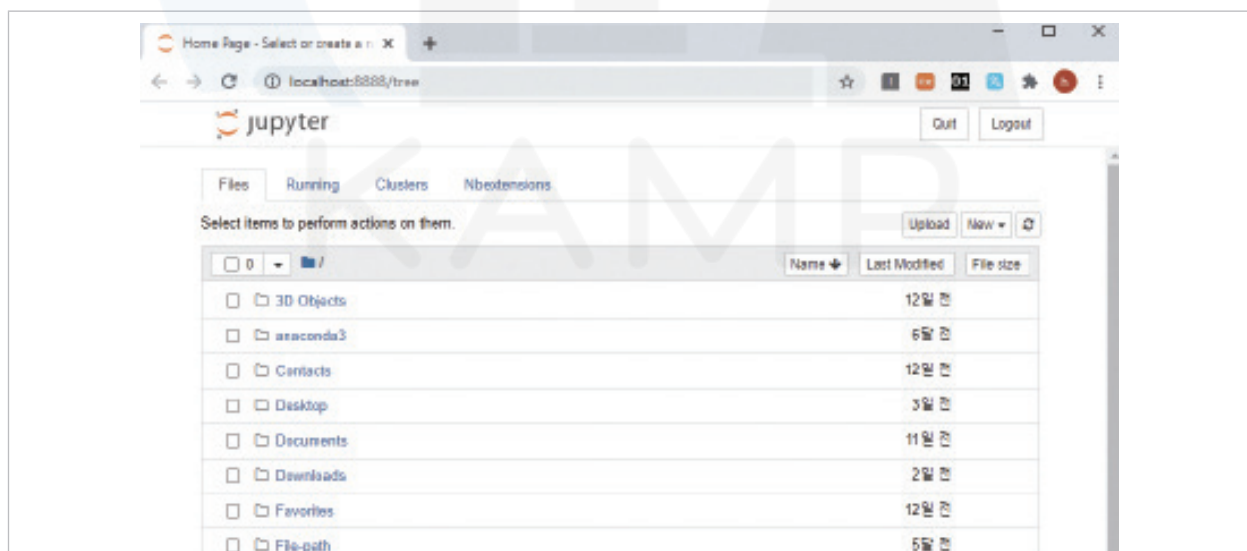
⑨ 다음과 같은 설치창이 뜨면 완료가 될 때까지 대기 (5분이상 소요)

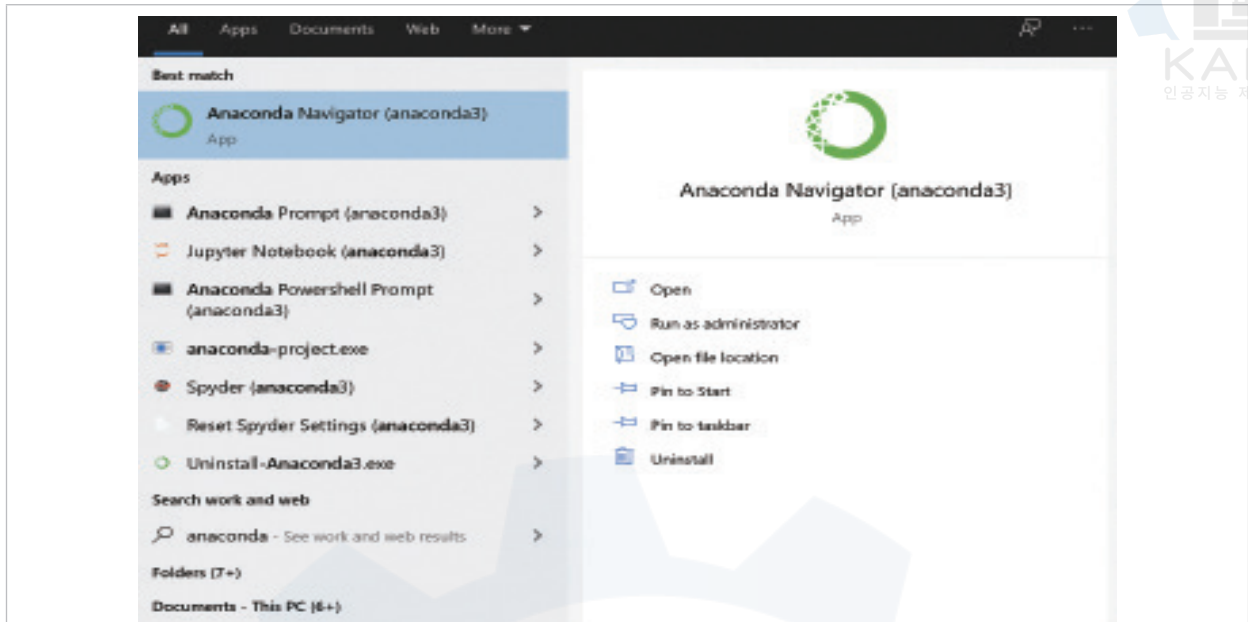


⑩ 마지막 화면에서, 모두 체크 해제한 후, 'Finish'를 눌러 설치를 완료한다.



⑪ 화면상의 '홈(Windows)'키를 눌러서 화면과 같이 anaconda prompt가 잘 설치 되었는지를 확인한다. Anaconda Navigator, Anaconda Prompt, Jupyter Notebook 등의 다른 응용 프로그램들도 함께 확인이 된다면 설치가 완료된 것이다.

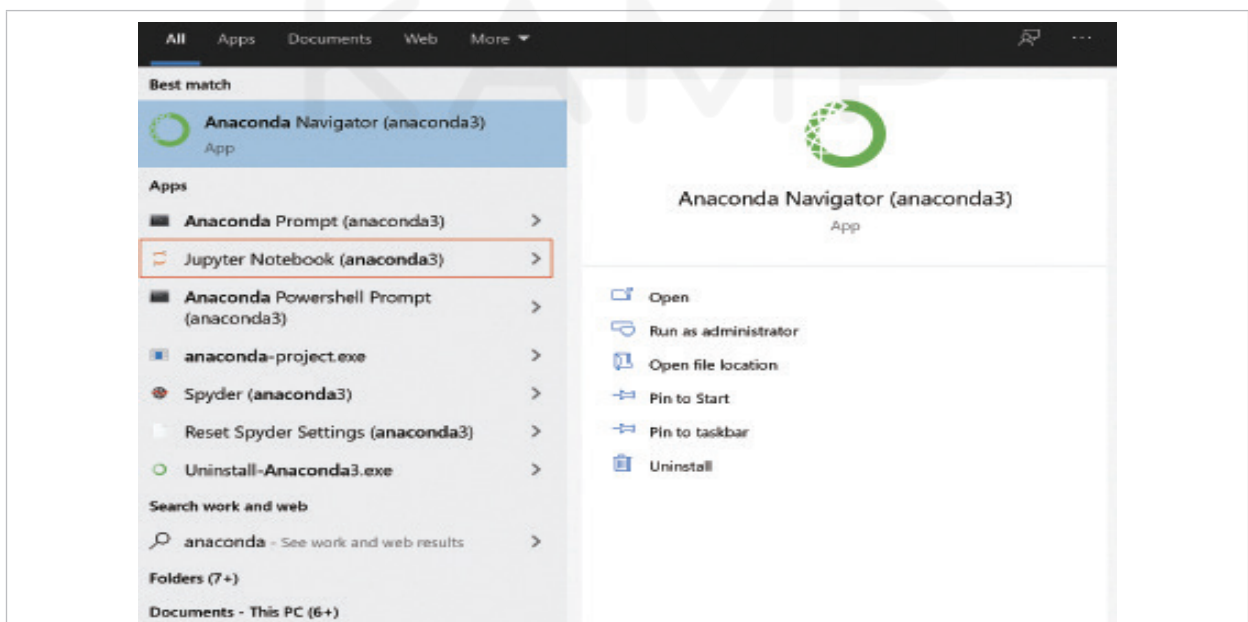




3. 주피터 노트북 (Jupyter Notebook) 실행

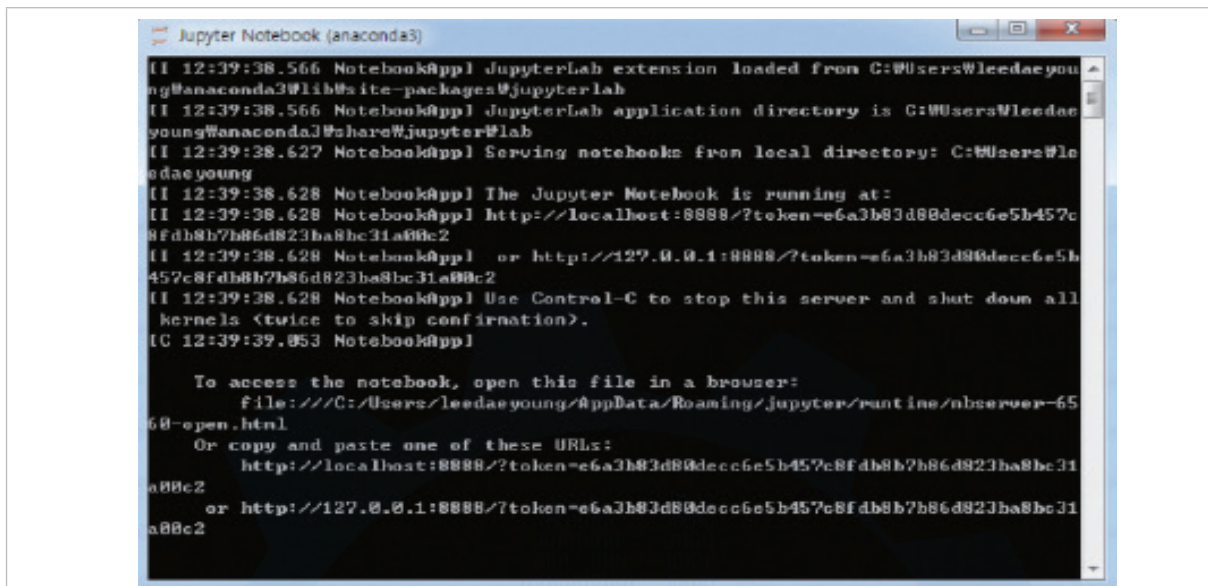
주피터 노트북은 실제로 사용자가 코딩(분석 문장 작성)을 할 수 있는 도구이다. 쉽게 비교하자면, 문서 도구로 마이크로소프트 사의 'Word' 프로그램이나, 한컴소프트 사의 '한글' 프로그램 등과 같은 도구라고 생각할 수 있다. 데이터 분석에 다양한 입력, 실행 도구가 있지만, 본 가이드 북에서는 주피터 노트북을 활용하는 방법을 안내하기로 한다. Anaconda를 설치한 이후 주피터 노트북(Jupyter Notebook) 설치 방법을 확인하면 된다.

- ① 화면상의 '홈(Windows)'키를 눌러서 화면과 같이 'anaconda' 검색 및 폴더 리스트 중 'Jupyter Notebook (anaconda3)'을 실행한다.



② Jupyter Notebook을 클릭시 아래처럼 2개의 윈도우가 실행된다.

- (1) 검은색 배경의 화면은 주피터 노트북이 실행되는 환경에 대한 상태를 나타내는 상태 표시 창이다. 주피터 노트북을 사용하는 동안 종료하면 안된다.



```

Jupyter Notebook (anaconda3)
[I 12:39:38.566 NotebookApp] JupyterLab extension loaded from C:\Users\Wleedae\you
ng\Anaconda3\lib\site-packages\jupyterlab
[I 12:39:38.566 NotebookApp] JupyterLab application directory is C:\Users\Wleedae
young\Anaconda3\share\jupyterlab
[I 12:39:38.627 NotebookApp] Serving notebooks from local directory: C:\Users\Wle
edae\young
[I 12:39:38.628 NotebookApp] The Jupyter Notebook is running at:
[I 12:39:38.628 NotebookApp] http://localhost:8888/?token=e6a3b83d88decc6e5b457c
8fd8b7b86d823ba8bc31a88c2
[I 12:39:38.628 NotebookApp] or http://127.0.0.1:8888/?token=e6a3b83d88decc6e5b
457c8fd8b7b86d823ba8bc31a88c2
[I 12:39:38.628 NotebookApp] Use Control-C to stop this server and shut down all
kernels (twice to skip confirmation).
[C 12:39:39.053 NotebookApp]

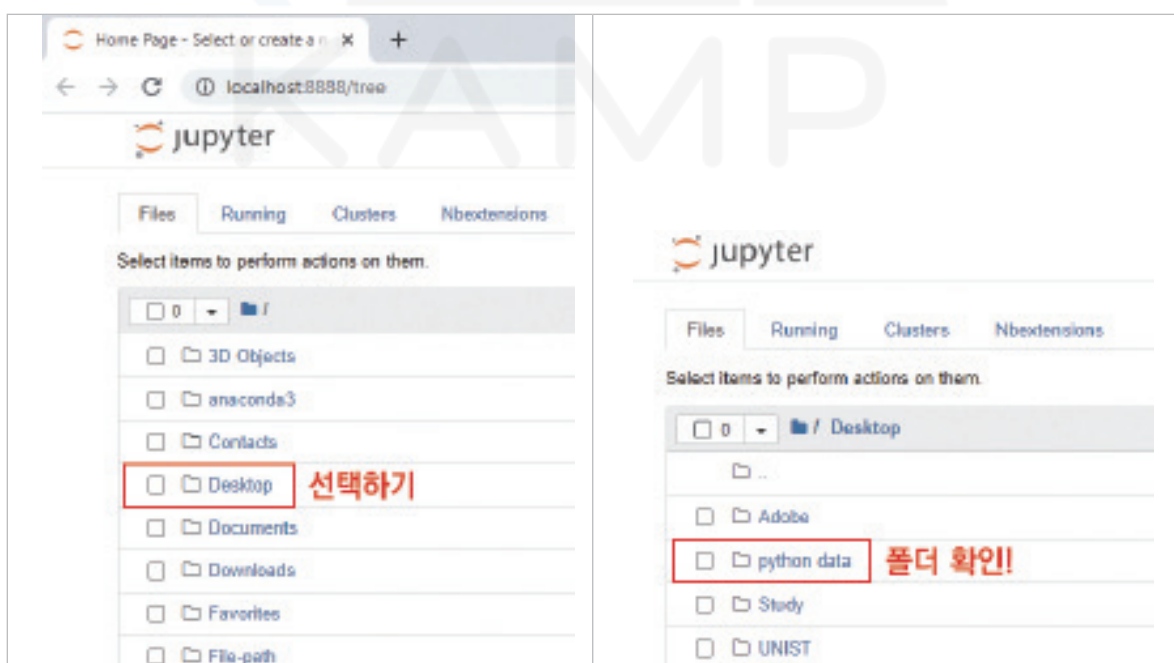
To access the notebook, open this file in a browser:
file:///C:/Users/leedae/young/AppData/Roaming/jupyter/runtime/nbserver-65
68-open.html
Or copy and paste one of these URLs:
http://localhost:8888/?token=e6a3b83d88decc6e5b457c8fd8b7b86d823ba8bc31
a88c2
or http://127.0.0.1:8888/?token=e6a3b83d88decc6e5b457c8fd8b7b86d823ba8bc31
a88c2
  
```

- (2) 사용하는 인터넷 프로그램(크롬, 인터넷 익스플로러)에 주피터 노트북이 열린다. (다른 확 장 프로그램 사용하고 싶다면 기본 브라우저를 변경해주어야한다)

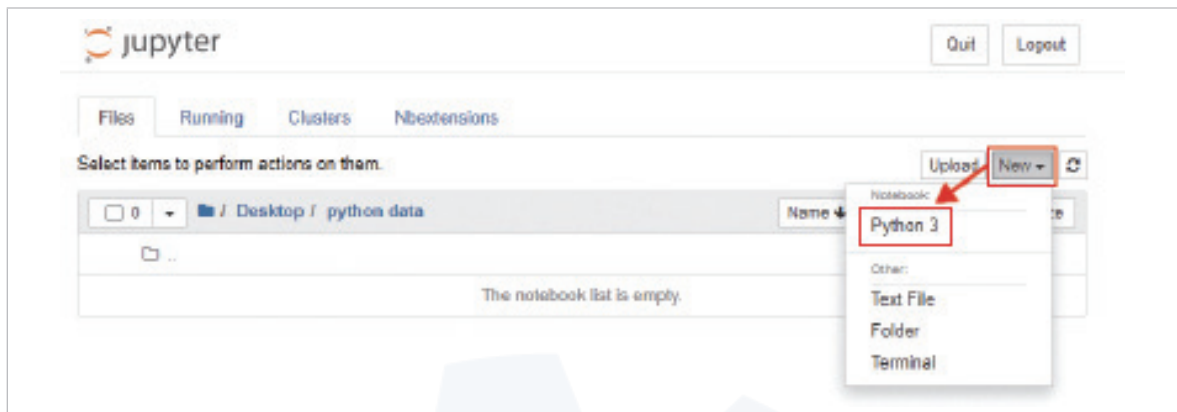
③ 데이터를 저장하고, 불러오고, 분석할 경로의 폴더를 하나 생성한다.

- ▶ (예) '바탕화면'에 'python data' 폴더를 생성 후 (미리 생성), 파이썬 코드 실행하고 저장한다.

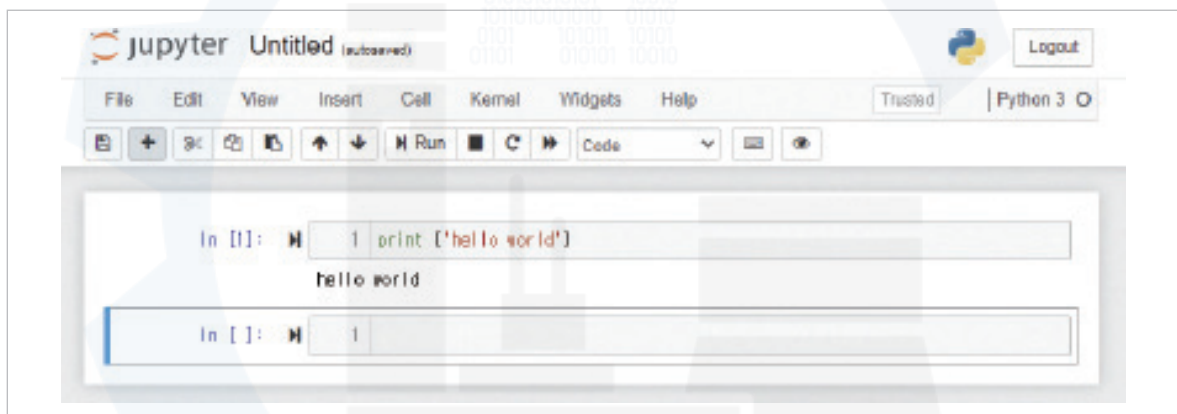
- (1) 주피터 노트북에서 'python data' 폴더를 확인한다.



- (2) python data에서 파이썬 파일을 생성한다. 오른쪽 상단의 'New'를 누른 후, 'Python 3'을 선택한다.



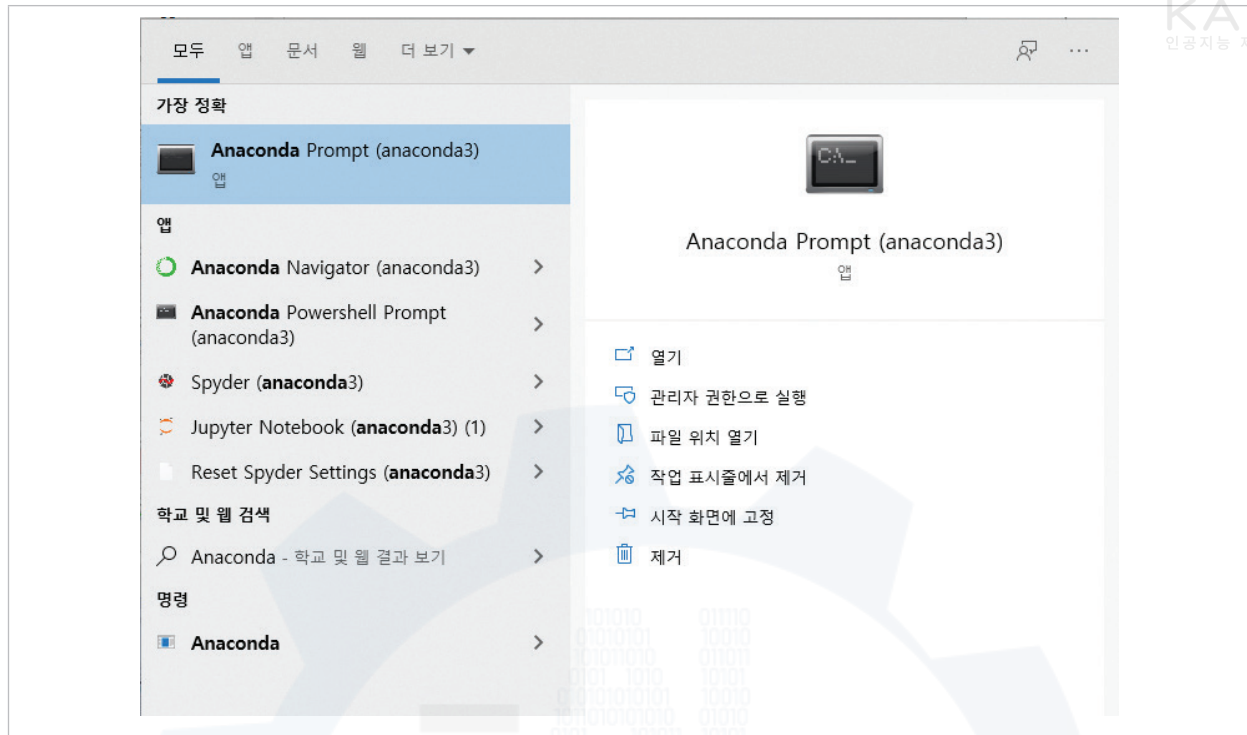
- (3) 'hello world' 출력을 확인해본다. 보이는 In [] 우측 회색 창에 print('hello world') 입력 후, 상단의 [Run] 버튼 혹은 shift + Enter 키를 눌러서 실행한다. 코드 파일의 확장자는 '.ipynb'로 저장된다.



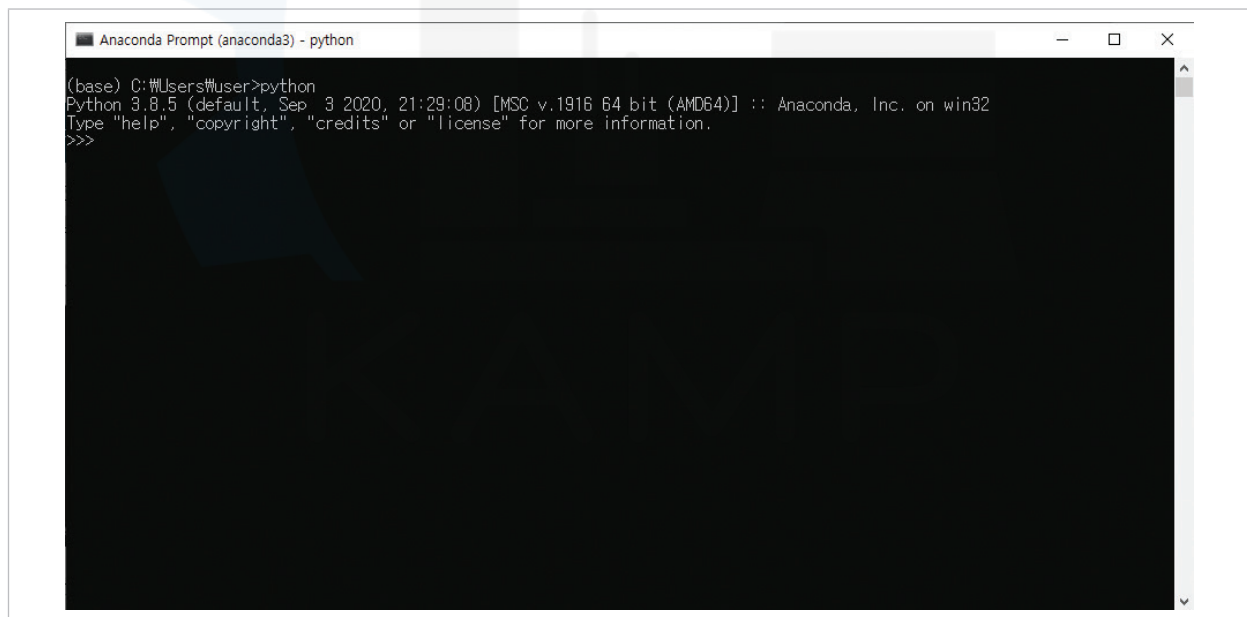
4. 아나콘다 가상환경 설정

가상환경 : 독립적인 작업환경에서 패키지 및 버전 관리를 하기 위한 가상의 환경을 의미한다. 각 패키지(모듈, 라이브러리)는 버전에 따라 의존성을 갖고 있으므로 오류가 발생할 수 있으며, 이를 관리하기 위해 가상환경을 사용하여 가상환경마다 다른 패키지 버전을 사용할 수 있다.

① 'Anaconda Prompt' 실행

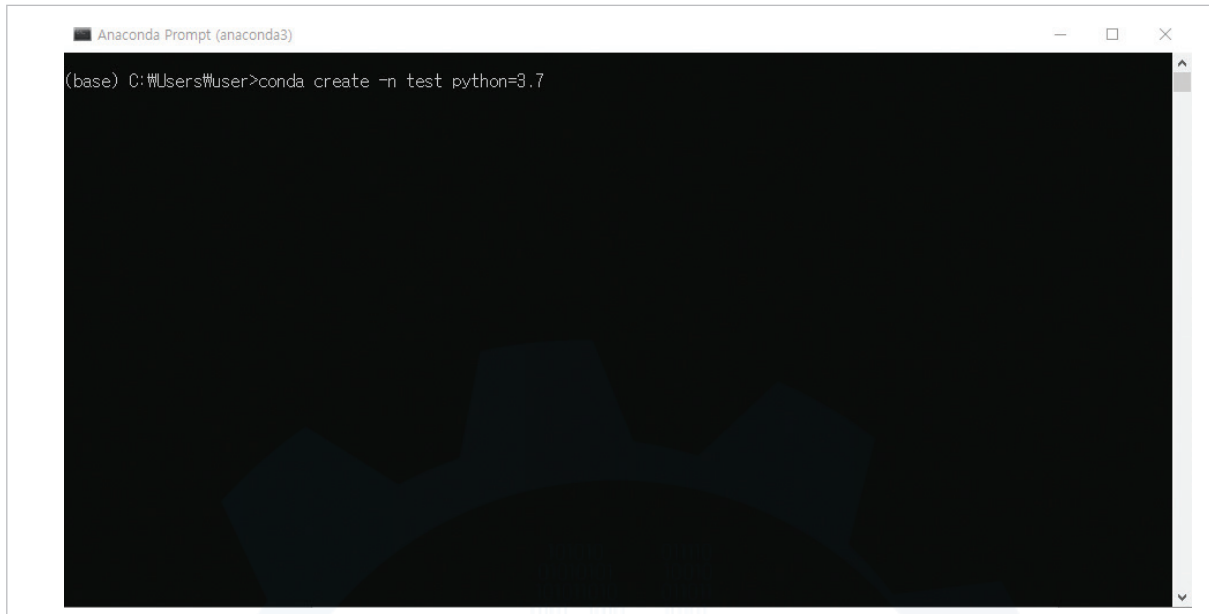


* 'Anaconda Prompt'는 실습을 진행하는 동안 종료시키면 안 되며, 켜둔 상태로 진행



② 가상환경 생성

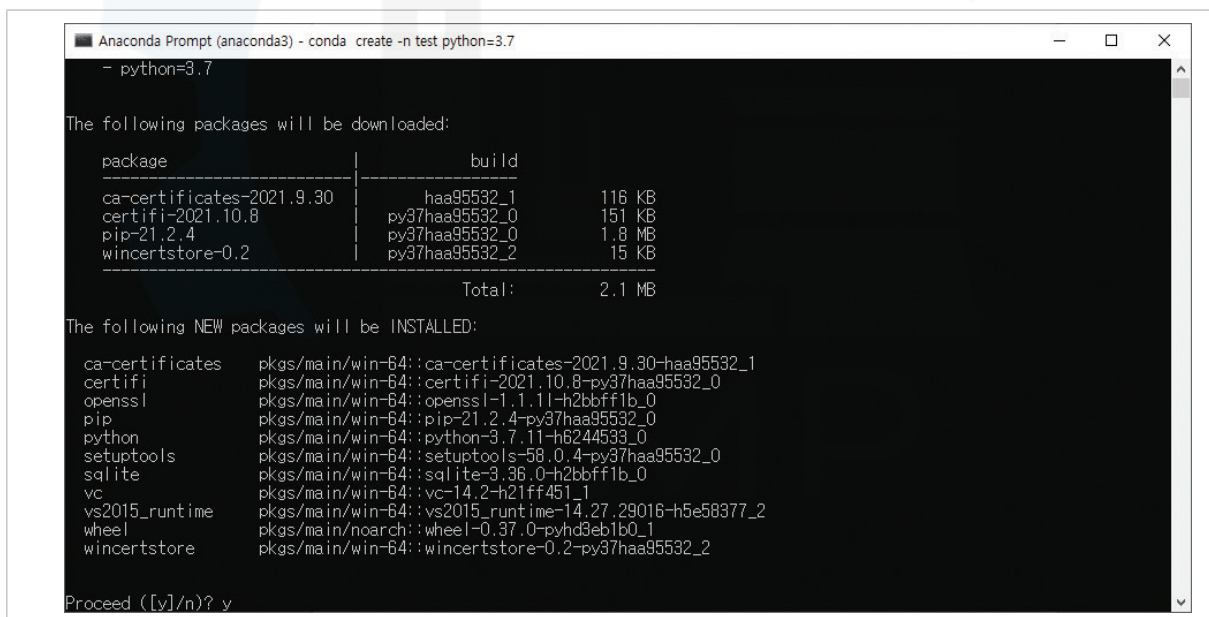
1) 'conda create -n test python=3.7' 입력 후 엔터



```

Anaconda Prompt (anaconda3)
(base) C:\Users\User>conda create -n test python=3.7
  
```

2) 'y' 입력 후 엔터



```

Anaconda Prompt (anaconda3) - conda create -n test python=3.7
- python=3.7

The following packages will be downloaded:

package | build | size
-----|-----|-----
ca-certificates-2021.9.30 | haa95532_1 | 116 KB
certifi-2021.10.8 | py37haa95532_0 | 151 KB
pip-21.2.4 | py37haa95532_0 | 1.8 MB
winertstore-0.2 | py37haa95532_2 | 15 KB
Total: 2.1 MB

The following NEW packages will be INSTALLED:

ca-certificates | pkgs/main/win-64::ca-certificates-2021.9.30-haa95532_1
certifi | pkgs/main/win-64::certifi-2021.10.8-py37haa95532_0
openssl | pkgs/main/win-64::openssl-1.1.1-h2bfff1b_0
pip | pkgs/main/win-64::pip-21.2.4-py37haa95532_0
python | pkgs/main/win-64::python-3.7.11-h6244533_0
setuptools | pkgs/main/win-64::setuptools-58.0.4-py37haa95532_0
sqlite | pkgs/main/win-64::sqlite-3.36.0-h2bfff1b_0
vc | pkgs/main/win-64::vc-14.2-h21ff451_1
vs2015_runtime | pkgs/main/win-64::vs2015_runtime-14.27.29016-h5e58377_2
wheel | pkgs/main/noarch::wheel-0.37.0-pyhd9eb1b0_1
winertstore | pkgs/main/win-64::winertstore-0.2-py37haa95532_2

Proceed ([y]/n)? y
  
```

3) 가상환경 생성 완료

```

Anaconda Prompt (anaconda3)
done
#
# To activate this environment, use
#
#     $ conda activate test
#
# To deactivate an active environment, use
#
#     $ conda deactivate
#
(base) C:\Users\User>

```

4) 가상환경 진입 - 'conda activate test' 입력 후 엔터

```

Anaconda Prompt (anaconda3)
done
#
# To activate this environment, use
#
#     $ conda activate test
#
# To deactivate an active environment, use
#
#     $ conda deactivate
#
(base) C:\Users\User>conda activate test
(test) C:\Users\User>

```

5) 가상환경 종료 - 'conda deactivate' 입력 후 엔터

```

Anaconda Prompt (anaconda3) - conda install jupyter notebook - conda install jupyter notebook - conda deactivate
(test) C:\Users\User>conda deactivate
(base) C:\Users\User>

```

5. 주피터 노트북(Jupyter Notebook) 설정

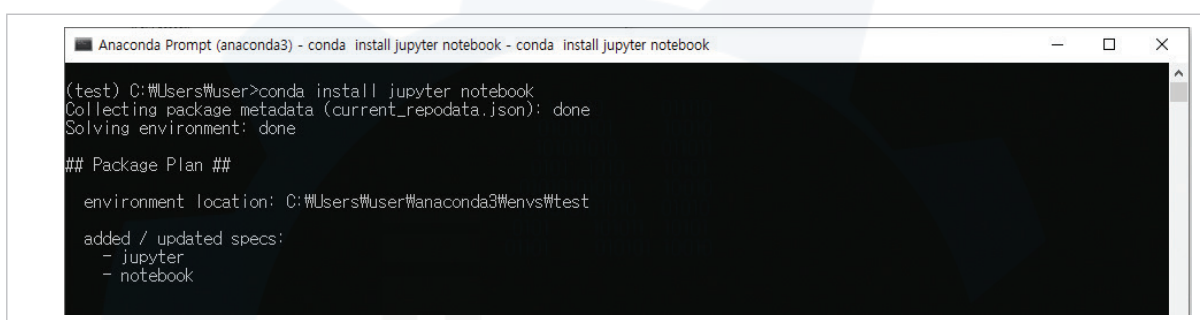
주피터 노트북(Jupyter Notebook) : 파이썬 코딩을 할 수 있는 대화형 인터프리터 (Interpreter)로 웹 브라우저 환경에서 파이썬 코드를 작성 및 실행할 수 있는 툴이다.

① 가상환경 진입

‘conda activate test’ 입력 후 엔터

② 주피터 노트북 설치

‘conda install jupyter notebook’ 입력 후 엔터, 중간에 ‘y’ 입력 후 엔터



```

Anaconda Prompt (anaconda3) - conda install jupyter notebook - conda install jupyter notebook

(test) C:\Users\User>conda install jupyter notebook
Collecting package metadata (current_repodata.json): done
Solving environment: done

## Package Plan ##

  environment location: C:\Users\User\anaconda3\envs\test

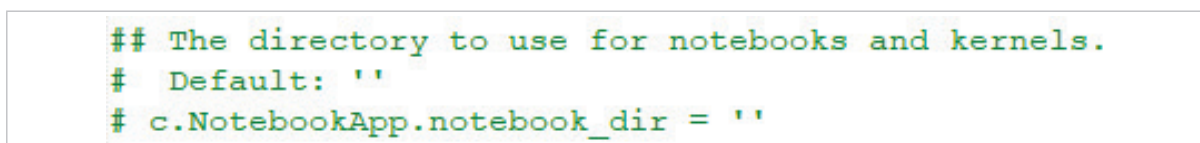
added / updated specs:
- jupyter
- notebook
  
```

③ 주피터 노트북 테스트

‘jupyter notebook’ 입력 후 엔터

④ 주피터 노트북 시작 폴더 설정

- 1) ‘jupyter notebook --generate-config’ 입력 후 엔터
- 2) 입력 후 아래에 출력되는 경로에 진입하여 ‘jupyter_notebook_config.py’ 파일을 메모장으로 열기
- 3) 메모장에서 ‘#c.NotebookApp.notebook_dir=”’ 문장 찾기
- 4) ‘#c.NotebookApp.notebook_dir=”’를 찾아서 ‘c.NotebookApp.notebook_dir=folder_directory’로 변경



```

## The directory to use for notebooks and kernels.
# Default: ''
# c.NotebookApp.notebook_dir = ''
  
```

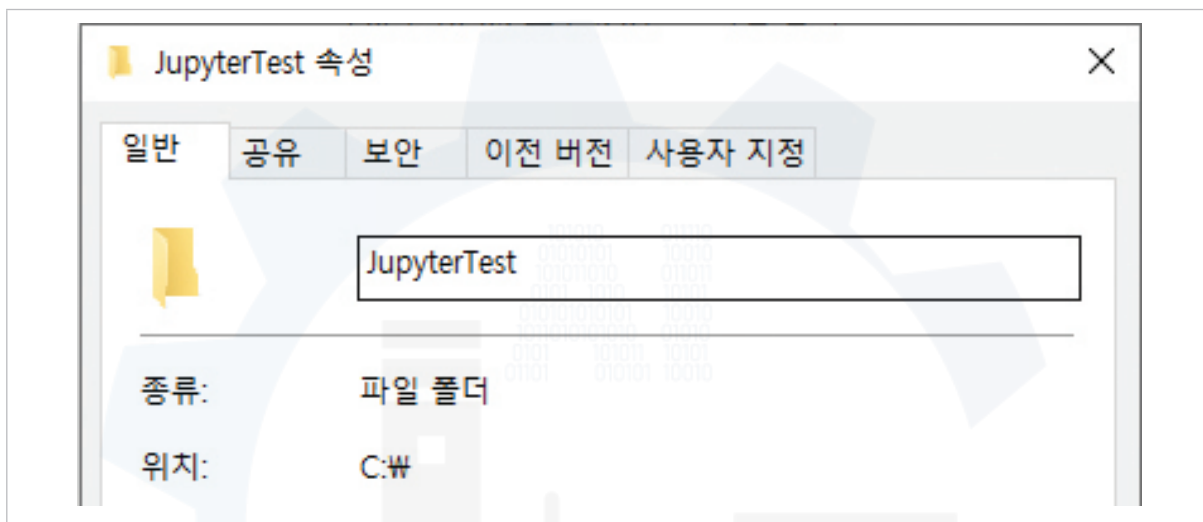
[그림 37] 기존 가상환경 Base Directory 코드

```
## The directory to use for notebooks and kernels.  
# Default: ''  
c.NotebookApp.notebook_dir = 'C:\JupyterTest'
```

[그림 38] 변경하는 가상환경 Directory 코드

* folder_directory는 주피터 노트북을 시작하고 저장하는 기본 폴더로 해당 위치에 모든 작업물이 저장된다. 시작 전 folder_directory를 해당 위치에 생성시켜줘야 함.

* folder_directory 예시 : 'C:\JupyterTest' (folder_directory는 작은따옴표까지 포함되어야 함)



5) 변경 완료 후 저장 및 메모장 종료

⑤ 주피터 노트북 커널 추가

- 1) 'pip install ipykernel' 입력 후 엔터
- 2) 'python -m ipykernel install --user --name test --display-name "test"' 입력 후 엔터

※ 아나콘다를 이용하는 이유

데이터 분석이나 데이터 모델링을 하다 보면 패키지나 라이브러리(library)에 따라 같은 Python을 쓰더라도 3.6을 써야 할 때도 있고 3.7을 써야 할 때도 있다. 이렇게 되면 다른 패키지에서 버전 충돌이 발생하게 된다.

이를 방지하기 위해 버전의 호환성을 위해 가상환경 별로 독립적인 라이브러리(library)의 버전 관리가 가능하기 때문에 아나콘다를 주로 많이 사용한다. 그렇다면 왜 다른 버전 쓰는 것인지 간략하게 설명한다.

주관적으로 보았을 때 라이브러리의 버전의 안전성이 가장 큰 원인이라고 생각된다.

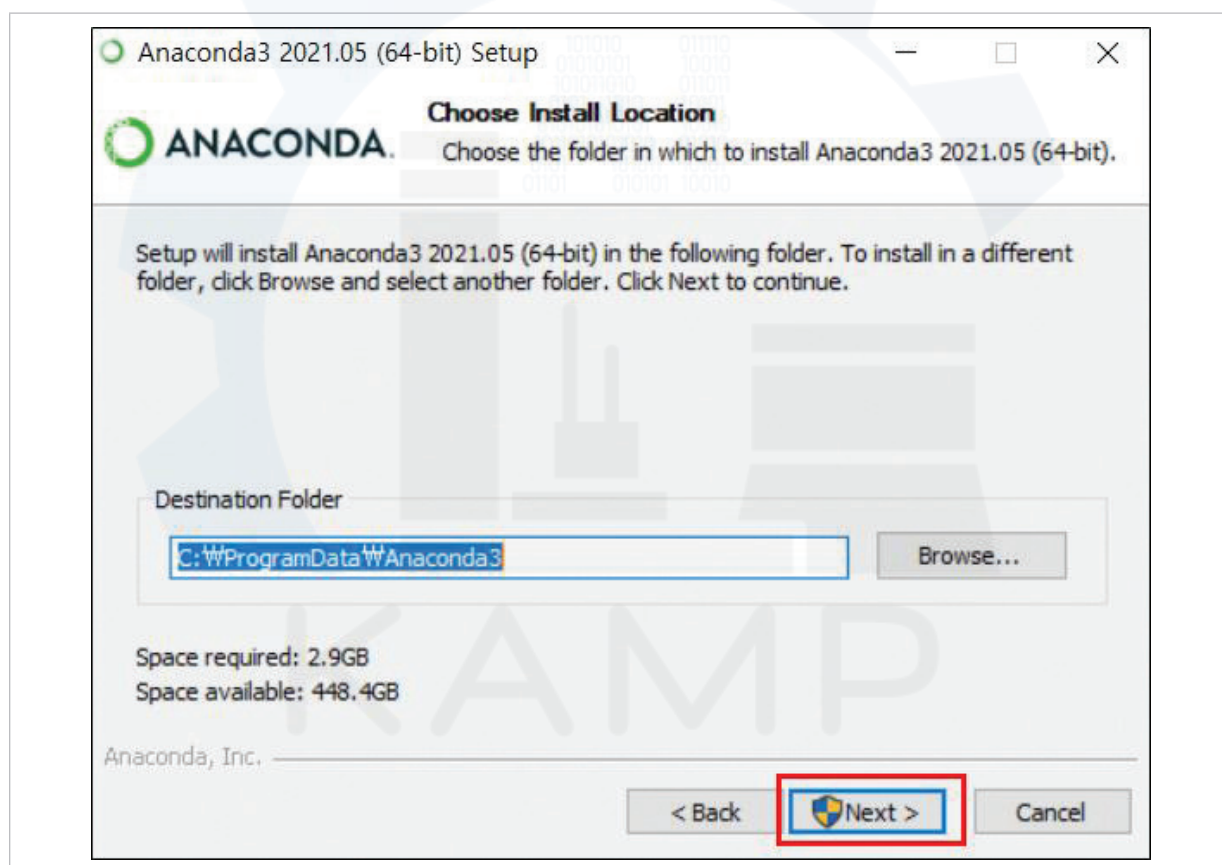
보통은 케라스(Keras)나, 텐서플로(TensorFlow)가 허용하는 버전들이 있다. 그리고 안정적이라고 판단하는 버전들이 대부분 하위 버전인 경우가 많다. 텐서플로(TensorFlow) 같은 경우 1버전과 2버전의 코드까지도 다르다. 코드의 알고리즘적인 부분이나 사용되는 원리나 구조는 같지만 코드가 다르다. 그런 경우 버전 1에서는 버전 2가 돌아가지 않는 상황이 발생한다. 그러므로 가상환경으로 각각의 텐서플로(TensorFlow) 버전을 분리해두고 상황에 따라 버전 1과 버전 2를 불러와서 사용한다.

※ 안되는 이유

설치 혹은 실행시 알 수 없는 에러(검색 해도 없는)가 발생하는 경우는 아래와 같이 예측해 볼 수 있다.

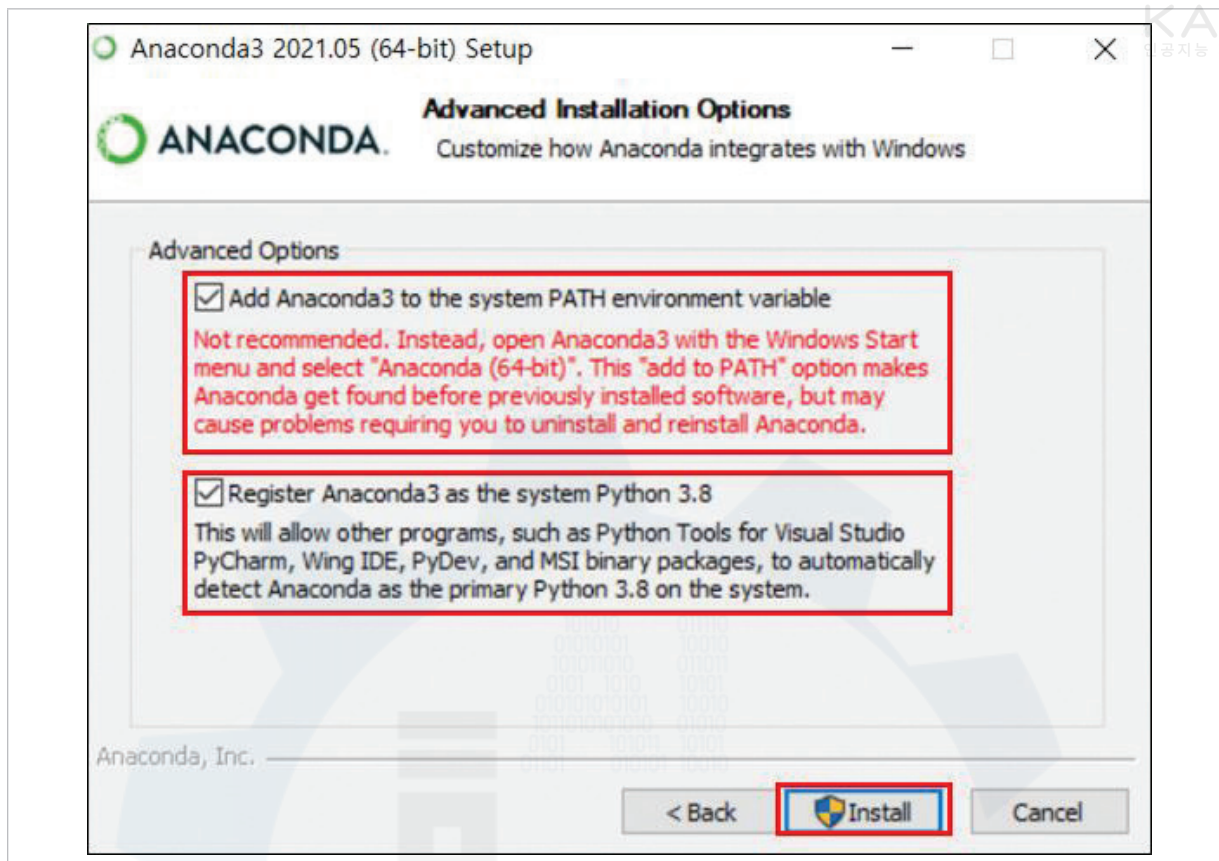
• 아나콘다 패스 설정 오류

- 아래의 경로를 최대한 변경하지 않는 것이 좋다. 모든 모듈은 저 베이스를 가본으로 하므로 특이한 사항이 아니라면 저 경로 그대로 진행하도록 한다.



만일 경로를 다르게 지정했다면 보통은 잘 진행된다. 하지만 특이 케이스에는 모듈을 불러올 때 에러가 발생할 수 있다는점 인지하고 설치하기 바란다. 이 이슈가 발생한다면 아나콘다를 삭제하고 재설치 작업을 진행하기를 추천한다.

- Add Anaconda3 to the system PATH environment variable 미 체크 경우



- 패키지끼리의 버전 충돌 혹은 호환되지 않는 각각의 패키지

- 케라스(Keras)라던지, 텐서플로(TensorFlow)처럼 타 패키지에 영향을 많이 받는 모듈은 최대한 가상환경을 생성하고 base보다는 가상환경을 생성해서 설치하는 것이 좋다.

이유는 아래와 같다.

- conda의 경우는 아나콘다에서 지원하는 혹은 사용 가능한 패키지만 관리한다.

- pip 같은 경우는 python에서 사용 가능하다고 한 것만 관리하는 패키지이다.

여기서 주의할 점은 간혹 같은 패키지이지만 설치 방법이 다른 경우가 있고 호환이 안 될 수도 있다는 점이다.

부득이하게 conda install 안될 때가 있다. 이렇게 설치가 안 된다면 pip로 시도해보거나 그래도 동작하지 않는다면

아래와 같이 진행하면 된다.

1. 콘솔 창을 연다.
2. 아나콘다 경로 아래에 bin이라는 폴더 경로까지 이동한후
3. 'pip install 설치 패키지'로 설치하면 동작하는 것을 확인 할 수 있다. 코드 1과 같이 설치하면 된다.

```
C:\Users\User>cd %
C:\>cd C:\Users\User\anaconda3\bin% pip install pandas
```

[코드 1] 콘솔 창에서 pip로 패키지 설치



4 부록 _ 데이터 품질 전처리 [실습 코드]

앞선 '2.분석 실습'에서 데이터 품질 전처리에 필요한 6가지 특성에 대하여 논의하였다.
(완전성, 유일성, 유효성, 일관성, 정확성, 무결성)

▶ 품질지수 확인용 사전 작업

```
dd = csv_read_(data_5_dir, f_lists[0])
for i in range(1, len(f_lists)):
    dd = pd.merge(dd, csv_read_(data_5_dir, f_lists[i]), how='outer')
dd = dd.drop('Index', axis=1)
dd = dd.set_index('Time')
dedicated_data = dd
```

```
data_5_dir = '5공정_180sec'
f_lists = os.listdir(data_5_dir)
f_lists = [f for f in f_lists if f.endswith('.csv')]
f_lists = f_lists[1:]
```

```
def csv_read_(data_dir, data_list):
    tmp = pd.read_csv(os.path.join(data_dir, data_list), sep=',')
    y, m, d = map(int, data_list.split('-')[-1].split('.')[:-1])
    time = tmp['Time']
    tmp['DTime'] = '-' + time
    ctime = time.apply(lambda _ : _.replace(u'오후', 'PM').replace(u'오전', 'AM'))
    ctime = ctime.apply(lambda _ : _.split('.')[0]) # 추가
    n_time = ctime.apply(lambda _ : datetime.strptime(_, "%p %I:%M:%S"))
    newtime = n_time.apply(lambda _ : _.replace(year=y, month=m, day=d))
    tmp['Time'] = newtime
    return tmp
```

	Process	Temp	Current	Date	DTime
Time					
2021-09-06 16:24:03	1	75.139142	1.610	2021-09-06	2021-09-06
2021-09-06 16:24:08	1	76.660421	1.530	2021-09-06	2021-09-06
2021-09-06 16:24:13	1	77.177660	1.701	2021-09-06	2021-09-06
2021-09-06 16:24:18	1	76.586434	1.736	2021-09-06	2021-09-06
2021-09-06 16:24:23	1	77.877104	1.748	2021-09-06	2021-09-06
...	...	...	...	...	...
2021-10-27 18:45:03	43	67.312474	1.560	2021-10-27	2021-10-27
2021-10-27 18:45:08	43	65.533664	1.472	2021-10-27	2021-10-27
2021-10-27 18:45:13	43	66.740424	1.523	2021-10-27	2021-10-27
2021-10-27 18:45:18	43	68.045185	1.483	2021-10-27	2021-10-27
2021-10-27 18:45:23	43	68.472490	1.511	2021-10-27	2021-10-27

[그림 1] 병합된 데이터 셋

```
data_columns = ['Temp', 'Current']
```

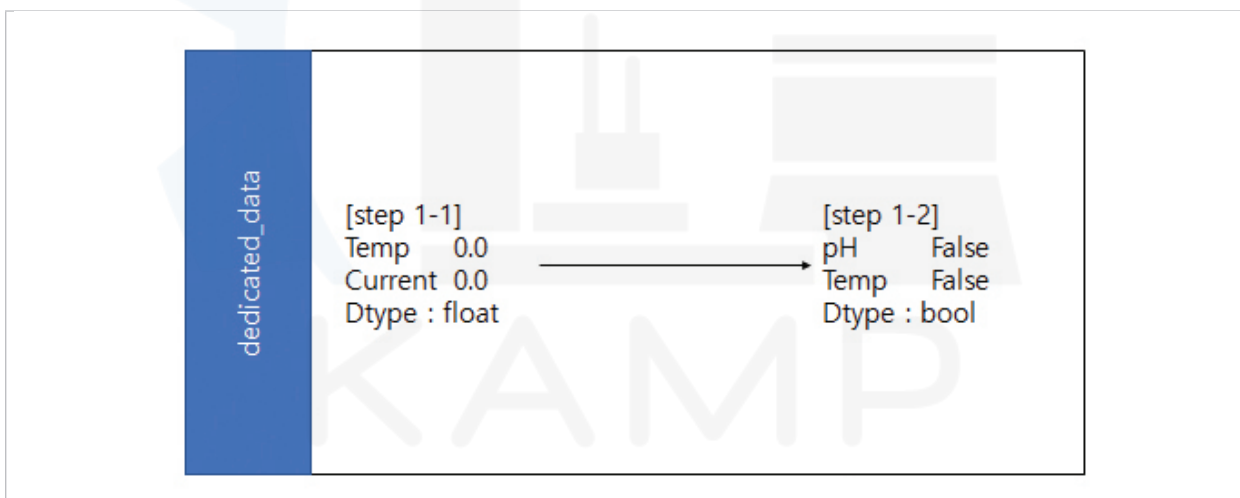
▶ 완전성 품질 지수 = (1-(결측치/전체 데이터 수)) * 100

```
perc = 30
for df_name in list(data_columns):
    print(f'DataFrame name: {df_name}')
    df = dedicated_data[df_name]
    print([step 1-1])
    print(round(df.isnull().sum()/len(df)*100,2))
    print([step 1-2])
    print(df.isnull().sum()/len(df)*100>perc)
    print([step 2-1])
    print(df.isnull().head())
    print([step 2-2])
    print(df.isnull().sum())
    print([step 2-3])
    cmpt_len = df.isnull().sum().sum()
    print(cmpt_len)
    print("결측치 = %d개 \n완전성 지수 : %.2f%%" %(cmpt_len,(1-cmpt_len/len(df))*100))
    print("\n\n")
```

- ① Null 값이 30% 이상인 데이터들은 데이터의 완전성이 떨어지기 때문에 열별 Null 값의 비율을 확인하여 삭제한다. 이 데이터는 30%를 넘는 열이 존재하지 않으므로 열을 제거하지 않는다.

```
print([step 1-1])
print(round(df.isnull().sum()/len(df)*100,2))
```

[코드 2] 완전성 품질지수 코드 (1)

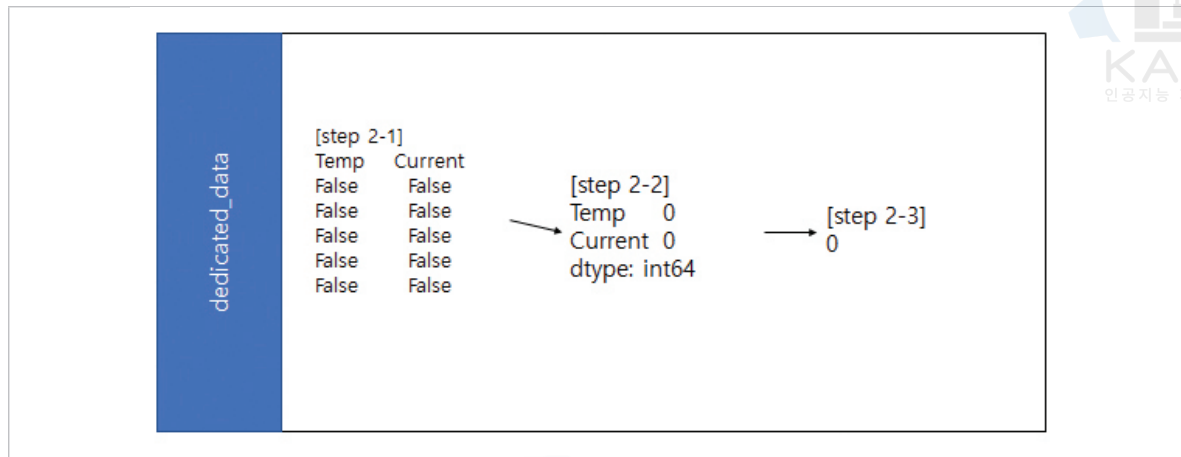


[그림 1] 분석 도구를 활용한 완전성 품질 지수 확인 (1)

- ② 데이터의 결측치를 확인하기 위하여 isnull() 함수를 사용한 뒤, sum() 함수를 이용하여 총 결측치 개수를 구한다.

```
print([step 2-1])
print(df.isnull().head())
```

[코드 3] 완전성 품질지수 코드 (2)



[그림 2] 분석 도구를 활용한 완전성 품질지수 확인 (2)

③ 구한 결측치의 개수를 이용하여 완전성 품질 지수를 구한다.

```
print("결측치 = %d개 \n완전성 지수 : %.2f%%"%(cmpt_len,(1-cmpt_len/len(df))*100))
print("\n\n")
```

[코드 4] 분석 도구를 활용한 완전성 품질 지수 구하기

▶ 유일성 품질 지수 = ((유일한 데이터 수)/전체 데이터 수) * 100

```
check_unique = dedicated_data[data_columns].value_counts().reset_index()
perc_check_unique_item_urgent_info = round(
    (len(check_unique)-len(check_unique[check_unique[0]>1]))
    /len(check_unique)*100,2)

print(f"The percentage of uniqueness for {data_columns[0]}<->{data_columns[1]} : {perc_check_unique_item_urgent_info}")
print("유일성 지수 : %.2f%%"%(perc_check_unique_item_urgent_info))
```

The percentage of uniqueness for Temp<->Current : 52.16
유일성 지수 : 52.16%

[그림 3] 유일성 품질 지수 결과

① value_counts() 함수를 이용하여 key 값들 개수를 확인한다. 개수가 큰 순서로 기본 정렬된다. reset_index() 함수는 데이터 프레임 인덱스를 재부여한다.

② key 값들 개수로 정렬했을 때, 1 이상인 행 개수 비율의 평균이 유일성 품질 지수다.

▶ 유효성 품질 지수 = (유효성 만족 데이터 수/전체 데이터 수)*100

- 유효성 지수를 만족하는 데이터만 vald_df에 저장, 최종적으로 완성된 데이터를 이용하여 유효성 품질 지수를 구한다.

```
df = dedicated_data
Current_lb = df['Current']>=0
Current_ub = df['Current']<=3
temp_lb = df['Temp']>=50
temp_ub = df['Temp']<=140
vald_df = df[Current_lb & Current_ub & temp_lb & temp_ub]
print(f'[Step 1] 데이터 범위를 벗어난 데이터 수: {len(df)-len(vald_df)}')

d0 = '2021-09-06'
d1 = '2021-10-27'
vald_df = vald_df.loc[d0:d1]
print(f'[Step 2] 수집된 날짜를 벗어나는 데이터 수: {len(df)-len(vald_df)}')

vald_df['DTime'] = vald_df['DTime'].apply(lambda x: isinstance(datetime, datetime))
vald_df['Current'] = vald_df['Current'].apply(lambda x: isinstance(x, float))
vald_df['Temp'] = vald_df['Temp'].apply(lambda x: isinstance(x, float))
print(f'[Step 3] 데이터 형식을 벗어나는 데이터 수: {len(df)-len(vald_df)}')

vald_df[
    (vald_df['Current'] == True)
    & (vald_df['Temp'] == True)
]
vald_len = len(vald_df)
item_vald = vald_len / len(df) * 100
print(f"유효성 지수 : %.2f%%" % (item_vald))

[Step 1] 데이터 범위를 벗어난 데이터 수 : 0
[Step 2] 수집된 날짜를 벗어나는 데이터 수 : 0
[Step 3] 데이터 형식을 벗어나는 데이터 수 : 0
유효성 지수 : 100.00%
```

[그림 4] 유효성 품질 지수 결과

① 데이터가 유효범위 내에 들어가 있는가?

- 데이터셋에 정의된 수집 범위를 이용하여 조건(유효범위)을 모두 충족하는 데이터들을 vald_df에 저장한다.

```
Current_lb = df['Current']>=0
Current_ub = df['Current']<=3
temp_lb = df['Temp']>=50
temp_ub = df['Temp']<=140
vald_df = df[Current_lb & Current_ub & temp_lb & temp_ub]
print(f'[Step 1] 데이터 범위를 벗어난 데이터 수: {len(df)-len(vald_df)}')
```

[코드 8] 유효성 품질지수 확인 코드 (1)

② 수집된 날짜 안에 들어가 있는가?

- 데이터셋 중 날짜 데이터를 포함하는 데이터셋 만 확인한다.
- 2021.09.06 ~ 2021.10.27에 수집된 데이터이기 때문에, 데이터의 날짜가 이 기간을 벗어나지 않았는지 확인한다.

```
d0 = '2021-09-06'
d1 = '2021-10-27'
vald_df = vald_df.loc[d0:d1]
print(f'[Step 2] 수집된 날짜를 벗어나는 데이터 수: {len(df)-len(vald_df)}')
```

[코드 9] 유효성 품질 지수 확인 코드 (2)

③ 데이터가 형식에 맞는가?

- 데이터 열은 다음 형식이어야 한다.

dedicated_data	
Temp	float
Current	float

- 'isinstance()' 함수를 사용하여 데이터 형식을 만족하는지 확인한다.

```
vald_df['DTime']=vald_df['DTime'].apply(lambda x:isinstance(datetime.datetime))
vald_df['Current']=vald_df['Current'].apply(lambda x:isinstance(x,float))
vald_df['Temp']=vald_df['Temp'].apply(lambda x:isinstance(x,float))
print(f'[Step 3] 데이터 형식을 벗어나는 데이터 수: {len(df)-len(vald_df)}')
```

[코드 10] 유효성 품질 지수 확인 코드 (3)

④ 유효성 품질지수 값은 다음과 같다.

```
vald_df[
    (vald_df['Current']==True)
    &(vald_df['Temp']==True)
]
vald_len =len(vald_df)
item_vald =vald_len/len(df)*100
print("유효성 지수 : %.2f%%"%(item_vald))
```

[코드 11] 유효성 품질 지수 확인 코드 (4)

⑤ 최종 유효성 품질 지수는 모든 데이터 유효성 품질지수의 평균이다.

```
print("유효성 지수 : %.2f%%"%(item_vald))
```

[코드 12] 유효성 품질 지수 확인하기

▶ 일관성 품질 지수 = (일관성 만족 데이터수/전체 데이터 수)*100

- 모든 열값은 독립적이므로 정확성 지수를 확인하지 않음

▶ 정확성 품질 지수 = (1-(정확성 위배 데이터 수/전체 데이터 수))*100

- 모든 열값은 독립적이므로 정확성 지수를 확인하지 않음

▶ 무결성 품질지수 = (1-(유일성, 유효성, 일관성 지수중 100%가 아닌 지수 개수 / 3))*100

- dedicated_data 데이터는 일관성을 제외한 두 가지 지수중 유일성 지수가 100%를 만족하지 못하므로 (99.47%) 무결성 품질 지수는 50%이다.


```
Integrity_result = f'{(1-(1/2))*100}%'
print(f'무결성 품질 지수: 유일성 {perc_check_unique_item_urgent_info}%, 유효성 {item_vald}%, 일관성 배제 => {Integrity_result}')
```

[코드 13] 무결성 품질 지수 확인하기

무결성 품질 지수: 유일성 52.16%, 유효성 100.0%, 일관성 배제 => 50.0%

[그림 5] 무결성 품질 지수 결과

▶ 가중치 지수 = 품질 지수 * 가중치

▶ 데이터 품질 지수 = ∑ 가중치 지수

구분	품질지수	가중치	가중치 지수	오류율
완정성 (누락)	100%	60%	60%	0%
유일성 (중복)	52.16%	10%	5.216%	0.53%
유효성 (유효)	100%	20%	20%	0%
일관성 (표현)	-	-	-	-
정확성 (실제)	-	-	-	-
무결성	50%	10%	5%	50%
품질지수		100%	90.216%	9.784%

「장비이상 조기탐지 AI 데이터셋」 분석실습 가이드북



KAMP